

EdgeJS QuickJS WASIX

A White Paper and Development Book

Sonia Sathbh Kolasinska in collaboration with Christoph Herzog, Wasmer

May 7, 2026

Abstract

This white paper organizes the QuickJS WASIX plan corpus for EdgeJS into a single book-length technical record. It preserves the source notes while arranging them by program intent, implementation chronology, cleanup work, framework compatibility, and Wasmer deployment. Together, the chapters describe the design of an embedded QuickJS-backed N-API provider, the native and WASIX bootstrap path, HTTP serving under Wasmer, framework adapter strategy, runtime containment decisions, and the active compatibility gaps that remain.

Contents

Abstract	i
I Program Definition	1
1 task	3
2 QuickJS WASM Implementation Plan	4
II Development Narrative	10
3 Edge QuickJS development index	12
4 QuickJS N-API EdgeJS merge analysis	15
5 QuickJS N-API EdgeJS execution and bootstrap troubleshooting	23
6 Edge QuickJS REPL TTY troubleshooting	29
7 Edge QuickJS REPL promise hooks and microtasks	36
8 Edge QuickJS WASIX Wasmer run enablement	40
9 Edge QuickJS web app enablement for Astro, Vite, and Next.js	45
10 Edge QuickJS framework standalone builds	55
11 Edge QuickJS runtime change containment rollback	61
12 Node compatibility test failure analysis	64
III Runtime Cleanup And Containment Subtasks	70
13 Shared runtime rollback	72
14 Native inspector fallback	73
15 Intl fallback module	75
16 Compile-time trace diagnostics	77
17 Verification and remaining cleanup	79
IV Troubleshooting Registry	80
18 Edge QuickJS framework troubleshooting	82

V Astro SSR Compatibility Investigations	87
19 Astro SSR: es-module-lexer WebAssembly Import	89
20 Astro SSR: depd CallSite Method Compatibility	91
21 Astro SSR: CommonJS Re-Export Named Exports	94
22 Astro SSR: Missing Intl Global	97
23 Astro SSR: Listen EPERM On Localhost	99
24 Astro SSR: Floating UI Utils DOM Subpath	101
25 Astro SSR: React Remove Scroll Bar Constants Subpath	103
26 Astro SSR: Zustand Ind Create Export	105
27 Astro SSR: Zustand ESM Default Export	108
28 Astro SSR: Use Gesture Controller Export	110
29 Astro SSR: Route Stack Overflow	112
30 Astro SSR: WASIX pnpm Symlink Resolution	114
31 Astro SSR: Lucide React ChevronDown Export	116
32 Astro SSR: pnpm Deploy Externalized Runtime Links	118
VI Vite App Standalone Investigation	121
33 Vite App Standalone Build Findings	123
VII Next.js Standalone Investigations	125
34 Next App Standalone: require("v8") / Serdes Findings	127
35 Next App Standalone: require("inspector") Stub	130
36 Next App Standalone: Route Request Stack Exhaustion	132
VIII Wasmer Deployment And WASIX Packaging	135
37 Wasmer Deploy: pnpm Directory Symlinks In WEBC Package	137
38 Wasmer Deploy: QuickJS WASIX N-API Import Module Mismatch	141
39 Wasmer Deploy: CI safe-mode missing QuickJS artifact	143

Part I

Program Definition

The opening chapters capture the original assignment and the architectural direction: embed QuickJS inside the WASIX guest, preserve EdgeJS's N-API boundary, and produce a Wasmer package that can run without host-provided N-API imports.

Chapter 1

task

Source note: task.md

this repo contains edgejs, a node.js alternative, driven through napi bindings, and wasmer / webassembly integration.

the current setup mainly targets using v8 as the engine, compiling edgejs itself to webassembly, and then using host napi bindings through WASM imports.

We want an alternative way to run edgejs in webassembly with the quickjs JS engine, but with quickjs compiled directly to webassembly instead of going through the napi host bridge.

do some research, and come up with a plan how to implement this. notes: * it might make sense to look for existing forks of quickjs that can already compile to webassembly. * use `nix run github:wasix-org/wasix#wasixcc` for a working wasixcc compiler * the end goal is to build a `quickjs-wasm/wasmer.toml` package definition that contains a wasm binary for edgejs, that uses quickjs directly from wasm and can be used to run code

be token efficient in your research and internal reasoning

come up with a high quality step by step plan for how to implement this, and write it to `./plan.md`

Chapter 2

QuickJS WASM Implementation Plan

Source note: `plan.md`

2.1 QuickJS WASM Implementation Plan

2.1.1 Goal

Build a `quickjs-wasm/wasmer.toml` package that runs `Edge.js` as a WASIX WebAssembly binary with QuickJS compiled into the same guest module. This should avoid the current host-provided N-API bridge used by `EDGE_NAPI_PROVIDER=imports` and instead provide N-API from inside the `wasm` binary.

The important design point is that `Edge.js` is already organized around an engine boundary: `src/` uses N-API and must not depend on V8 directly. The QuickJS path should preserve that boundary by adding a new N-API provider rather than rewriting the runtime kernel.

2.1.2 Research Summary

- The current native path uses `EDGE_NAPI_PROVIDER=bundled-v8` and links `napi/v8`.
- The current WASIX path uses `EDGE_NAPI_PROVIDER=imports`, builds `build-wasix/edgejs.wasm`, and leaves N-API symbols unresolved for the Wasmer host to provide.
- `RunWithFreshEnv()` creates an environment through `unofficial_napi_create_env()`, then `RunScriptWithGlobals()` installs process, timers, module loading, primordials, builtins, and the event loop.
- Therefore the hard part is not compiling QuickJS to `wasm`. The hard part is implementing enough Node-API plus `unofficial_napi` over QuickJS for the existing Edge bootstrap and internal bindings.
- Existing QuickJS WASI projects are useful references, especially QuickJS-NG/WASI and `vercel-labs/quickjs-wasi`, but they are not drop-in Node-compatible runtimes.
- `quickjs-emscrip`ten is mostly a JS-host embedding library, not a good WASIX executable backend.
- Javy is useful as a QuickJS/WASI compilation reference, but it compiles applications to `wasm` rather than providing a reusable `Edge.js` runtime.
- StarlingMonkey is better aligned with WinterCG/Web APIs than this QuickJS requirement and would bypass the existing N-API architecture.

2.1.3 Chosen Direction

Add a new provider:

`EDGE_NAPI_PROVIDER=quickjs`

with implementation under:

`napi/quickjs/`

This provider should:

- compile QuickJS or QuickJS-NG as C/C++ sources into the Edge WASIX binary;
- implement the public `node_api.h` functions used by Edge;
- implement the private `unofficial_napi.h` hooks used by Edge bootstrap, modules, workers, diagnostics, and testing;
- expose a normal edge executable target with no imported N-API symbols;
- produce `quickjs-wasm/wasmer.toml` pointing at the resulting `wasm` binary.

Prefer QuickJS-NG for the first prototype because it has active WASI work, CMake-based integration, and references for reactor/event-loop APIs. Keep the provider boundary narrow enough that switching to upstream QuickJS later remains possible.

2.1.4 Implementation Phases

2.1.4.1 1. Establish the Build Scaffold

1. Add `quickjs-wasm/` with:
 - `wasmer.toml`;
 - a build script, likely `quickjs-wasm/build.sh`;
 - a short README with the supported smoke commands.
2. Mirror the existing WASIX build flow from `wasix/build-wasix.sh`, but use a separate build directory such as `build-quickjs-wasix`.
3. Run the build with `wasixcc` available from:

```
nix run github:wasix-org/wasix#wasixcc
```

or an equivalent shell where `wasixcc`, `wasixcc++`, `wasixar`, and `wasixranlib` are on `PATH`.

4. Extend `CMakeLists.txt`:
 - allow `EDGE_NAPI_PROVIDER=quickjs`;
 - add `napi/quickjs` as a subdirectory for that provider;
 - link `edge_node_api` and `edge_runtime` against `napi_quickjs`;
 - make the same lifecycle hook paths currently gated by `EDGE_BUNDLED_NAPI_V8` available to QuickJS, preferably through a neutral compile definition such as `EDGE_EMBEDDED_NAPI_PROVIDER`;
 - do not enable `EDGE_ALLOW_UNDEFINED_IMPORTS` for this provider except for unrelated WASIX system imports.
5. Ensure `napi/quickjs` CMake source lists only checked-in files before the first configure/build attempt.
6. Add a link check that fails if the final QuickJS wasm still imports N-API symbols.

Deliverable: a wasm binary that links QuickJS into the guest and starts, even if it only prints a provider initialization error.

2.1.4.2 2. Build a Minimal QuickJS Provider

1. Add `napi/quickjs` with:
 - `CMakeLists.txt`;
 - a CMake `FetchContent` import for QuickJS-NG from a pinned release archive with `URL_HASH SHA256`;
 - `napi_quickjs_env.*`;
 - `napi_quickjs_values.*`;
 - `napi_quickjs_unofficial.*`.
2. Implement environment creation and teardown:
 - `unofficial_napi_create_env`;
 - `unofficial_napi_create_env_with_options`;
 - `unofficial_napi_release_env`;
 - `unofficial_napi_release_env_with_loop`;
 - cleanup and destroy callbacks.
3. Map `napi_env`, `napi_value`, and `napi_ref` to QuickJS runtime/context, `JSValue`, and explicit reference records.
 - Placeholder C++ value graphs are acceptable only for link bring-up; replace them with QuickJS `JSValue` ownership before relying on GC or runtime correctness.

4. Implement the primitive value APIs needed by bootstrap:
 - undefined/null/boolean/number/string creation;
 - type checks and conversions;
 - object creation;
 - property get/set/has/delete;
 - function creation and calls;
 - error creation and pending exception state.
5. Add a tiny provider-only C++ test that creates an env, evaluates `1 + 1`, and tears down without Edge bootstrap.

Deliverable: the provider can create a QuickJS context and evaluate simple JS through N-API-like calls.

2.1.4.3 3. Run the Smallest Edge Bootstrap

1. Start with `EdgeRunScriptSource()` rather than full CLI script loading.
2. Support enough N-API for:
 - `console`;
 - process object installation;
 - `internalBinding`;
 - builtin execution through `EdgeExecuteBuiltin`;
 - CommonJS wrapper calls.
3. Implement `unofficial_napi_process_microtasks` with QuickJS job draining.
4. Stub non-critical V8-specific diagnostics with conservative values where Edge can continue:
 - heap statistics;
 - CPU/heap profiler hooks;
 - V8 version-like metadata.
5. Do not implement workers, ESM, snapshots, inspector, or profiling in this phase unless bootstrap requires a narrow stub.

Deliverable:

```
edge -e "console.log('hello from quickjs')"
```

runs inside the QuickJS-backed WASIX binary.

2.1.4.4 4. Expand N-API by Runtime Surface

Implement APIs in the order Edge is likely to hit them:

1. Core values:
 - arrays;
 - symbols;
 - property descriptors;
 - strict equality;
 - instance checks;
 - private data / wrap / unwrap.
2. Functions and callbacks:
 - callback info;
 - `this`;
 - constructor calls;
 - thrown exceptions;
 - fatal error hooks.
3. References and lifecycle:
 - strong and weak references;
 - finalizers;

- env cleanup hooks;
 - external values.
4. Buffers and binary data:
 - ArrayBuffer;
 - TypedArray;
 - DataView;
 - Buffer compatibility;
 - external backing-store ownership rules.
 5. Promises and async:
 - promise creation;
 - promise details;
 - handled markers;
 - async work hooks needed by timers and libuv.
 6. Module support:
 - CommonJS first;
 - ESM/source text module support once enough `ModuleWrap` behavior exists;
 - CommonJS/ESM facades for Node compatibility, because Node's V8 path already has mature loader integration while QuickJS must expose equivalent V8-shaped `unofficial_napi` hooks;
 - synthetic modules and cached data can initially be unsupported unless tests or bootstrap require them.

Use compile errors, missing-symbol reports, and failing smoke tests to maintain a tracked N-API gap list. Avoid implementing broad APIs before a caller needs them.

Deliverable: a checked-in compatibility matrix for implemented, stubbed, and unsupported N-API / `unofficial_napi` calls.

2.1.4.5 5. Package as quickjs-wasm

1. Create `quickjs-wasm/wasmer.toml` similar to the root `wasmer.toml`, but point at:

```
../build-quickjs-wasix/edgejs.wasm
```

or copy the artifact into `quickjs-wasm/edgejs.wasm` during packaging.

2. Mount certificates as the current package does:

```
[fs]
"/usr/local/ssl" = "../ssl-certs"
```

3. Decide whether JS builtins are:

- compiled into `builtin_catalog.cc` only; or
- mounted from `lib/` during development.

Prefer compiled builtins for distribution and optional mounts for debugging.

4. Add package smoke commands:

```
wasmer run quickjs-wasm/wasmer.toml -- -e "console.log(1 + 1)"
wasmer run quickjs-wasm/wasmer.toml -- examples/hello.js
```

Deliverable: `quickjs-wasm/wasmer.toml` can run the QuickJS-backed Edge binary through Wasmer.

2.1.4.6 6. Bring Up Runtime Features Incrementally

After basic CLI execution works, add features in this order:

1. Filesystem-backed `require()` and path resolution.

2. Timers and event-loop checkpoints.
3. Buffer and encoding paths.
4. `fs`, `path`, `os`, and process APIs.
5. HTTP parser and minimal server/client networking.
6. Crypto and TLS.
7. Workers.
8. ESM and top-level await.
9. Diagnostics, profiling, heap snapshots, and inspector-like APIs.

Each feature should land with one end-to-end WASIX smoke test and the smallest provider API expansion needed to support it.

2.1.4.7 7. Optimize Only After Correctness

Do not start with snapshots or bytecode caching. First make source execution correct and observable.

Then evaluate:

- QuickJS bytecode for builtins;
- precompiled builtin catalog generation;
- startup snapshots based on QuickJS-NG/WASI references;
- binary size reductions;
- `wasm-opt --emit-exnref` and the same exception mode used by the current WASIX build.

Snapshots are a later optimization because they interact with native callbacks, libuv state, file descriptors, promises, and embedder data.

2.1.5 Testing Strategy

Use a staged gate:

1. Provider unit tests:
 - values;
 - functions;
 - exceptions;
 - references;
 - `ArrayBuffer/Buffer`;
 - promises and microtasks.
2. Native Edge runner tests that already call `EdgeRunScriptSource()`.
3. WASIX smoke tests using the QuickJS package.
4. Selected Node compatibility tests for the feature being brought up.
5. Benchmark only after the API surface is stable enough to compare.

Useful early smoke cases:

```
console.log("ok")
console.log(process.argv.length)
setTimeout(() => console.log("timer"), 0)
const fs = require("fs"); console.log(typeof fs.readFileSync)
```

2.1.6 Main Risks

- N-API compatibility depth is the dominant risk.
- QuickJS has different GC, job queue, stack, exception, and module semantics from V8.
- Some current `unofficial_napi` APIs are V8-shaped and should be stubbed, redesigned, or isolated for QuickJS instead of copied literally.
- Workers and structured clone are likely to be expensive because they require cross-context value transfer semantics.

- Full ESM support requires a deeper QuickJS module-loader integration than simple CommonJS execution. QuickJS itself does not lack CommonJS more than V8 does; Node implements CJS/ESM around the engine. The QuickJS risk is implementing enough `ModuleWrap`, package resolution, dynamic import, and CJS facade behavior so Node's JS loaders can drive the same semantics.
- WASIX threading, atomics, and exceptions must stay aligned with the existing `wasix/build-wasix.sh` flags.

2.1.7 Success Criteria

The first complete version is successful when:

- `quickjs-wasm/wasmer.toml` exists and points at a QuickJS-backed wasm binary;
- the binary has no imported N-API symbols;
- `edgejs -e "console.log('hello')"` works through Wasmer;
- simple file execution works;
- the implemented N-API surface is documented;
- unsupported runtime features fail clearly rather than crashing.

Part II

Development Narrative

These chapters are the chronological implementation record, from branch analysis and native bootstrap through promise hooks, WASIX HTTP serving, framework adapters, standalone builds, runtime containment, and Node compatibility test analysis.

Chapter 3

Edge QuickJS development index

Source note: `development/index.md`

3.1 Edge QuickJS development index

This is the entry point for the QuickJS WASIX development notes. The numbered phase files record chronological milestones; the troubleshooting notes capture framework-specific investigations that should be planned before code changes.

3.1.1 Start Here

- Use the phase index below to recover the development narrative.
- Use [troubleshooting/index.md](#) for active and historical Astro SSR, Vite app, and Next app investigations.
- For any new framework troubleshooting issue, create an issue-specific plan in the matching troubleshooting subdirectory before changing code.

3.1.2 Development Phases

3.1.2.1 001_merge_analysis.md: QuickJS N-API EdgeJS merge analysis

Why: establish what the other branch had already solved, what belonged in our cleaner QuickJS N-API implementation, and what EdgeJS integration pieces needed to be ported.

What was done: compared the two EdgeJS trees and N-API submodules, identified provider-selection plumbing, WASIX package scaffolding, runtime hooks, symbol ownership concerns, and the gaps in our QuickJS provider before integration.

3.1.2.2 002_native_bootstrap_contextify.md: QuickJS N-API EdgeJS execution and bootstrap troubleshooting

Why: get the native QuickJS-backed Edge CLI far enough through Node bootstrap to run `quickjs-wasm/echo-server.js`.

What was done: added bootstrap and contextify diagnostics, fixed `ContextifyScript` by defining it as a N-API class, installed Node-required well-known symbols such as dispose symbols, and verified the HTTP echo server path worked in the native QuickJS CLI while recording remaining teardown issues.

3.1.2.3 003_repl_tty_readline.md: Edge QuickJS REPL TTY troubleshooting

Why: understand why the QuickJS CLI REPL showed a prompt but then ignored typed input.

What was done: traced TTY reads through libuv, native Edge stream wrappers, and JavaScript stream/readline code; confirmed bytes reached JS; found the REPL was paused during persistent history initialization; documented `EDGE_TRACE_TTY` and the temporary `NODE_REPL_HISTORY=""` workaround.

3.1.2.4 004_promise_hooks_microtasks.md: Edge QuickJS REPL promise hooks and microtasks

Why: fix the missing async continuation that kept REPL history initialization from completing under QuickJS.

What was done: compared the other promise-hook approach, added QuickJS promise hook registration in the N-API layer, implemented real QuickJS microtask/job enqueueing and draining, patched vendored

QuickJS to emit before/after promise hook events around promise reaction jobs, and verified the REPL worked without disabling history.

3.1.2.5 **005_wasix_wasmer_http.md: Edge QuickJS WASIX Wasmer run enablement**

Why: move from native CLI success to a WASIX package that boots under Wasmer and can run an HTTP server.

What was done: documented the WASIX build shape and Makefile helpers, enabled QuickJS atomics under WASIX when wasm atomics are available, diagnosed the HTTP server hang with `EDGE_TRACE_NET`, fixed stream-base lookup for TCP/Pipe/TTY wrappers so the HTTP parser attaches to the correct listener, and verified `wasmer run` can serve the echo server.

3.1.2.6 **006_framework_app_adapters.md: Edge QuickJS web app enablement for Astro, Vite, and Next.js**

Why: prove the published Edge QuickJS package can host real framework outputs, not only small hand-written scripts.

What was done: captured the published package shape `sadhbh-c0d3/edgejs-quickjs@0.0.1`, documented static HTTP adapter patterns for Astro and Vite, described the Next.js App Router adapter, dynamic shell generation, table route fix, `.dist` staging, verification commands, and the remaining boundaries between static serving and full framework SSR.

3.1.2.7 **007_framework_standalone_builds.md: Edge QuickJS framework standalone builds**

Why: move Astro, Vite, and Next.js from app-owned router glue toward framework-standard standalone build outputs.

What was done: documented Astro's standalone Node adapter output, the `vite-plugin-standalone` path for Vite, and Next.js official standalone output; reduced the QuickJS Next standalone failure to `require("v8")` and an empty QuickJS `internalBinding("serdes")`; confirmed the Edge V8 inspector failure was not a branch regression; and adjusted native builtin failure handling so extra builtin context is printed without replacing the original JS exception formatting. Later Astro SSR work also added a deliberately minimal `Intl.DateTimeFormat` fallback for QuickJS runtimes without a real `Intl` object, covering framework bootstrap time formatting without claiming full ECMA-402 support.

3.1.2.8 **008_runtime_change_containment_rollback.md: Edge QuickJS runtime change containment rollback**

Why: restore shared runtime directories from the upstream `wasmer-io/edgejs` tree and keep QuickJS-only compatibility changes out of `lib/`, `napi/v8/`, `napi/src/`, and `napi/include/`.

What was done: copied those directories back from the donor tree, moved required QuickJS compatibility into native/runtime code, kept the common N-API headers upstream-clean, and recorded the embedded QuickJS WASIX linkage rule that Edge targets including N-API headers must compile with `NAPI_EXTERN=`. This avoided `wasm-ld` import module mismatches for `napi_*` symbols and restored `quickjs-wasm/build.sh`.

3.1.2.9 **009_node_test_failures_analysis.md: Node compatibility test failure analysis**

Why: explain the `make test-only TEST_JOBS=4` failure clusters in the Node compatibility categories and separate shared root causes from downstream test noise.

What was done: reproduced the failing command locally, compared it with the pasted CI signatures, and grouped the failures around profiler/config mismatch, global console output, `--print` result handling, File structured clone, deprecation stack classification, unsupported inspector/profiler features, debug-output shape, and string-decoder maximum string handling.

3.1.3 Troubleshooting

3.1.3.1 [troubleshooting/index.md](#): framework troubleshooting registry

Why: keep issue-specific Astro SSR, Vite app, and Next app investigations close to the development timeline while preserving the rule that new issues get a written action plan before runtime or adapter changes.

What is tracked: Astro SSR ESM dependency compatibility, Vite standalone build shape, Astro's minimal QuickJS Intl.DateTimeFormat fallback, Next.js standalone v8 / serdes compatibility, and Wasmer/QuickJS WASIX packaging or link failures.

3.1.4 Current State

The QuickJS WASIX development path now supports:

- native QuickJS-backed Edge CLI bootstrap;
- QuickJS REPL input with history enabled;
- WASIX Edge QuickJS package startup under Wasmer;
- HTTP server request parsing and response writing under Wasmer;
- static Astro and Vite app serving through small CJS adapters;
- QuickJS CommonJS/ESM facade compatibility for real package graphs, including recursive CommonJS named export discovery and shared package/path resolution helpers;
- Next.js static App Router artifacts plus generated dynamic HTML/RSC shells;
- standard Next.js standalone build layout understood, with the remaining QuickJS runtime blocker narrowed to v8 / serdes.
- QuickJS WASIX rebuilds with the embedded provider and passes the final no-N-API-imports check in `quickjs-wasm/build.sh`.

The main unresolved runtime item remains proper QuickJS teardown: `JS_FreeRuntime` is still disabled in the N-API QuickJS env release path until GC-owned object lifetime issues are fixed.

Chapter 4

QuickJS N-API EdgeJS merge analysis

Source note: `development/001_merge_analysis.md`

4.1 QuickJS N-API EdgeJS merge analysis

Date: 2026-05-04

4.1.1 Scope

This note compares:

- Theirs EdgeJS tree: `$\sim$/src/wasmer-io/edgejs`
- Theirs N-API submodule: `$\sim$/src/wasmer-io/edgejs/napi`
- Our EdgeJS tree: `$\sim$/src/edgejs`
- Our N-API submodule: `$\sim$/src/edgejs/napi`

The theirs EdgeJS checkout is on branch `quickjs` at `c2e04af1` update `napi`. Its `napi` submodule points to `wasmerio/napi@1d818e52` (origin/`quickjs`, more `async` fixes).

Our current tree is on `napi/quickjs-integration` and its `napi` submodule is at `06207826` (Unofficial N-API impl), with the refactored QuickJS provider and the standalone QuickJS N-API test harness.

4.1.2 Executive Summary

The theirs branch has the EdgeJS-side integration we need:

- `EDGE_NAPI_PROVIDER=quickjs` in top-level CMake.
- embedded-provider compile definitions for Edge lifecycle hooks.
- WASIX build/package scaffolding under `quickjs-wasm/`.
- a `wasm` link check that rejects imported `napi_*`, `node_api_*`, and `unofficial_napi_*` symbols.
- small runtime workarounds needed to get the QuickJS-backed Edge binary further through startup and smoke tests.

The theirs `napi/quickjs` implementation is much more prototype-shaped than ours:

- one big `env` header plus three large source files;
- fetches QuickJS-NG v0.14.0 with `FetchContent`;
- patches QuickJS-NG `quickjs.c` at CMake time for promise hook behavior;
- has no local `napi/quickjs/tests` harness;
- contains several broad stubs/defaults to satisfy Edge/WASIX linkage.

Our QuickJS provider is cleaner and better tested, but it is not yet integrated as an EdgeJS provider. The right integration direction is to port the EdgeJS integration and packaging ideas from the theirs branch, while keeping our refactored provider architecture and filling any missing Edge-required N-API symbols deliberately.

4.1.3 Theirs EdgeJS Integration

4.1.3.1 CMake Provider Plumbing

In `$\sim$/src/wasmer-io/edgejs/CMakeLists.txt`, the theirs branch:

- Extends `EDGE_NAPI_PROVIDER` from `bundled-v8|imports` to `bundled-v8|imports|quickjs`.
- Adds:
 - `add_subdirectory("${PROJECT_ROOT}/napi/quickjs" ...)`
 - `target_link_libraries(edge_node_api PUBLIC napi_quickjs)`
 - `target_link_libraries(edge_runtime PUBLIC napi_quickjs)`
- Changes undefined import policy:
 - WASIX + imports keeps `EDGE_ALLOW_UNDEFINED_IMPORTS=ON`.
 - quickjs forces `EDGE_ALLOW_UNDEFINED_IMPORTS=OFF`.
- Generalizes V8-only embedded behavior to `EDGE_EMBEDDED_NAPI_PROVIDER`.
- Keeps `EDGE_BUNDLED_NAPI_V8` for the V8 provider where V8-specific code still needs it.
- Adds `EDGE_QUICKJS_OWNS_NAPI_SYMBOLS` to `edge_node_api` when provider is QuickJS.

That last point matters: theirs QuickJS provider owns many public N-API symbols directly, so `src/node_api.cc` is partially compiled out to avoid duplicate definitions.

4.1.3.2 Header Export Behavior

In `src/wasmer-io/edgejs/napi/include/js_native_api.h`, the theirs branch changes wasm symbol attributes:

- normal WASIX/imports mode still marks N-API declarations as wasm imports;
- `defined(__wasm__)` && `defined(EDGE_EMBEDDED_NAPI_PROVIDER)` marks them as exported/default-visible symbols instead.

This is necessary for an embedded QuickJS provider, because the final wasm binary should define N-API symbols itself rather than import them from Wasmer.

Our current `src/edgejs/napi/include/js_native_api.h` does not have this embedded-provider wasm export branch yet.

4.1.3.3 Edge Runtime Hooks

Two Edge runtime files switch from `EDGE_BUNDLED_NAPI_V8` to the provider-neutral `EDGE_EMBEDDED_NAPI_PROVIDER`:

- `src/wasmer-io/edgejs/src/edge_environment.cc`
- `src/wasmer-io/edgejs/src/edge_task_queue.cc`

This lets QuickJS receive the same environment attachment, cleanup, context token, and promise rejection hooks that the bundled V8 path already used.

4.1.3.4 Duplicate Symbol Avoidance

`src/wasmer-io/edgejs/src/node_api.cc` wraps many fallback N-API functions in:

```
#if !defined(EDGE_QUICKJS_OWNS_NAPI_SYMBOLS)
...
#endif
```

This avoids duplicate definitions when `napi_quickjs` provides those symbols. For our integration, this is a design choice:

- either keep this model and make our provider own all required symbols;
- or keep more of `edge_node_api` active as provider-neutral fallback glue.

I prefer the second path initially unless linkage proves it impractical. It reduces the first integration blast radius.

4.1.3.5 WASIX Package Scaffold

The theirs branch adds `$/src/wasmer-io/edgejs/quickjs-wasm/`:

- `build.sh`
- `README.md`
- `wasmer.toml`
- `echo-server.js`

`quickjs-wasm/build.sh` does useful work we should reuse:

- sanitizes host include/link environment variables before cross-build;
- calls `wasix/setup-wasix-deps.sh`;
- builds static WASIX OpenSSL if missing;
- configures CMake with:
 - `-DCMAKE_TOOLCHAIN_FILE=wasix/wasix-toolchain.cmake`
 - `-DEDGE_NAPI_PROVIDER=quickjs`
 - `-DEDGE_BUILD_CLI=ON`
 - `-DBUILD_TESTING=OFF`
- runs `wasm-opt --emit-exnref` when available;
- emits `edge.wasm` and `edgejs.wasm`;
- parses the `wasm` import section and fails if any N-API symbols remain imports.

That final link check is especially valuable and should come across almost unchanged.

4.1.3.6 Runtime/Library Workarounds Outside N-API

The theirs branch also changes these areas:

- `lib/internal/modules/esm/utils.js`
 - Uses `SafeMap` instead of `SafeWeakMap` when `process.versions.v8` is `0.0.0-node.0`.
 - This is a QuickJS `WeakRef`/GC workaround. It is useful to know about, but we should only port it with a focused repro.
- `src/edge_module_loader.cc`
 - Adds better last-N-API-error reporting when `unofficial_napi_contextify_compile_function` fails.
 - This is low-risk and worth porting.
- `src/edge_http_parser.cc`
 - Reads numeric callback return values and handles `HPE_PAUSED`.
 - Likely a correctness improvement independent of QuickJS; worth testing against HTTP parser behavior before porting.
- `src/edge_stream_base.cc`
 - Replaces encoding-aware string conversion with UTF-8 text to buffer copy and ignores `encoding_name`.
 - This is suspicious as a general change. Do not port blindly; it may have been a workaround for a missing QuickJS Buffer/encoding path.
- `wasix/src/wasix_compat.cc`
 - Adds WASIX stubs for `uv_get_free_memory`, `uv_get_total_memory`, `uv_resident_set_memory`, `uv_cpu_info`, `uv_interface_addresses`, `uv_free_interface_addresses`, and `OSL_set_max_threads`.
 - These are useful for WASIX link/runtime stability.

4.1.3.7 Docs/Plans

The theirs branch adds:

- `sim$/src/wasmer-io/edgejs/docs/quickjs.md`
- `sim$/src/wasmer-io/edgejs/plans/quickjs-wasm/plan.md`
- `sim$/src/wasmer-io/edgejs/plans/quickjs-wasm/task.md`

These capture the intended architecture: QuickJS compiled into the guest wasm binary as an embedded N-API provider, rather than using the host-provided N-API imports bridge.

4.1.4 Theirs napi/quickjs

The theirs provider lives at:

`sim$/src/wasmer-io/edgejs/napi/quickjs`

It contains only six files:

- `CMakeLists.txt`
- `patch_quickjs_ng.cmake`
- `internal/napi_quickjs_env.h`
- `napi_quickjs_env.cc`
- `napi_quickjs_values.cc`
- `napi_quickjs_unofficial.cc`

4.1.4.1 Build Model

The theirs provider fetches QuickJS-NG v0.14.0:

```
FetchContent_Declare(quickjs_ng_sources
  URL "https://github.com/quickjs-ng/quickjs/archive/refs/tags/v0.14.0.tar.gz"
  URL_HASH SHA256=928e9406add99eb8623348f2cfcd916eade9a263c60d42be79bc7aee4ee8453
)
```

It builds `quickjs_ng` from `dtoa.c`, `libregex.c`, `libunicode.c`, and `quickjs.c`, then links `napi_quickjs` against it.

This is different from our current standalone provider, which expects a QuickJS library already built from `sim$/src/edgejs/quickjs` into `build-quickjs-napi/quickjs/libqjs.a`.

4.1.4.2 QuickJS-NG Patch

`patch_quickjs_ng.cmake` mutates fetched `quickjs.c` at configure time to add a helper that recovers the promise from QuickJS promise resolving functions and to fire promise hooks around `promise_reaction_job`.

This likely made Edge async hooks or promise context behavior work sooner, but it is fragile:

- it depends on private QuickJS internals;
- it is tied to a specific QuickJS-NG source shape;
- it rewrites third-party source in the CMake build directory.

For our provider, we should treat this as a behavioral clue, not as code to copy directly. If promise hook parity requires QuickJS internals, prefer an explicit patch file or a small local QuickJS fork/change rather than CMake string replacement.

4.1.4.3 Runtime Shape

The theirs provider stores nearly all state in public structs:

- `napi_value__`
- `napi_ref__`
- `napi_handle_scope__`
- `napi_escapable_handle_scope__`
- `napi_callback_info__`

- `napi_deferred__`
- `napi_async_work__`
- `napi_env__`

It tracks `JSRuntime*`, `JSContext*`, last error, pending exception, promise hooks, cleanup hooks, refs, values, native callbacks, Edge environment hooks, and context-token callbacks directly on `napi_env__`.

This is straightforward for a prototype, but it is much less maintainable than our internal class split.

4.1.4.4 API Surface

The theirs provider implements or stubs a broad set of symbols:

- core N-API value creation, conversion, properties, references, exceptions, arrays, buffers, typed arrays, promises, dates, BigInts;
- env cleanup hooks;
- callback scopes;
- async work;
- threadsafe-function stubs/defaults;
- `napi_get_uv_event_loop` as unsupported;
- many `unofficial_napi_*` hooks used by Edge;
- contextify run/compile/cached-data helpers;
- a small set of module-wrap status/default helpers;
- stable empty/default responses for V8-only profiling/heap APIs.

This broad symbol ownership explains why the EdgeJS branch disabled parts of `src/node_api.cc` under `EDGE_QUICKJS_OWNS_NAPI_SYMBOLS`.

4.1.5 Our napi/quickjs

Our provider lives at:

`$\sim$/src/edgejs/napi/quickjs`

It is at 06207826 and has the newer refactor:

- `src/js_native_api_quickjs.cc`
- `src/unofficial_napi.cc`
- small internal classes under `src/internal/`
- standalone test runners under `tests/runners/`
- `napi/quickjs/tests/CMakeLists.txt`

4.1.5.1 Strengths

Compared with the theirs provider, ours has:

- better internal ownership boundaries;
- RAIL-ish wrappers for values, refs, scopes, callbacks, cleanup hooks, externals, and functions;
- a dedicated QuickJS N-API test suite;
- recent passing baseline of the QuickJS N-API suite;
- more complete unofficial N-API behavior in several areas:
 - env creation/release from an existing QuickJS context for tests;
 - contextify make/run/dispose/compile/cache-data;
 - source-map/error formatting helpers;
 - structured clone and serialize/deserialize via QuickJS object read/write;
 - heap/memory/hash approximations;
 - module-wrap APIs with explicit stable fallbacks.

4.1.5.2 Gaps For EdgeJS Integration

Our EdgeJS root does not yet have the theirs top-level integration:

- `$\sim$/src/edgejs/CMakeLists.txt` only accepts `bundled-v8|imports`.
- `EDGE_EMBEDDED_NAPI_PROVIDER` is not used by Edge runtime hooks yet.
- `napi/include/js_native_api.h` still marks wasm N-API symbols as imports even when an embedded provider is desired.
- `quickjs-wasm/` packaging does not exist in our root.

Our provider may also need additional public N-API symbols if we choose the theirs `EDGE_QUICKJS_OWNS_NAPI_SYMBOLS` model. Examples visible in the theirs provider but not obviously present in our current `js_native_api` surface include:

- async work APIs;
- callback scope APIs;
- threadsafe-function APIs;
- `napi_get_uv_event_loop`;
- some `node_api_*` helpers;
- finalizer/post-finalizer style glue currently supplied by Edge's `src/node_api.cc`.

We should decide symbol ownership before porting the `src/node_api.cc` guard.

4.1.6 Recommended Development Plan

4.1.6.1 1. Port The Provider Selection Plumbing

Bring the top-level CMake pieces into `$\sim$/src/edgejs`:

- add `quickjs` to `EDGE_NAPI_PROVIDER`;
- add the `napi/quickjs` subdirectory for that provider;
- link `edge_node_api` and `edge_runtime` to `napi_quickjs`;
- set `EDGE_ALLOW_UNDEFINED_IMPORTS=OFF` for `quickjs`;
- introduce `EDGE_EMBEDDED_NAPI_PROVIDER` for embedded providers.

Do this without immediately copying `EDGE_QUICKJS_OWNS_NAPI_SYMBOLS` behavior unless the link requires it.

4.1.6.2 2. Add Embedded WASM Export Semantics

Port the `js_native_api.h` export/import distinction:

- wasm + imports provider: keep N-API declarations as imports;
- wasm + embedded provider: make N-API declarations default-visible exports.

This is required for a final QuickJS wasm that has no imported N-API symbols.

4.1.6.3 3. Make Our Provider Embeddable By EdgeJS CMake

Adapt our `$\sim$/src/edgejs/napi/quickjs/CMakeLists.txt` so it can work both standalone and as an EdgeJS subdirectory:

- honor parent-provided `NAPI_INCLUDE_ROOT`;
- allow `NAPI_QUICKJS_BUILD_TESTS=OFF` for Edge builds;
- avoid hardcoding `build-quickjs-napi/quickjs/libqjs.a` in the Edge provider path;
- decide whether Edge/WASIX should use our checked-in `/quickjs` build or a vendored/fetched QuickJS-NG source.

Conservative recommendation: first use the existing `/quickjs` build approach already passing our tests, then revisit QuickJS-NG only if WASIX build support requires it.

4.1.6.4 4. Decide N-API Symbol Ownership

Before disabling Edge's fallback `src/node_api.cc` symbols, run a link attempt and list missing/duplicate symbols.

Preferred first pass:

- keep `src/node_api.cc` fallbacks where they are provider-neutral;
- let `napi_quickjs` own core engine-specific symbols;
- only add `EDGE_QUICKJS_OWN_NAPI_SYMBOLS` after our provider implements the same symbol breadth as the theirs provider.

This should avoid forcing `async/threadsafe/uv-loop` stubs into our provider before we know Edge actually needs them for smoke tests.

4.1.6.5 5. Port Runtime Hooks And Low-Risk Fixes

Port these early:

- `EDGE_EMBEDDED_NAPI_PROVIDER` in `edge_environment.cc`;
- `EDGE_EMBEDDED_NAPI_PROVIDER` in `edge_task_queue.cc`;
- better contextify compile error reporting in `edge_module_loader.cc`;
- WASIX compatibility stubs in `wasix/src/wasix_compat.cc`.

Hold back or gate these until reproduced:

- `SafeWeakMap` to `SafeMap` workaround in ESM utils;
- `edge_stream_base.cc` string-write encoding change;
- HTTP parser callback return-value change, unless tests show it is required.

4.1.6.6 6. Add The WASIX Package Scaffold

Port `quickjs-wasm/` into our root after native provider linking works.

Keep the theirs build script's `wasm` import-section check. That check should be part of the acceptance criteria for QuickJS/WASIX.

4.1.6.7 7. Verification Sequence

Suggested order:

1. `make build-napi-quickjs`
2. `make test-napi-quickjs-only`
3. Configure native Edge with `-DEDGE_NAPI_PROVIDER=quickjs` `-DEDGE_BUILD_CLI=OFF` and verify `napi_quickjs`, `edge_node_api`, and `edge_runtime` link.
4. Configure native Edge CLI if feasible and run:
 - `edge --version`
 - `edge -e "console.log('hello from quickjs')"`
5. Configure WASIX with `EDGE_NAPI_PROVIDER=quickjs`.
6. Verify final `wasm` has no imported N-API symbols.
7. Run Wasmer smoke tests:
 - `wasmer run quickjs-wasm/ -- --version`
 - `wasmer run quickjs-wasm/ -- -e "console.log(1 + 1)"`
 - builtin require smoke tests for `buffer`, `events`, `path`, and `http`.

4.1.7 Open Questions

- Should the embedded provider use our existing QuickJS source tree or switch to QuickJS-NG for WASIX?

- Do we need promise hooks deep enough to require QuickJS source changes?
- Should `edge_node_api` remain a provider-neutral fallback library, or should each embedded provider own every N-API symbol?
- Which theirs runtime workarounds are true QuickJS requirements versus temporary gaps in the prototype provider?
- What is the minimum EdgeJS smoke-test matrix for declaring the first development phase done?

4.1.8 Proposed First Implementation Slice

Start with a narrow integration branch in `$\sim$/src/edgejs`:

1. Add `EDGE_NAPI_PROVIDER=quickjs` to top-level CMake.
2. Add `EDGE_EMBEDDED_NAPI_PROVIDER` macro handling.
3. Update `napi/include/js_native_api.h` for embedded wasm exports.
4. Make `napi/quickjs` build as a subdirectory with tests disabled.
5. Attempt native link and resolve only the symbol issues that appear.
6. Keep all runtime workaround ports separate and test-driven.

This should let us preserve our cleaner provider while borrowing the theirs branch's proven EdgeJS integration shape.

Chapter 5

QuickJS N-API EdgeJS execution and bootstrap troubleshooting

Source note: `development/002_native_bootstrap_contextify.md`

5.1 QuickJS N-API EdgeJS execution and bootstrap troubleshooting

Date: 2026-05-04

5.1.1 Scope

This note records the execution and troubleshooting work done after `001_merge_analysis.md`.

Goal for this pass:

```
./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js
```

should bootstrap EdgeJS with the QuickJS N-API provider, run the user script, bind the HTTP server, and answer a request.

All our code changes were made under:

- `$(sim$)/src/edgejs`
- `$(sim$)/src/edgejs/napi`

5.1.2 Initial Symptom

Running the script appeared to fail at QuickJS runtime teardown:

Assertion failed: (`list_empty(&rt->gc_obj_list)`), function `JS_FreeRuntime`, file `quickjs.c`, line 2323.

The important finding was that this assertion was not the first/root failure. It was cleanup noise after EdgeJS had already failed during bootstrap.

We used LLDB to break at `JS_FreeRuntime` and inspect `error_out` before the teardown assertion terminated the process. The original error was:

Failed to execute `internal/bootstrap/node`: undefined[Error: Failed to execute builtin 'internal/bootstrap/node

So the user script was not running yet. EdgeJS was failing while executing `internal/bootstrap/node`.

5.1.3 Instrumentation Added

5.1.3.1 Edge Builtin/Binding Tracing

In `$(sim$)/src/edgejs/src/edge_module_loader.cc`:

- Added gated builtin compile tracing:

```
EDGE_TRACE_BUILTINS=1
```

This prints lines such as:

```
EDGE_TRACE_BUILTINS compile internal/vm
```

- Added gated `internalBinding()` tracing:

```
EDGE_TRACE_INTERNAL_BINDING=1
```

This prints requests, cache hits, resolved value type, and contextify resolver state.

- Added `DescribeAndClearPendingException()` and used it when `ExecuteBuiltinFromNative()` fails in `napi_call_function()`.

This preserves the thrown JS stack in the higher-level native error instead of returning a generic builtin execution failure.

5.1.3.2 QuickJS Contextify Compile Diagnostics

In `$\sim$/src/edgejs/napi/quickjs/src/unofficial_napi.cc`:

- Added contextify compile exception annotations:
 - `node:quickjsContextifyCompile`
 - `node:quickjsCompileResourceName`
 - `node:quickjsCompileBuiltinId`
 - `node:quickjsCompileLineOffset`
 - `node:quickjsCompileColumnOffset`
 - `node:quickjsCompileQuickJSLine`
 - `node:quickjsCompileMappedLine`
- Added optional compile tracing:

```
EDGE_TRACE_QUICKJS_CONTEXTIFY=1
```

or:

```
EDGE_TRACE_BUILTINS=1
```

When enabled, compile exceptions get a one-line summary prepended to the stack.

- Preserved the caught QuickJS exception in N-API's pending/last-exception slot before returning `napi_pending_exception`.
- Added `sourceURL` to Function-constructor compiled source so the result object can record the intended source resource name.

5.1.4 Bootstrap Failure 1: internal/vm

The first real JavaScript failure was:

```
TypeError: Cannot convert undefined or null to object
```

The chain was:

- `/lib/internal/bootstrap/node.js:303`

```
} = require('internal/process/execution');
```

- `/lib/internal/process/execution.js:41`

```
} = require('internal/vm');
```

- `/lib/internal/vm.js:13`

```
runInContext,
```

The failing code in `internal/vm.js` is:

```
const {
  runInContext,
} = ContextifyScript.prototype;
```

internalBinding('contextify') did resolve, but ContextifyScript.prototype was missing.

5.1.4.1 Root Cause

ResolveContextifyBinding() created ContextifyScript with napi_create_function(), then tried to read and modify its prototype.

Our QuickJS napi_create_function() creates a callable QuickJS C function, but it does not install a JS constructor prototype. Therefore:

```
ContextifyScript.prototype
```

was undefined, and destructuring from it triggered QuickJS JS_ToObject() on undefined.

5.1.4.2 Fix

Changed ResolveContextifyBinding() to create ContextifyScript with napi_define_class() and define prototype methods there:

- runInContext
- createCachedData

This gives Node's internal/vm the constructor shape it expects.

After this change, bootstrap moved past internal/vm.

5.1.5 Runtime Failure 2: Missing Dispose Symbols

After fixing ContextifyScript, EdgeJS reached the user script and started loading http, but failed while evaluating _http_server.

The new chain reached:

- /lib/_http_server.js:617

```
Server.prototype[SymbolAsyncDispose] = assignFunctionName(SymbolAsyncDispose, async function() {
```

The stack included:

```
at get description (native)
at call (native)
at assignFunctionName (<input>:943:61)
```

The relevant JS was:

- /lib/internal/util.js:941

```
const symbolDescription = SymbolPrototypeGetDescription(name);
```

5.1.5.1 Root Cause

The embedded QuickJS runtime did not provide Node's expected well-known symbols:

- Symbol.dispose
- Symbol.asyncDispose

Node's primordials therefore captured SymbolAsyncDispose as undefined. Later, _http_server passed that undefined into assignFunctionName(), which attempted to read Symbol.prototype.description from a non-symbol.

5.1.5.2 Fix

Added a QuickJS env initialization shim in `/napi/quickjs/src/unofficial_napi.cc`:

- `EnsureNodeWellKnownSymbols(ctx)`
- `EnsureSymbolProperty(ctx, symbol_ctor, "dispose", "Symbol.dispose")`
- `EnsureSymbolProperty(ctx, symbol_ctor, "asyncDispose", "Symbol.asyncDispose")`

This runs immediately after `JS_NewContext(rt)` in `unofficial_napi_create_env_with_options()`, before EdgeJS executes `internal/per_context/primordials`.

After this change, the http path got through `_http_server`.

5.1.6 Verification

Rebuilt the native QuickJS Edge CLI:

```
cmake --build build-edge-quickjs-cli --target edge -j4
```

Verified the target command:

```
./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js
```

Expected output:

```
quickjs edge echo listening on 3000
```

Verified a request:

```
curl -sS http://127.0.0.1:3000/
```

Response:

```
quickjs edge echo: GET /
```

Important testing note: direct server runs and local `curl` from Codex's sandbox can be misleading because the sandbox may block bind/connect operations. The successful verification was done with the command allowed to run outside the sandbox, matching the behavior expected from a normal user shell.

5.1.7 Current State

The echo server works when run normally outside the sandbox:

```
./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js
```

and responds to HTTP requests on port 3000.

The main EdgeJS-side functional fix is:

- `ContextifyScript` now uses `napi_define_class()` so `ContextifyScript.prototype.runInContext` exists for `internal/vm.js`.

The main QuickJS-side functional fix is:

- QuickJS env creation now installs Node-required `Symbol.dispose` and `Symbol.asyncDispose` when missing.

The main diagnostic improvements are:

- builtin compile tracing via `EDGE_TRACE_BUILTINS`
- internal binding tracing via `EDGE_TRACE_INTERNAL_BINDING`
- contextify compile exception annotations via `EDGE_TRACE_QUICKJS_CONTEXTIFY`
- better propagated JS stacks when native builtin execution fails

5.1.8 Things To Clean Up Or Decide

5.1.8.1 Debug Tracing

The tracing is gated behind environment variables, so it is not noisy by default. Still, before a final cleanup pass we should decide whether to keep all of these:

- `EDGE_TRACE_BUILTINS`
- `EDGE_TRACE_INTERNAL_BINDING`
- `EDGE_TRACE_QUICKJS_CONTEXTIFY`

`EDGE_TRACE_BUILTINS` and `contextify` exception annotations are useful. The broader `EDGE_TRACE_INTERNAL_BINDING` trace may be more temporary.

5.1.8.2 QuickJS Teardown Assertion

The original `JS_FreeRuntime` assertion is not the current blocker for `echo-server.js` working. It can still appear if the process exits during an earlier failure path or when testing inside the sandbox.

We explicitly deferred that cleanup/GC assertion while chasing bootstrap and script execution. It should be handled separately with a focused QuickJS lifetime/handle-scope audit, not mixed into bootstrap fixes.

5.1.8.3 N-API Test Coverage

Existing targeted QuickJS N-API tests were added/kept around:

- `napi_create_arraybuffer()` accepting `data == nullptr`
- private symbol creation accepting `NAPI_AUTO_LENGTH`
- `contextify` compile/cached-data behavior

Before considering this development slice done, rerun:

```
CMAKE_BUILD_TYPE=Debug make build-napi-quickjs
make test-napi-quickjs-only
```

and then rebuild the Edge CLI again.

5.1.9 Commands That Were Useful

Capture pre-teardown failure state:

```
lldb --batch \
-o "breakpoint set -n JS_FreeRuntime" \
-o run \
-o "frame select 3" \
-o "frame variable error_out" \
-- ./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js
```

Trace builtin compilation:

```
EDGE_TRACE_BUILTINS=1 \
./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js
```

Trace internal binding resolution:

```
EDGE_TRACE_INTERNAL_BINDING=1 \
./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js
```

Trace QuickJS `contextify` compile failures:

```
EDGE_TRACE_QUICKJS_CONTEXTIFY=1 \
./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js
```

Verify the working server:

```
./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js  
curl -sS http://127.0.0.1:3000/
```


Chapter 6

Edge QuickJS REPL TTY troubleshooting

Source note: `development/003_repl_tty_readline.md`

6.1 Edge QuickJS REPL TTY troubleshooting

6.1.1 Context

This note captures the debugging session for the QuickJS-backed Edge REPL hang. The user-visible symptom was:

```
./build-edge-quickjs-cli/edge
```

The REPL banner and prompt appeared, but typed input was not echoed, evaluated, or otherwise handled.

The goal of this investigation was to follow the TTY read path from libuv's `kqueue` polling through Edge's native TTY wrapper and into the JavaScript REPL stack, then identify where input stopped moving.

6.1.2 Short version

The TTY is reading correctly.

The typed bytes wake `kevent()`, libuv dispatches the read watcher, Edge's native TTY `OnRead()` receives the bytes, `EdgeStreamBaseOnUvRead()` forwards them to JavaScript, and `lib/internal/stream_base_commons.js:onStreamRead()` receives a valid `FastBuffer`.

The bytes do not reach `readline` because the REPL input stream has already been paused by REPL history initialization. History initialization pauses `readline` while it opens and prepares `/Users/sadhbh/.node_repl_history`.

The immediate hang is inside the async history initialization flow:

```
const hnd = await fs.promises.open(this[kHistoryPath], 'a+', 0o0600);
await hnd.close();
```

Native file handle close completes and resolves its N-API promise, but the JavaScript continuation after `await hnd.close()` does not run. The current suspect is QuickJS promise/job behavior around the `FileHandle.close()` wrapper, especially:

```
SafePromisePrototypeFinally(
  this[kHandle].close(),
  () => { this[kClosePromise] = undefined; },
)
```

So the REPL is not blocked because `kqueue` or TTY read is broken. It is blocked because REPL history paused `stdin` and an `fs` promise/finally chain never resumes the history initialization.

6.1.3 LLDB investigation

The requested breakpoint was placed at:

```
$\sim$/src/edgejs/deps/uv/src/unix/kqueue.c:293
```

When typing into the hung REPL, `kevent()` returned a real readable event:

```
nfds = 1
timeout = -1
```

```
events[0].ident = 12
events[0].filter = -1    // EVFILT_READ
events[0].flags = 1
events[0].data = 1
```

Stepping forward in `uv__io_poll()` showed:

```
fd = 12
loop->watchers[fd] = 0x...
w->fd = 12
w->pevents = 1
w->events = 1
w->cb = uv__stream_io at deps/uv/src/unix/stream.c:1189
```

This means the kernel event was real, libuv had an active watcher for the fd, and the event was dispatched into libuv's stream read machinery.

The native stack from the read looked like:

```
uv__io_poll
uv__stream_io
uv__read
edge_tty_wrap.cc::OnRead
EdgeStreamBaseOnUvRead
EdgeStreamEmitRead
DefaultOnRead
CallJsOnRead
EdgeAsyncWrapMakeCallback
```

Inside `edge_tty_wrap.cc::OnRead`, LLDB showed:

```
nread = 4
buf->base starts with "2+3\n"
buf->len = 65536
```

The important detail is `nread = 4`; the backing allocation contained stale bytes after the first four bytes, but the stream contract is to consume only `nread` bytes. This was not the source of the hang.

6.1.4 Native Edge callback result

The read callback reached `CallJsOnRead()` in:

```
$\sim$/src/edgejs/src/edge_stream_base.cc
```

The stream state was set:

```
SetStreamState(base->env, kEdgeReadBytesOrError, static_cast<int32_t>(nread));
SetStreamState(base->env, kEdgeArrayBufferOffset, static_cast<int32_t>(offset));
```

The JS callback existed and was callable. `EdgeAsyncWrapMakeCallback()` returned `napi_ok`, produced a result value, and there was no pending exception immediately after the callback.

That ruled out:

- kqueue not waking
- libuv watcher not registered
- native TTY `OnRead()` not firing
- missing `onread` callback
- immediate JS exception from `onStreamRead()`

6.1.5 JavaScript stream trace

Temporary tracing was added behind `EDGE_TRACE_TTY` in:

```
$\sim$/src/edgejs/src/edge_tty_wrap.cc
```

```

$\sim$/src/edgejs/src/edge_stream_base.cc
$\sim$/src/edgejs/lib/internal/stream_base_commons.js
$\sim$/src/edgejs/lib/internal/readline/emitKeypressEvents.js
$\sim$/src/edgejs/lib/internal/readline/interface.js
$\sim$/src/edgejs/lib/internal/streams/readable.js
$\sim$/src/edgejs/lib/internal/repl/history.js
$\sim$/src/edgejs/src/internal_binding/binding_fs.cc

```

After typing 2+3\n, the JavaScript stream trace showed:

```

EDGE_TRACE_TTY onread fd=0 nread=4 len=65536
EDGE_TRACE_TTY js stream_base onStreamRead nread= 4 destroyed= false readableLength= 0 flowing= false hasDataLi
EDGE_TRACE_TTY js stream_base push-before offset= 0 buf.length= 4 buf= 2+3
EDGE_TRACE_TTY js stream_base push-after result= false readableLength= 4 flowing= false
EDGE_TRACE_TTY js stream_base push-backpressure-readStop

```

This proves:

- `onStreamRead()` receives the input.
- The `FastBuffer` is correct and has length 4.
- The stream is not flowing.
- `stream.push(buf)` buffers the data instead of emitting it to readline.
- Because the high-water mark is zero and the stream is paused, `push()` returns false and `stream_base` calls `handle.readStop()`.

So the REPL looked locked because stdin was paused at the JS stream layer.

6.1.6 Why stdin was paused

Tracing in `lib/internal/streams/readable.js`, `lib/internal/readline/interface.js`, and `lib/internal/repl/history.js` showed the sequence:

```

Welcome to Edge.js 0.0.0-f287153 (Node.js v24.13.2).
Type ".help" for more information.
> EDGE_TRACE_TTY js repl_history initialize pause /Users/sadhbh/.node_repl_history
EDGE_TRACE_TTY js readline pause
EDGE_TRACE_TTY js repl_history open a+ begin
EDGE_TRACE_TTY js readable resume_ before flowing= false data= 1 readable= 0
EDGE_TRACE_TTY readStart fd=0 rc=0 has_ref=1 loop_alive=1
EDGE_TRACE_TTY js readable resume_ after flowing= false data= 1 readable= 0
EDGE_TRACE_TTY readStop ignored fd=0 raw_mode=true
EDGE_TRACE_TTY js repl_history open a+ done

```

The pause comes from `ReplHistory.initialize()`:

```

this[kContext].pause();
this[kInitializeHistory](onReadyCallback).catch(...);

```

That is normal Node behavior. REPL history pauses input while it initializes persistent history, and it resumes the context once history is ready.

In our QuickJS run, history initialization starts but never reaches the resume point.

6.1.7 File handle close trace

History initialization reached:

```

const hnd = await fs.promises.open(this[kHistoryPath], 'a+', 0o0600);
await hnd.close();

```

The trace printed:

```

EDGE_TRACE_TTY js repl_history open a+ begin

```

```
EDGE_TRACE_TTY js repl_history open a+ done
```

It did not print:

```
EDGE_TRACE_TTY js repl_history close a+ done
```

Native tracing around FileHandleClose showed:

```
EDGE_TRACE_TTY fs FileHandleClose start fd=13 rc=0 loop=0x...
```

```
EDGE_TRACE_TTY fs FileHandleClose after fd=13 result=0
```

```
EDGE_TRACE_TTY fs FileHandleClose finish fd=13 result=0 deferred=0x...
```

This proves:

- `fs.promises.open()` completes.
- `FileHandle.close()` enters native code.
- `uv_fs_close()` is submitted successfully.
- libuv calls the close completion callback.
- native resolves the N-API deferred promise.
- `EdgeRunCallbackScopeCheckpoint(env)` is called after resolving.

Despite that, the JS async function waiting on `await hnd.close()` does not continue.

6.1.8 VSCode LLDB trace confirmation

Running the same build from VSCode LLDB with:

```
"env": {
  "EDGE_TRACE_TTY": "1"
}
```

produced the same sequence:

```
Welcome to Edge.js 0.0.0-f287153 (Node.js v24.13.2).
```

```
Type ".help" for more information.
```

```
EDGE_TRACE_TTY readStop fd=0 rc=0 has_ref=1 loop_alive=0
```

```
EDGE_TRACE_TTY js readable resume before flowing= null data= 1 readable= 0
```

```
EDGE_TRACE_TTY js readable resume after flowing= true data= 1 readable= 0
```

```
EDGE_TRACE_TTY setRawMode fd=0 flag=true rc=0
```

```
EDGE_TRACE_TTY js readable resume before flowing= true data= 1 readable= 0
```

```
EDGE_TRACE_TTY js readable resume after flowing= true data= 1 readable= 0
```

```
> EDGE_TRACE_TTY js repl_history initialize pause /Users/sadhbh/.node_repl_history
```

```
EDGE_TRACE_TTY js readline pause
```

```
EDGE_TRACE_TTY js repl_history open a+ begin
```

```
EDGE_TRACE_TTY js readable resume_ before flowing= false data= 1 readable= 0
```

```
EDGE_TRACE_TTY readStart fd=0 rc=0 has_ref=1 loop_alive=1
```

```
EDGE_TRACE_TTY js readable resume_ after flowing= false data= 1 readable= 0
```

```
EDGE_TRACE_TTY readStop ignored fd=0 raw_mode=true
```

```
EDGE_TRACE_TTY js repl_history open a+ done
```

```
EDGE_TRACE_TTY fs FileHandleClose start fd=13 rc=0 loop=0x...
```

```
EDGE_TRACE_TTY fs FileHandleClose after fd=13 result=0
```

```
EDGE_TRACE_TTY fs FileHandleClose finish fd=13 result=0 deferred=0x...
```

The important missing line is still:

```
EDGE_TRACE_TTY js repl_history close a+ done
```

When a key was pressed after that point, the trace showed:

```
EDGE_TRACE_TTY onread fd=0 nread=1 len=65536
```

```
EDGE_TRACE_TTY js stream_base onStreamRead nread= 1 destroyed= false readableLength= 0 flowing= false hasDataLi
```

```
EDGE_TRACE_TTY js stream_base push-before offset= 0 buf.length= 1 buf= v
```

```
EDGE_TRACE_TTY js stream_base push-after result= false readableLength= 1 flowing= false
```

```
EDGE_TRACE_TTY js stream_base push-backpressure-readStop
```

EDGE_TRACE_TTY readStop ignored fd=0 raw_mode=true

This confirms the same conclusion from inside VSCode:

- TTY input still reaches native code.
- The typed byte is delivered to `onStreamRead()`.
- The stream is paused: `isPaused= true, flowing= false`.
- The byte is buffered and readline does not receive it.
- The pause comes from REPL history initialization, not from the debugger or kqueue.

6.1.9 Quick confirmation workaround

To confirm the diagnosis without fixing promise handling yet, disable persistent REPL history in the VSCode launch configuration:

```
{
  "type": "lldb",
  "request": "launch",
  "name": "Debug Edge QuickJs",
  "program": "${workspaceFolder}/build-edge-quickjs-cli/edge",
  "args": [],
  "cwd": "${workspaceFolder}",
  "env": {
    "EDGE_TRACE_TTY": "1",
    "NODE_REPL_HISTORY": ""
  }
}
```

`NODE_REPL_HISTORY=""` makes `ReplHistory.initialize()` take the disabled-history path, which avoids the `fs.promises.open() / FileHandle.close()` setup that currently wedges the REPL. If the REPL becomes interactive with that setting, it is another confirmation that the core issue is the file-handle close promise continuation, not TTY reading.

6.1.10 Current hypothesis

The current best hypothesis is a QuickJS promise/job issue, not a TTY issue.

The wrapper in `lib/internal/fs/promises.js` returns:

```
this[kClosePromise] = SafePromisePrototypeFinally(
  this[kHandle].close(),
  () => { this[kClosePromise] = undefined; },
);
```

`this[kHandle].close()` is the native N-API promise. Native resolves that promise, but the promise chain created by `SafePromisePrototypeFinally()` does not appear to settle in a way that wakes the awaiting async function.

Relevant implementation areas:

```
$\sim$/src/edgejs/napi/quickjs/src/js_native_api_quickjs.cc
$\sim$/src/edgejs/napi/quickjs/src/unofficial_napi.cc
$\sim$/src/edgejs/src/edge_runtime.cc
$\sim$/src/edgejs/src/internal_binding/binding_fs.cc
$\sim$/src/edgejs/lib/internal/per_context/primordials.js
$\sim$/src/edgejs/lib/internal/fs/promises.js
```

In particular:

- `napi_resolve_deferred()` in the QuickJS backend calls the stored promise resolve function.
- `unofficial_napi_process_microtasks()` calls `JS_ExecutePendingJob()` until the QuickJS job queue is

empty.

- `EdgeRunCallbackScopeCheckpoint()` invokes `unofficial_napi_process_microtasks()` when appropriate.
- Native file close calls `EdgeRunCallbackScopeCheckpoint()` after resolving the deferred.

The missing piece is why the promise/finally/async-await continuation does not run even after the native deferred has been resolved and a callback checkpoint is requested.

6.1.11 Temporary probe: ignoring readStop

One temporary probe changed `edge_tty_wrap.cc` so `readStop()` was ignored for fd 0 while raw mode was enabled.

That was not a real fix. It was useful because it proved native TTY reads could continue and that `OnRead()` saw typed bytes. With that probe, input still did not reach readline because JS remained paused.

This probe should not be treated as the final solution.

6.1.12 What to do next

1. Build a small reproduction around `fs.promises.open(...).then(h => h.close())` and `await h.close()` under the QuickJS Edge CLI.
2. Add targeted tracing or tests for `SafePromisePrototypeFinally()` with a native N-API promise:

```
const p = nativePromise();
await SafePromisePrototypeFinally(p, () => {});
```

3. Inspect whether `napi_resolve_deferred()` enqueues the expected QuickJS jobs and whether all jobs are executed by `unofficial_napi_process_microtasks()`.
4. Check whether QuickJS's `Promise.prototype.finally` behavior with `SafePromise` subclassing matches what Node's primordials expect.
5. Once promise continuation works, remove the temporary TTY/readline/history/fs instrumentation and remove the `readStop()` ignore probe.
6. Re-test:

```
./build-edge-quickjs-cli/edge
```

Expected behavior: the REPL prompt accepts input, echoes/evaluates commands, and remains interactive.

6.1.13 Useful commands

Build:

```
cmake --build build-edge-quickjs-cli --target edge -j4
```

Run traced REPL:

```
EDGE_TRACE_TTY=1 ./build-edge-quickjs-cli/edge
```

Run traced REPL with persistent history disabled:

```
EDGE_TRACE_TTY=1 NODE_REPL_HISTORY="" ./build-edge-quickjs-cli/edge
```

LLDB entry point:

```
lldb ./build-edge-quickjs-cli/edge
```

Breakpoint requested for the kqueue investigation:

```
breakpoint set --file $sim$/src/edgejs/deps/uv/src/unix/kqueue.c --line 293
```

Useful native breakpoints:

```
breakpoint set --file src/edge_tty_wrap.cc --name OnRead  
breakpoint set --file src/edge_stream_base.cc --name EdgeStreamBaseOnUvRead  
breakpoint set --file src/edge_stream_base.cc --name CallJsOnRead
```

6.1.14 Bottom line

The REPL is not failing because QuickJS Edge cannot read from TTY. It can.

The REPL is failing because persistent history pauses readline and then gets stuck awaiting `FileHandle.close()`. Native close resolves the promise, but the JavaScript promise/finally/await continuation does not run. The next fix should focus on QuickJS N-API promise resolution and microtask/job draining, especially with `SafePromisePrototypeFinally()`.

Chapter 7

Edge QuickJS REPL promise hooks and microtasks

Source note: `development/004_promise_hooks_microtasks.md`

7.1 Edge QuickJS REPL promise hooks and microtasks

7.1.1 Context

The QuickJS-backed Edge CLI could enter REPL mode only when persistent REPL history was disabled:

```
NODE_REPL_HISTORY="" ./build-edge-quickjs-cli/edge
```

With normal history enabled, the prompt appeared but input was stuck. TTY tracing showed that native TTY reads were working and bytes reached the JS stream layer, but `readline` stayed paused. The pause came from `lib/internal/repl/history.js`, which pauses the REPL while it initializes persistent history.

The last observed history trace before the lock was:

```
open a+ done
FileHandleClose finish ...
```

The expected next trace was:

```
close a+ done
```

So the stalled point was the async continuation after:

```
await hnd.close();
```

This made the problem look like a promise/async continuation issue rather than a TTY or libuv read issue.

7.1.2 What the colleague branch showed

The colleague branch was useful as implementation evidence, but not as a native-CLI proof. Its `quickjs-wasm/build.sh` only builds the WASIX target with:

```
-DCMAKE_TOOLCHAIN_FILE=wasix/wasix-toolchain.cmake
-DEDGE_NAPI_PROVIDER=quickjs
-DEDGE_BUILD_CLI=ON
```

It does not demonstrate that a native QuickJS Edge REPL works.

Still, two relevant ideas were present in the colleague N-API implementation:

1. Register a QuickJS runtime promise hook with `JS_SetPromiseHook(...)`.
2. Preserve and restore `continuation_preserved_embedder_data` around promise jobs.

Their QuickJS-NG CMake patch also injected `JS_PROMISE_HOOK_BEFORE` and `JS_PROMISE_HOOK_AFTER` around `promise_reaction_job()`. Our local QuickJS had promise hook types and `JS_SetPromiseHook(...)`, but ordinary promise reaction jobs were not emitting before/after hooks.

7.1.3 Root cause

Our N-API QuickJS backend stored promise hook callbacks from `unofficial_napi_set_promise_hooks()`, but it did not register a QuickJS runtime hook and did not preserve async context frames across promise jobs.

Separately, our local QuickJS did not emit `JS_PROMISE_HOOK_BEFORE` and `JS_PROMISE_HOOK_AFTER` around normal `promise_reaction_job()` execution. That means even a registered N-API hook would not see the core `.then / await` continuations that Node's async context and REPL history initialization depend on.

One extra wrinkle: QuickJS represents `async/await` continuations with `JS_CLASS_ASYNC_FUNCTION_RESOLVE / JS_CLASS_ASYNC_FUNCTION_REJECT`, not only with normal promise resolve/reject function objects. A direct copy of the colleague helper would recover promise identity for normal promise reactions, but could still miss `async-function` continuations.

7.1.4 Why this currently requires a QuickJS source change

For the current vendored QuickJS source, the fix cannot live purely in the N-API layer. The missing transition happens inside QuickJS's internal `promise_reaction_job()`. By the time an expression such as:

```
await hnd.close();
```

resumes, QuickJS is running an engine-owned queued promise job, not calling back through N-API. If `promise_reaction_job()` does not emit `JS_PROMISE_HOOK_BEFORE` and `JS_PROMISE_HOOK_AFTER`, the N-API backend has no reliable point where it can restore and then unwind `continuation_preserved_embedder_data`.

So there are three realistic paths:

1. Keep the small vendored QuickJS patch. This is what we did, and it is the direct fix for our current source tree.
2. Upgrade or switch to a QuickJS/QuickJS-NG version that already emits promise hooks around normal reaction jobs. To qualify, its `promise_reaction_job()` must call the runtime promise hook before and after invoking the reaction handler.
3. Monkey-patch Promise behavior in JS. This is theoretically possible but not recommended: it is fragile, changes global Promise behavior, and is likely to miss `async/await` or other internal promise paths.

In short: we do not inherently need a permanent fork of QuickJS, but we do need an engine that exposes before/after promise reaction hooks. Our current vendored QuickJS does not, so this repo currently needs the source patch unless we move to an upstream version with equivalent behavior.

7.1.5 Fix in our codebase

7.1.5.1 QuickJS runtime

File:

```
$\sim$/src/edgejs/quickjs/quickjs.c
```

Added helpers near the promise reaction data definitions:

- `js_promise_function_promise(...)`
- `js_promise_reaction_promise(...)`

The first helper recovers the promise associated with normal QuickJS promise resolve/reject functions.

The second helper also handles `async-function` resolve/reject objects by walking from the `async-function` resolver back to the `async function`'s public promise. This is the important difference from the colleague patch for the REPL-history case, because `await hnd.close()` resumes through the `async-function` machinery.

Then `promise_reaction_job()` now:

1. Recovers the promise identity from the fulfillment or rejection resolving function.
2. Emits `JS_PROMISE_HOOK_BEFORE` before invoking the reaction handler.
3. Emits `JS_PROMISE_HOOK_AFTER` after invoking the reaction handler.

7.1.5.2 QuickJS N-API backend

File:

```
$\sim$/src/edgejs/napi/quickjs/src/unofficial_napi.cc
```

Added per-env storage for promise context frames:

```
std::unordered_map<void *, JSValue> promise_context_frames;
std::vector<JSValue> promise_context_frame_stack;
```

Added a QuickJS promise hook implementation:

- On JS_PROMISE_HOOK_INIT, capture the current continuation_preserved_embedder_data for the newly-created promise.
- On JS_PROMISE_HOOK_BEFORE, push the current frame and restore the frame captured for that promise.
- On JS_PROMISE_HOOK_AFTER, restore the previous frame and remove the completed promise frame entry.
- Forward registered JS promise hooks (init, before, after, settled) when they are present.

Registered the hook during env creation:

```
JS_SetPromiseHook(rt, QuickjsPromiseHook, env);
```

Also updated:

- unofficial_napi_set_promise_hooks(...) to install the QuickJS hook after storing callbacks.
- unofficial_napi_set_promise_reject_callback(...) to register JS_SetHostPromiseRejectionTracker(...).
- unofficial_napi_enqueue_microtask(...) to enqueue a real QuickJS job with JS_EnqueueJob(...) instead of synthesizing a Promise.resolve().then(...).

7.1.6 Verification

Rebuilt the QuickJS Edge CLI:

```
cmake --build build-edge-quickjs-cli --target edge -j4
```

Verified the REPL works with persistent history enabled by pointing HOME at a temporary directory:

```
env HOME=/private/tmp/edge-qjs-home ./build-edge-quickjs-cli/edge
```

Inside the REPL:

```
> 10+32
42
>
```

Verified async context survives promise continuations:

```
./build-edge-quickjs-cli/edge --async-context-frame -e \
  "const { AsyncLocalStorage } = require('async_hooks'); const als = new AsyncLocalStorage(); als.run(123,
```

Observed:

```
als 123
```

Rebuilt and ran the QuickJS N-API tests:

```
env CMAKE_BUILD_TYPE=Debug make build-napi-quickjs
make test-napi-quickjs-only
```

Result:

```
100% tests passed, 0 tests failed out of 43
```

7.1.7 Remaining separate issue

Exiting the QuickJS CLI still hits the existing teardown assertion:

Assertion failed: (list_empty(&rt->gc_obj_list)), function JS_FreeRuntime

That is separate from the REPL lock. The REPL now becomes ready, resumes after history initialization, accepts input, and evaluates commands with persistent history enabled.

Chapter 8

Edge QuickJS WASIX Wasmer run enablement

Source note: `development/005_wasix_wasmer_http.md`

8.1 Edge QuickJS WASIX Wasmer run enablement

8.1.1 Context

The goal for this phase was to make the QuickJS-backed Edge WASIX build work under Wasmer, including both the default package entrypoint and running a JS script through the package:

```
wasmer run .  
wasmer run --volume ./:/app . -- /app/echo-server.js
```

The desired end state was not just “the binary starts”; the HTTP echo server also needed to accept a connection, dispatch the request into JS, and return a response.

8.1.2 Build shape

The relevant local Edge build output is:

```
$\sim$/src/edgejs/build-quickjs-wasix/edge
```

For local package testing, the built executable is also copied to the package names Wasmer expects:

```
$\sim$/src/edgejs/build-quickjs-wasix/edge.wasm  
$\sim$/src/edgejs/build-quickjs-wasix/edgejs.wasm
```

The direct build commands used during debugging were:

```
cmake --build build-quickjs-wasix --target edge -j4  
cmake --build build-edge-quickjs-cli --target edge -j4
```

The Makefile now has helper targets for these:

```
make build-quickjs-wasix-edge JOBS=4  
make build-edge-quickjs-cli-edge JOBS=4  
make build-quickjs-edge-targets JOBS=4  
make clean-quickjs-wasix  
make clean-edge-quickjs-cli  
make clean-quickjs-edge-targets  
make test-only-quickjs
```

8.1.3 Wasmer compatibility notes

The Wasmer binary that worked for local testing was built from `$/sim$/src/wasmer` with the CLI features that allow artifact loading and LLVM execution:

```
cargo build --release \  
  --manifest-path lib/cli/Cargo.toml \  
  --features llvm,napi-v8,wasmer-artifact-create,static-artifact-create,wasmer-artifact-load,static-artifact-load
```

```
--bin wasmer \
--locked
```

Earlier, a Wasmer invocation failed with:

```
compile error: Validate("exception refs not supported without the exception handling feature ...")
```

That was a Wasmer/runtime feature mismatch for this WASIX artifact. Running with a compatible locally-built Wasmer, and during investigation using:

```
--llvm --enable-exceptions --disable-cache
```

got us past validation. Later, after the artifact and runtime path were in a good state, the simpler package commands worked too:

```
wasmer run .
wasmer run --volume ./:/app . -- /app/echo-server.js
```

For network tests, Wasmer needs networking enabled:

```
wasmer run --net --volume ./:/app . -- /app/echo-server.js
```

8.1.4 Bootstrap failure: Atomics under WASIX

After temporarily getting past the QuickJS teardown assertion so that startup errors were visible, Wasmer reported:

```
Failed to execute internal/per_context/primordials: undefinedError
    at ownKeys (native)
    at copyPropsRenamed (<input>:78:36)
```

The failure happened very early in Node bootstrap, while executing `internal/per_context/primordials`.

The underlying problem was that QuickJS's Atomics/thread support was disabled for all `__wasi__` builds. Edge's Node bootstrap expects Atomics and SharedArrayBuffer to exist for this WASIX target when `wasm` atomics are available.

We changed the QuickJS guards so WASIX can use atomics when `__wasm_atomics__` is defined:

```
$\sim$/src/edgejs/quickjs/quickjs.c
$\sim$/src/edgejs/quickjs/cutils.h
```

The important condition became:

```
(!defined(__wasi__) || defined(__wasm_atomics__))
```

instead of excluding `__wasi__` unconditionally.

After that, this WASIX check succeeded:

```
wasmer run . -- -e "console.log(typeof Atomics, typeof SharedArrayBuffer)"
```

Expected output:

```
object function
```

That fixed the bootstrap failure in `internal/per_context/primordials`.

8.1.5 HTTP server hang

Once bootstrap worked, the echo server could start:

```
quickjs edge echo listening on 3000
```

But `curl http://127.0.0.1:3000/` connected and then hung. Adding:

```
console.log(`quickjs edge echo: ${req.method} ${req.url}`)
```

inside `echo-server.js` showed that the HTTP request callback fired in the native QuickJS CLI but did not fire

in the WASIX run.

We added gated network tracing with `EDGE_TRACE_NET=1` across:

```
$\sim$/src/edgejs/src/edge_tcp_wrap.cc
$\sim$/src/edgejs/src/edge_stream_base.cc
$\sim$/src/edgejs/src/edge_stream_listener.cc
$\sim$/src/edgejs/src/edge_http_parser.cc
$\sim$/src/edgejs/lib/_http_server.js
$\sim$/src/edgejs/lib/internal/stream_base_commons.js
```

The trace showed that libuv networking was working:

```
tcp bind6 ... rc=0(OK)
tcp listen ... rc=0(OK)
tcp onconnection ... status=0(OK)
tcp accept ... rc=0(OK)
tcp read_start ... rc=0(OK)
stream read ... nread=77
```

But the request bytes were delivered to the wrong stream listener. The key trace shape was:

```
http_parser consume ... stream=0x...dd0
stream uv_read base=0x...dd8 ...
stream missing_onread ...
```

The parser listener was attached at `0x...dd0`, while libuv reads arrived on `0x...dd8`.

8.1.6 Root cause of the stream pointer mismatch

`TcpWrap` is laid out with the N-API env first and the stream base second:

```
struct TcpWrap {
  napi_env env = nullptr;
  EdgeStreamBase base{};
  uv_tcp_t handle{};
  int socket_type = kTcpSocket;
};
```

So:

```
0x...dd0 == TcpWrap*
```

```
0x...dd8 == &TcpWrap::base
```

`parser.consume(socket._handle)` calls `EdgeStreamBaseFromValue(...)`.

On QuickJS, `socket._handle` was reported as `napi_external`, because our QuickJS N-API constructor path creates class instances with the same QuickJS class id used for N-API externals. The conversion code therefore treated the wrapped TCP object as a raw external pointer and returned the wrapper address instead of unwrapping it and returning `&wrap->base`.

That attached the HTTP parser listener to the wrong `EdgeStreamBase`. The real libuv read path continued using the correct `&wrap->base`, so the parser never saw the request bytes.

8.1.7 Stream fix

We added wrapper-specific stream-base accessors and made the generic stream conversion prefer them before falling back to raw external data:

```
$\sim$/src/edgejs/src/edge_tcp_wrap.h
$\sim$/src/edgejs/src/edge_tcp_wrap.cc
$\sim$/src/edgejs/src/edge_pipe_wrap.h
$\sim$/src/edgejs/src/edge_pipe_wrap.cc
$\sim$/src/edgejs/src/edge_tty_wrap.h
```

```
$\sim$/src/edgejs/src/edge_tty_wrap.cc
$\sim$/src/edgejs/src/edge_stream_base.cc
```

The TCP/Pipe/TTY helpers now:

1. Accept `napi_object`, `napi_function`, and QuickJS's current `napi_external`-reported class instances.
2. Call `napi_unwrap(...)`.
3. Validate the native wrapper by checking `handle->data == wrap`.
4. Validate the libuv handle type, for example `UV_TCP` for TCP.
5. Return `&wrap->base`.

After the fix, the trace changed to the correct shape:

```
tcp get_stream_base unwrap_status=0 wrap=0x...dd0
tcp get_stream_base base=0x...dd8 handle=0x...e70 data=0x...dd0 type=12 expected=12
stream from_value tcp base=0x...dd8
http_parser consume ... stream=0x...dd8
stream uv_read base=0x...dd8 current=<http parser listener>
http_parser consumed_read ... nread=77
js http_server parserOnIncoming method= GET url= /
quickjs edge echo: GET /
```

curl then received a normal response:

```
HTTP/1.1 200 OK
content-type: text/plain
```

```
quickjs edge echo: GET /
```

8.1.8 Verification

The traced verification command used during debugging was:

```
$\sim$/src/wasmer/target/release/wasmer run \
  --llvm --enable-exceptions --disable-cache --net \
  --env EDGE_TRACE_NET=1 \
  --env PORT=3003 \
  --volume $\sim$/src/edgejs/quickjs-wasm:/app \
  . -- /app/echo-server.js
```

Then:

```
curl -v --max-time 8 http://127.0.0.1:3003/
```

This returned:

```
quickjs edge echo: GET /
```

The clean non-traced run also worked and stayed alive after the keep-alive close window.

The final user-confirmed commands also work:

```
wasmer run --volume ./:/app . -- /app/echo-server.js
wasmer run .
```

8.1.9 Current status

The QuickJS Edge WASIX package can now bootstrap under Wasmer and run the echo server path correctly.

The important implementation lessons are:

1. WASIX QuickJS needs `Atomics` enabled when the target has `wasm atomics`.
2. QuickJS N-API class instances may currently appear as `napi_external`, so code that accepts both wrapper objects and raw externals must try `napi_unwrap(...)` first when a known wrapper type is possible.

3. The HTTP parser consume path depends on the listener being attached to the exact `EdgeStreamBase` used by the libuv read callbacks.
4. Embedded QuickJS WASIX targets that include N-API headers must compile with the embedded-provider declaration mode (`NAPI_EXTERN=`). If one static library sees default `__wasm__` N-API imports from module `napi` while another sees the same unresolved calls through `env`, the final `wasm-ld` link fails with an import module mismatch. The concrete fixed case was `edge_environment_core`.

Longer term, it may be cleaner for the QuickJS N-API implementation to avoid representing constructed N-API class instances with the same QuickJS class id as plain `napi_external` values. The stream wrapper fix is intentionally narrow and validated, but the type-reporting behavior is still worth revisiting.

Chapter 9

Edge QuickJS web app enablement for Astro, Vite, and Next.js

Source note: `development/006_framework_app_adapters.md`

9.1 Edge QuickJS web app enablement for Astro, Vite, and Next.js

9.1.1 Context

`001_merge_analysis.md` through `005_wasix_wasmer_http.md` cover the provider integration, bootstrap debugging, REPL microtasks, WASIX atomics, and the HTTP stream listener bug that prevented a simple echo server from handling requests under Wasmer.

This note captures the next layer of work: using the published Edge QuickJS Wasmer package to run real framework outputs:

- an Astro static site at `sim$/src/astro-app`;
- a Vite/React SPA at `sim$/src/vite-app`;
- a Next.js app at `sim$/src/next-app`.

The important distinction is that Edge QuickJS now runs the HTTP/static adapter inside WASIX. Framework build systems still run outside the WASIX runtime during the build step.

9.1.2 Edge QuickJS package shape

The root package definition is:

```
sim$/src/edgejs/wasmer.toml
```

Current package identity:

```
[package]
entrypoint = "edgejs-quickjs"
name = "sadhbh-c0d3/edgejs-quickjs"
description = "Edge.js with an embedded QuickJS N-API provider"
version = "0.0.1"
```

The command exports the built WASIX Edge binary as the `edge` command:

```
[[module]]
name = "edge"
source = "../build-quickjs-wasix/edgejs.wasm"

[[command]]
name = "edge"
module = "edge"
runner = "https://webc.org/runner/wasi"

[command.annotations.wasi]
atom = "edge"
```

The package also mounts SSL certificates:

```
[fs]
"/usr/local/ssl" = "./ssl-certs"
```

Applications consume it with:

```
[dependencies]
"sadhbh-c0d3/edgejs-quickjs" = "0.0.1"
```

or, where semver range is wanted:

```
[dependencies]
"sadhbh-c0d3/edgejs-quickjs" = "^0.0.1"
```

and point their command at:

```
module = "sadhbh-c0d3/edgejs-quickjs:edge"
runner = "https://webc.org/runner/wasi"
```

The app-specific `main-args` then name the JavaScript adapter file to execute.

9.1.3 Runtime changes that made framework adapters possible

These are the Edge QuickJS fixes that matter for Astro, Vite, and Next.js.

9.1.3.1 ContextifyScript must be a class

Node's `lib/internal/vm.js` constructs `ContextifyScript` with `new ContextifyScript(...)`.

The original QuickJS path registered `ContextifyScript` with `napi_create_function()`. That produced a callable function, but not a N-API class shape compatible with Node's new `ContextifyScript(...)` path.

The fix was in:

```
$\sim$/src/edgejs/src/edge_module_loader.cc
```

`ResolveContextifyBinding()` now defines `ContextifyScript` with `napi_define_class(...)`.

This was necessary before `internal/vm`, CommonJS compilation, and user script execution could work reliably.

9.1.3.2 QuickJS promise hooks and microtasks

The REPL investigation showed that bytes reached the JS stream layer, but some promise continuations did not run. The missing piece was QuickJS promise hook integration plus proper microtask/job draining.

The fix is described in `004_promise_hooks_microtasks.md` and involved:

```
$\sim$/src/edgejs/quickjs/quickjs.c
```

```
$\sim$/src/edgejs/napi/quickjs/src/unofficial_napi.cc
```

The relevant behavior:

- `unofficial_napi_set_promise_hooks(...)` stores Node's hook callbacks and registers a QuickJS runtime promise hook.
- `unofficial_napi_enqueue_microtask(...)` enqueues a real QuickJS job instead of relying on a fake callback path.
- `unofficial_napi_process_microtasks(...)` drains QuickJS pending jobs with `JS_ExecutePendingJob(...)`.
- The vendored QuickJS source emits before/after promise hook events around the internal promise reaction job.

This matters for web app adapters because framework code often relies on Promise, async callbacks, stream scheduling, file reads, and server startup continuations even when the final adapter looks like a simple static server.

9.1.3.3 Atomics and SharedArrayBuffer under WASIX

Wasmer startup initially failed in `internal/per_context/primordials` because QuickJS disabled atomics for every `__wasi__` target.

The fix is described in `005_wasix_wasmer_http.md` and lives in the QuickJS submodule:

```
$\sim$/src/edgejs/quickjs/quickjs.c
$\sim$/src/edgejs/quickjs/cutils.h
```

The important condition changed from excluding `__wasi__` unconditionally to allowing atomics when the WASIX build defines `wasm atomics`:

```
(!defined(__wasi__) || defined(__wasm_atomics__))
```

Without this, Node bootstrap can fail before any app adapter gets to run.

9.1.3.4 HTTP stream listener attachment

The echo-server hang under Wasmer was caused by the HTTP parser listener being attached to the wrapper address instead of the wrapper's `EdgeStreamBase`.

The fix is described in `005_wasix_wasmer_http.md` and touched:

```
$\sim$/src/edgejs/src/edge_tcp_wrap.h
$\sim$/src/edgejs/src/edge_tcp_wrap.cc
$\sim$/src/edgejs/src/edge_pipe_wrap.h
$\sim$/src/edgejs/src/edge_pipe_wrap.cc
$\sim$/src/edgejs/src/edge_tty_wrap.h
$\sim$/src/edgejs/src/edge_tty_wrap.cc
$\sim$/src/edgejs/src/edge_stream_base.cc
```

The generic stream conversion now tries wrapper-specific unwrapping first:

- TCP;
- Pipe;
- TTY.

Those helpers validate the unwrapped native handle and return the exact `EdgeStreamBase` used by libuv reads. Only after those checks does the generic path fall back to raw external data.

This is the core networking fix that allowed:

```
http.createServer(...)
```

to accept a connection, parse the request, call the JS request callback, and write a response under Wasmer.

9.1.3.5 Network tracing

The debug flag added during the HTTP work remains useful:

```
EDGE_TRACE_NET=1
```

It traces through native and JS layers:

```
$\sim$/src/edgejs/src/edge_tcp_wrap.cc
$\sim$/src/edgejs/src/edge_stream_base.cc
$\sim$/src/edgejs/src/edge_stream_listener.cc
$\sim$/src/edgejs/src/edge_http_parser.cc
$\sim$/src/edgejs/lib/_http_server.js
$\sim$/src/edgejs/lib/internal/stream_base_commons.js
```

The useful success shape is:

```
tcp bind/listen/accept/read_start ... OK
http_parser consume ... stream=<same EdgeStreamBase as uv_read>
```

```
http_parser consumed_read ... nread=<request bytes>
js http_server parserOnIncoming method= GET url= /
```

The old broken shape was `stream missing_onread` after bytes were read.

9.1.3.6 QuickJS teardown is still not solved

The N-API QuickJS submodule currently has `JS_FreeRuntime(...)` commented out in the env release path:

```
$\sim$/src/edgejs/napi/quickjs/src/unofficial_napi.cc
```

The submodule commit is:

```
a200c14 Disabling JS_FreeRuntime until gc leaks are solved
```

This prevents the known teardown assertion from aborting successful Wasmer app runs:

```
Assertion failed: list_empty(&rt->gc_obj_list)
```

This is intentionally not a real GC/leak fix. It is a current compatibility workaround so the runtime can exit without aborting while the remaining QuickJS object lifetime issues are investigated.

9.1.4 Running static sites through Edge QuickJS

The common pattern for Astro and Vite is:

1. Build the framework output with its normal Node-based build tool.
2. Mount the generated static directory into WASIX.
3. Run a small CommonJS `http + fs` adapter under Edge QuickJS.
4. Pass `--net` to Wasmer when serving HTTP.

The adapter should be conservative:

- use CommonJS (`.cjs`) so Edge can execute it through the current CJS path;
- use Node builtins we know are working: `fs`, `path`, `http`, `url` where needed;
- avoid depending on the framework's dev server or native Node add-ons at runtime;
- normalize request paths and reject traversal;
- support GET and HEAD where possible;
- send explicit content types for JS, CSS, JSON, images, WASM, HTML, and RSC payloads.

9.1.5 Astro: `$\sim$/src/astro-app`

Astro's build output can be served as static files. The adapter added there is:

```
$\sim$/src/astro-app/server-edgejs.cjs
```

It is a minimal static server:

- root is `/app/dist`;
- port defaults to `process.env.PORT || 3000`;
- `/` maps to `index.html`;
- directories map to `index.html`;
- files are served with a small content type table;
- errors are logged and returned as `500`.

The working shape is:

```
cd $\sim$/src/astro-app
npm run build
wasmer run \
  --net \
  --env PORT=3000 \
  --volume ./:/app \
```

```
sadhbh-c0d3/edgejs-quickjs@0.0.1 \
-- /app/server-edgejs.cjs
```

This uses the published Edge QuickJS package directly and mounts the project at `/app`, so the adapter can read `/app/dist`.

This is static serving, not Astro SSR. It proves the Edge QuickJS WASIX package can run a normal Node-style static HTTP server around Astro's generated output.

9.1.6 Vite: `$\sim$/src/vite-app`

The Vite version was changed from a PHP/WinterJS package shape to an Edge QuickJS package shape.

Important files:

```
$\sim$/src/vite-app/wasmer.toml
```

```
$\sim$/src/vite-app/server/router.cjs
```

The package depends on the published runtime:

```
[dependencies]
"sadhbh-c0d3/edgejs-quickjs" = "^0.0.1"
```

The package mounts only app artifacts:

```
[fs]
"/dist" = "./dist"
"/server" = "./server"
```

The command executes the CJS router:

```
[[command]]
name = "run"
module = "sadhbh-c0d3/edgejs-quickjs:edge"
runner = "https://webc.org/runner/wasi"

[command.annotations.wasi]
main-args = ["/server/router.cjs"]
```

`server/router.cjs` is a Vite SPA static adapter:

- root is `/dist`;
- port defaults to `process.env.PORT || 8080`;
- supports GET and HEAD;
- rejects path traversal;
- serves existing files and directories;
- falls back to `/dist/index.html` for SPA routes;
- sends immutable cache headers for `/assets/`;
- has content types for `.html`, `.js`, `.mjs`, `.css`, `.json`, `.wasm`, common images, and `.xcf`.

The expected run flow is:

```
cd $\sim$/src/vite-app
npm run build
wasmer run --net .
```

The Vite case mainly validated the published package consumption story:

- no local `$\sim$/src/edgejs/quickjs-wasm` path is needed;
- no `ssl-certs` package dependency is needed in the app;
- `wasmer.toml` should reference `sadhbh-c0d3/edgejs-quickjs@0.0.1` as a dependency and execute its `edge` atom.

9.1.7 Next.js: `$/sim$/src/next-app`

Next.js needed a more specialized adapter than Astro/Vite because the app uses App Router output, RSC payloads, static `_next` assets, public files, and a few dynamic routes.

Important files:

```
$/sim$/src/next-app/package.json
$/sim$/src/next-app/wasmer.toml
$/sim$/src/next-app/server/router.cjs
$/sim$/src/next-app/server/generate-next-dynamic-shells.cjs
$/sim$/src/next-app/server/build-edge-dist.cjs
```

9.1.7.1 Package shape

The app package depends on Edge QuickJS:

```
[dependencies]
"sadhbh-c0d3/edgejs-quickjs" = "0.0.1"
```

The Wasmer package now mounts staged runtime files, not the full `.next` directory:

```
[fs]
"/web" = ".dist/web"
"/public" = ".dist/public"
"/server" = ".dist/server"

[[command]]
name = "run"
module = "sadhbh-c0d3/edgejs-quickjs:edge"
runner = "https://webc.org/runner/wasi"

[command.annotations.wasi]
main-args = ["/server/router.cjs"]
```

The package scripts are:

```
"edge:build": "next build && node server/generate-next-dynamic-shells.cjs && node server/build-edge-dist.cjs"
"edge:stage": "node server/build-edge-dist.cjs",
"edge:preview": "wasmer run --net ."
```

The correct build command is:

```
npm run edge:build
```

not:

```
npm run build edge:build
```

9.1.7.2 Runtime router

`server/router.cjs` is the CJS adapter executed by Edge QuickJS.

It serves:

- `/_next/static/*` from `/web/static`;
- `/favicon.ico` from `/web/server/app/favicon.ico.body`;
- public files from `/public`;
- static App Router HTML/RSC files from `/web/server/app`;
- generated dynamic shells from `/server/generated`;
- `_not-found.html` as the 404 fallback.

It detects RSC requests with:

```
req.headers.rsc === "1" || url.searchParams.has("_rsc")
```

and serves text/x-component for .rsc responses.

The dynamic routes currently handled are:

```
/lobby/:id
```

```
/tables/:id?lobbyId=:lobbyId
```

Those routes use generated files with placeholders:

```
/server/generated/lobby.html
```

```
/server/generated/lobby.rsc
```

```
/server/generated/table.html
```

```
/server/generated/table.rsc
```

At request time the router replaces:

```
__EDGE_LOBBY_ID__
```

```
__EDGE_TABLE_ID__
```

with URL-safe route values.

/local-rpc is intentionally not implemented in the WASI static adapter and returns 502.

9.1.7.3 Dynamic shell generation

server/generate-next-dynamic-shells.cjs runs after next build on the host Node runtime, not inside Edge QuickJS.

It:

1. imports the emitted Next route module, for example .next/server/app/(site)/lobby/[id]/page.js;
2. starts a temporary localhost Node HTTP server;
3. calls the Next route module handler;
4. fetches both HTML and RSC variants;
5. writes placeholder-based generated shells into server/generated.

Routes currently generated:

```
{
  modulePath: ".next/server/app/(site)/lobby/[id]/page.js",
  url: "/lobby/__EDGE_LOBBY_ID__",
  html: "lobby.html",
  rsc: "lobby.rsc",
}
{
  modulePath: ".next/server/app/tables/[id]/page.js",
  url: "/tables/__EDGE_TABLE_ID__?lobbyId=__EDGE_LOBBY_ID__",
  html: "table.html",
  rsc: "table.rsc",
}
```

The generator also installs AsyncLocalStorage on globalThis for the Next route module:

```
globalThis.AsyncLocalStorage = globalThis.AsyncLocalStorage || AsyncLocalStorage;
```

9.1.7.4 Table route emission bug

The table dynamic shell was initially missing:

Skipping /tables/__EDGE_TABLE_ID__?lobbyId=__EDGE_LOBBY_ID__:

.next/server/app/tables/[id]/page.js was not emitted by next build

and clicking Join opened:

Next dynamic route shell is missing. Run `npm run edge:build`.

Root cause:

```
$\sim$/src/next-app/app/tables/[id]/layout.js
```

had:

```
export const runtime = 'edge';
```

Next emitted:

```
{
  "/tables/[id]/page": "app-edge-has-no-entrypoint"
}
```

instead of `.next/server/app/tables/[id]/page.js`. The generator can only render the Node server page module, so it skipped the table route.

Fix:

- remove the forced `runtime = 'edge'` from the table route layout;
- rebuild with `npm run edge:build`.

After that, Next reports:

```
f /tables/[id]
```

and emits:

```
.next/server/app/tables/[id]/page.js
```

The lobby Join/Return URL was also changed to carry the current lobby:

```
href={` /tables/${id}?lobbyId=${encodeURIComponent(lobbyId)}`}
```

so the generated table shell receives the right `searchParams.lobbyId`.

9.1.7.5 .dist staging

Mounting the whole `.next` directory worked but was unnecessarily large:

```
37M .next
```

The build now stages only the runtime files used by `server/router.cjs`:

```
$\sim$/src/next-app/server/build-edge-dist.cjs
```

It copies:

- `.next/static` to `.dist/web/static`;
- selected `.next/server/app` files to `.dist/web/server/app`;
- `public` to `.dist/public`;
- `server/router.cjs` to `.dist/server/router.cjs`;
- `server/generated` to `.dist/server/generated`.

Only these Next app artifact extensions are kept:

```
.body
.html
.rsc
```

The staging step excludes `.segments` directories because the current router does not serve those paths.

Observed size after staging:

```
37M .next
8.6M .dist
6.9M .dist/web/static
256K .dist/web/server/app
```


1.3M .dist/public
68K .dist/server

This is the directory Wasmer packages now mount.

9.1.7.6 Next.js verification commands

Build:

```
cd $sim$/src/next-app
npm run edge:build
```

Package check:

```
wasmer package build --check .
```

Preview:

```
wasmer run --net --env PORT=3022 .
```

Useful probes:

```
curl -I http://127.0.0.1:3022/
curl -I http://127.0.0.1:3022/main-lobby
curl -i 'http://127.0.0.1:3022/tables/123?lobbyId=1'
curl -i -H 'rsc: 1' 'http://127.0.0.1:3022/lobby/1'
curl -i http://127.0.0.1:3022/_next/static/chunks/49dced3c3039a42d.css
```

The verified result was 200 OK for the HTML, RSC, and CSS paths.

9.1.8 What this does and does not prove

9.1.8.1 Proven

- The published Edge QuickJS Wasmer package can run CommonJS HTTP server adapters.
- WASIX networking works for HTTP server workloads when run with `--net`.
- Static file serving with `fs.readFileSync`, `fs.statSync`, `path`, `Buffer`, content types, and headers works.
- Vite/React and Astro static outputs can be served directly.
- Next App Router static HTML/RSC output can be served by a custom adapter.
- Selected Next dynamic routes can be served by host-generated HTML/RSC shells plus placeholder replacement.

9.1.8.2 Not proven

- Full Next.js server runtime execution inside Edge QuickJS WASIX.
- Next route handlers, middleware, API routes, streaming SSR, or per-request server component execution inside Edge QuickJS.
- Astro SSR inside Edge QuickJS.
- Vite dev server or HMR inside Edge QuickJS.
- Native Node add-ons inside the WASIX app package.
- The QuickJS runtime teardown leak/assertion fix.

For now the strategy is:

framework build/SSR shell generation on host Node
small CJS HTTP adapter at runtime under Edge QuickJS WASIX

That is good enough for static and mostly-static sites, and it is a useful pressure test for Edge QuickJS's Node compatibility surface.

9.1.9 Practical app authoring rules

For a new static-ish app:

1. Build with the framework's normal command.
2. Add a small `.cjs` router using only stable Node builtins.
3. Mount generated artifacts explicitly in `wasmer.toml`.
4. Use `sadhbh-c0d3/edgejs-quickjs@0.0.1` as a dependency.
5. Run with `wasmer run --net ..`

For Next.js:

1. Avoid `export const runtime = 'edge'` on routes that the shell generator must import from `.next/server/app/.../page.js`.
2. Keep `server/generate-next-dynamic-shells.cjs` route list in sync with every dynamic route the adapter should serve.
3. Keep `server/router.cjs` route matching and placeholder replacement in sync with generated shells.
4. Run `npm run edge:build`, not plain `npm run build`, before `wasmer run`.
5. Package `.dist`, not the full `.next`, unless the router starts needing more Next internals.

9.1.10 Remaining work

1. Fix QuickJS teardown properly so `JS_FreeRuntime(...)` can be restored.
2. Revisit QuickJS N-API object type reporting so constructed class instances do not look like plain `napi_external` values.
3. Decide whether `EDGE_TRACE_NET` and `EDGE_TRACE_TTY` should stay in-tree as supported diagnostics or be reduced before merging.
4. Add a tiny framework smoke-test matrix for:
 - `echo-server.js`;
 - Astro static;
 - Vite static SPA fallback;
 - Next static route;
 - Next generated dynamic HTML route;
 - Next generated dynamic RSC route.
5. Consider a reusable adapter template package once the static-server pattern repeats one more time.
6. For real SSR, investigate whether framework server bundles can be made QuickJS-compatible, or whether build-time shell generation remains the intended model for this runtime.

Chapter 10

Edge QuickJS framework standalone builds

Source note: `development/007_framework_standalone_builds.md`

10.1 Edge QuickJS framework standalone builds

10.1.1 Context

`006_framework_app_adapters.md` captured the first working framework-app path for Edge QuickJS: small app-owned adapters for Astro, Vite, and Next.js outputs.

The next goal was to remove as much app-specific server/router glue as possible and prefer framework-standard standalone build outputs:

- Astro standalone Node adapter output;
- Vite SSR standalone packaging via `vite-plugin-standalone`;
- Next.js official output: "standalone" build.

The anonymized app paths used in notes are:

```
$\sim$/src/astro-app
```

```
$\sim$/src/vite-app
```

```
$\sim$/src/next-app
```

10.1.2 Astro standalone

Astro already has the cleanest standalone story. It can emit a server build with the Node adapter in standalone mode:

```
export default defineConfig({
  output: 'server',
  adapter: node({
    mode: 'standalone',
  }),
});
```

That build produces a server entry under:

```
dist/server/entry.mjs
```

For Edge QuickJS compatibility, the local follow-up was to bundle that emitted server entry into a CJS artifact that the current Edge CLI can execute:

```
dist/server/entry.cjs
```

The `.mjs` entry is still not the working EdgeJS execution path; Astro standalone currently works through the bundled CJS artifact.

The important point is that Astro owns the server semantics. The app does not need to keep a bespoke HTTP router once the build emits the standalone server entry.

One observed Astro app caveat was `astro-blog`, which failed because the standalone dependency graph pulled in `sharp`. That failure appears tied to `sharp`'s native / WebAssembly dependency shape rather than Astro standalone itself.

10.1.3 Vite standalone

Plain Vite SPA builds do not emit an HTTP server entry. A normal Vite build only produces static client files under:

`dist/`

So the earlier `vite-app` path used an app-owned static router to serve files, handle GET / HEAD, prevent path traversal, set content types, and fall back to `index.html` for client-side routes.

The standalone experiment found that `vite-plugin-standalone` is a useful fit when the app provides a server entry for the plugin to package. The plugin uses the SSR build path, `esbuild`, and `@vercel/nft` to produce a self-contained server artifact.

The resulting `vite-app` direction is:

- use `vite-plugin-standalone`;
- keep the minimal server semantics as a build-owned entry;
- drop the old manually staged `server/router.cjs` / `server/router.php` style;
- execute the standalone output from the Wasmer package.

This is not the same as Astro, because Vite still does not invent HTTP server semantics for a plain SPA. But it does let the project keep the standalone version as the maintained path, with dependency tracing and packaging handled by the Vite plugin instead of custom Edge staging scripts.

More detail is captured in:

`plans/quickjs-wasm/development/troubleshooting/vite-app/001_standalone_build.md`

10.1.3.1 Native CLI static-root caveat

The Vite standalone entry tested in the app used:

```
var root = process.env.STATIC_ROOT || "/dist";
```

That default works under:

```
wasmer run --net .
```

because `/dist` is resolved inside the WASIX / Wasmer package filesystem, where the packaged app assets are available.

It does not work the same way when running the host native QuickJS CLI directly:

```
$\sim$/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/standalone-entry.js
```

In that mode, `/dist` means the host machine's absolute `/dist`, not the app's local `dist/` directory. The server can start, but the first HTTP request falls through to `fs.readFileSync("/dist/index.html")`, which fails in `fs.openSync(...)` because the host path does not exist.

Two useful native-CLI fixes are:

```
STATIC_ROOT="$PWD/dist" $\sim$/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/standalone-entry.js
```

or make the entry default relative to the emitted server file:

```
var root = process.env.STATIC_ROOT || path.resolve(__dirname, "..");
```

Since the entry is emitted under `dist/server/`, that resolves to the app's `dist/` directory for native CLI runs and should still resolve to `/dist` in the Wasmer package shape.

The accompanying DEP0176 warning about `fs.F_OK` is separate from the failed read. It is triggered by the fs shim's deprecated `fs.F_OK` getter path, likely via `fs.existsSync(...)`.

10.1.4 Next.js standalone

Next.js also has an official standalone build mode:

```
const nextConfig = {
  output: "standalone",
};
```

The app-level target is:

```
.next/standalone/server.js
```

The old custom server/ directory is not part of the intended path. The standalone output works under stock Node:

```
node ./app/server.js
```

When copied into a test folder, the Edge QuickJS repro shape was:

```
cp -rf ./next/standalone ./testing/app/
cd ./testing
$sim$/src/dev/edgejs/build-edge-quickjs-cli/edge ./app/server.js
```

That failed before the server could start.

10.1.4.1 Native CLI main-entry caveat

Unlike the Vite standalone static server, the Next standalone server does not use `STATIC_ROOT` as its primary asset root. Its generated entry starts with:

```
const dir = path.join(__dirname)

process.env.NODE_ENV = 'production'
process.chdir(__dirname)
```

and then loads Next:

```
require('next')
const { startServer } = require('next/dist/server/lib/start-server')
```

So this command is not fixed by pointing `STATIC_ROOT` at `.testing`:

```
STATIC_ROOT="$PWD/.testing" $sim$/src/dev/edgejs/build-edge-quickjs-cli/edge .next/standalone/server.js
```

The observed native QuickJS CLI failure was:

```
Failed to execute builtin 'internal/main/run_main_module':
undefined    at normalizeRequirableId (<input>:302:35)
              at defaultResolveImpl (<input>:1061:36)
              at resolveForCJSWithHooks (<input>:1066:41)
              at wrapModuleLoad (<input>:247:34)
```

This is an earlier main-module / CJS resolution failure. A minimal native CLI repro showed that direct script execution was not reliably placing the script path in `process.argv[1]`; then `internal/main/run_main_module` reaches:

```
require('internal/modules/cjs/loader').Module.runMain(mainEntry);
```

with `mainEntry` undefined. That eventually calls the CJS resolver with an undefined request and fails in `Built-inModule.normalizeRequirableId(...)`.

This should be tracked separately from static asset root handling. The runtime fix is likely in the native QuickJS CLI argument / main-entry bootstrap path, before continuing to the already-known Next blockers such as QuickJS serdes support for `require("v8")`.

10.1.5 Next.js runtime findings

The Next standalone output is valid, but it exposed two separate Edge runtime gaps.

10.1.5.1 Edge V8 inspector limitation

Edge V8 fails on the same standalone server because Next's startup path eventually requires:

```
require("inspector");
```

The runtime reports:

Error: Inspector is not available

This is not a QuickJS branch regression. Comparing main with napi/quickjs-integration-int showed the relevant inspector files are unchanged:

```
src/internal_binding/binding_config.cc
lib/inspector.js
src/builtin_catalog.cc
```

Both main and the QuickJS integration branch currently set:

```
const bool has_inspector = false;
```

in internalBinding("config"), so lib/inspector.js throws as soon as it is required.

The concrete native V8 standalone repro was:

```
$\sim$/src/dev/edgejs/build-edge/edge .next/standalone/server.js
```

with:

```
Failed to execute builtin 'internal/main/run_main_module':
node:inspector:25
  throw new ERR_INSPECTOR_NOT_AVAILABLE();
  ^
```

```
Error: Inspector is not available
  code: 'ERR_INSPECTOR_NOT_AVAILABLE'
```

10.1.5.2 Edge WASIX builtin failure formatting

The WASIX package path fails with a less specific wrapped builtin error:

```
wasmer run --net .
```

reported:

```
undefined[Error: Failed to execute builtin 'internal/main/run_main_module':   at anonymous (<input>:331:1)
  at compileForInternalLoader (<input>:400:10)
  at compileForPublicLoader (<input>:339:10)
  at loadBuiltinModule (<input>:127:7)
  at loadBuiltinWithHooks (<input>:1197:33)
  at <anonymous> (<input>:1287:69)
  at traceSync (<input>:330:27)
  at wrapModuleLoad (<input>:247:34)
  at <anonymous> (<input>:1550:27)
]
```

This appears to be the WASIX runtime surfacing only the wrapped builtin failure instead of the more specific underlying module/runtime error.

10.1.5.3 Edge QuickJS v8 / serdes blocker

Edge QuickJS fails earlier. The standalone failure reduces to:

```
require("v8");
```

LLDB showed the original exception before Edge wrapped it:

TypeError: cannot read property 'prototype' of undefined

The failing source is in lib/v8.js:

```
Serializer.prototype._getDataCloneError = Error;
```

The V8 backend exports real `Serializer` and `Deserializer` constructors from `internalBinding("serdes")`. The QuickJS backend currently returns an empty object in:

napi/quickjs/src/unofficial_napi.cc

So `require("v8")` fails because `internalBinding("serdes").Serializer` is undefined.

The immediate QuickJS runtime fix is to implement a minimal `serdes` binding that exports stable `Serializer` and `Deserializer` constructors. Full `v8.serialize()` / `v8.deserialize()` behavior can use the existing QuickJS structured clone helpers based on `JS_WriteObject(...)` and `JS_ReadObject(...)`.

More detail is captured in:

plans/quickjs-wasm/development/troubleshooting/next-app/001_standalone_v8_serdes.md

10.1.6 Builtin error formatting

While debugging the Next standalone failure, native builtin execution failures in:

src/edge_module_loader.cc

were found to have regressed from Node-like formatting. The QuickJS debugging path had changed builtin failures to clear the pending JS exception and throw a new wrapper error:

Failed to execute builtin 'internal/main/run_main_module': <original error>

That exposed hidden QuickJS builtin failures, but it made Edge V8 less Node-like. The original Edge V8 output preserved Node's source frame and error object formatting:

```
node:inspector:25
  throw new ERR_INSPECTOR_NOT_AVAILABLE();
  ^
```

Error: Inspector is not available

The current fix preserves the pending JS exception for builtin call failures and only prints one extra context line:

```
Failed to execute builtin 'internal/main/run_main_module':
node:inspector:25
  throw new ERR_INSPECTOR_NOT_AVAILABLE();
  ^
```

Error: Inspector is not available

The rule is:

- add diagnostic context;
- do not clear and replace the original JS exception;
- let the normal fatal-exception path format the original error.

Synthetic Failed to execute builtin ... errors are still appropriate for setup or compile failures where there is no pending JS exception to preserve.

10.1.7 Verification

Both V8 and QuickJS binaries were rebuilt after the formatting change:

```
cmake --build build-edge --target edge -j4
cmake --build build-edge-quickjs-cli --target edge -j4
```

The V8 inspector repro now preserves original formatting with the added context:

```
build-edge/edge -e "require('inspector')"
```

The Next standalone repro under Edge V8 does the same:

```
build-edge/edge ./app/server.js
```

The QuickJS standalone repro still fails on the known v8 / serdes path, but the builtin failure path no longer replaces the original pending exception.

10.1.8 Current standalone status

- Astro has a framework-owned standalone server output and works with EdgeJS when the emitted `.mjs` server is bundled to CJS. The `.mjs` entry itself is not the working path yet. The native QuickJS runtime now includes a minimal `Intl.DateTimeFormat` fallback for framework bootstrap code that only needs simple time formatting, such as Astro's SSR logger. This is deliberately not a full ECMA-402 Intl implementation. Astro apps that pull in sharp can still fail on the sharp native / WebAssembly dependency shape.
- Vite can keep a standalone build path with `vite-plugin-standalone`, provided the project supplies the server semantics the plugin packages. This path is believed to work for the tested Vite app shape.
- Next.js emits the correct official standalone output, but its EdgeJS status depends on which Edge runtime is used:
 - WASIX currently reports only a wrapped `Failed to execute builtin 'internal/main/run_main_module'` error from wasmer `run --net ..`
 - native V8 reaches the known inspector limitation because `require("inspector")` throws `ERR_INSPECTOR_NOT_AVAILABLE`.
 - native QuickJS has the native CLI main-entry issue above, then still needs QuickJS serdes support for `require("v8")`.

10.1.9 Follow-up

The next runtime tasks for full Next standalone under Edge QuickJS are to fix the native CLI main-entry bootstrap path and then implement the QuickJS serdes surface needed by `require("v8")`.

Chapter 11

Edge QuickJS runtime change containment rollback

Source note: `development/008_runtime_change_containment_rollback.md`

11.1 Edge QuickJS runtime change containment rollback

11.1.1 Goal

Rollback the shared EdgeJS runtime directories so they match the upstream `wasmer-io/edgejs` tree again, and keep all QuickJS-specific compatibility work contained in:

- `napi/quickjs/`
- optionally `src/`

The directories to restore from `$\sim$/src/dev/wasmer-io/edgejs` are:

- `lib/`
- `napi/v8/`
- `napi/src/`
- `napi/include/`

This keeps the QuickJS backend from carrying hidden changes in shared JavaScript library files, the V8 provider, or the common N-API headers/sources.

11.1.2 Rollback Surface

Initial comparison showed:

- `napi/src/` already matches the donor tree.
- `napi/include/` already matches the donor tree.
- `napi/v8/` only has an extra `.DS_Store` file in the QuickJS worktree.
- `lib/` differs in eight files:
 - `_http_server.js`
 - `inspector.js`
 - `internal/modules/esm/utils.js`
 - `internal/readline/emitKeypressEvents.js`
 - `internal/readline/interface.js`
 - `internal/repl/history.js`
 - `internal/stream_base_commons.js`
 - `internal/streams/readable.js`

Most `lib/` differences are diagnostics for `EDGE_TRACE_NET` or `EDGE_TRACE_TTY`. Those should not remain in shared runtime files. If equivalent diagnostics are still needed, move them to QuickJS-native stream, TTY, or HTTP parser code under `napi/quickjs/` or to neutral native runtime tracing under `src/`.

The functional risk is `lib/inspector.js`. The current QuickJS tree contains a JavaScript fallback that makes `require("inspector")` return a stable unavailable stub when the build has no inspector. After rollback, the shared library returns to the upstream behavior of throwing `ERR_INSPECTOR_NOT_AVAILABLE` when `internal-Binding("config").hasInspector` is false. If framework code still needs `require("inspector")` to be linkable

under QuickJS, the fix should move below the JavaScript library layer.

11.1.3 Compatibility Strategy

1. Restore the requested directories from the donor tree exactly, including removing files that exist only in the QuickJS worktree under those paths.
2. Rebuild the native QuickJS Edge CLI and run focused smoke tests:
 - `./build-edge-quickjs-cli/edge --version`
 - `./build-edge-quickjs-cli/edge -e "console.log('hello from quickjs')"`
 - `./build-edge-quickjs-cli/edge ./quickjs-wasm/echo-server.js`
3. If `require("inspector")` is still needed by Next.js or another framework, keep `lib/inspector.js` upstream-clean and make the runtime expose a narrow native compatibility path in C/C++ instead of JavaScript. The chosen fallback is to report inspector module availability through `internalBinding("config")` and provide `internalBinding("inspector")` with unavailable/no-op behavior compatible with upstream `lib/inspector.js`. This allows imports to link while active debugger/session operations still fail explicitly.
4. If stream, REPL, or HTTP behavior regresses after losing JS tracing, diagnose from QuickJS-owned code and keep fixes in wrapper/native code rather than reintroducing shared `lib/` edits.
5. Rebuild WASIX after QuickJS-impacting fixes under `napi/quickjs/` or `src/` using:

```
cd $sim$/src/dev/edgejs/quickjs-wasm/ && ./build.sh
```

The QuickJS WASIX target links the embedded QuickJS N-API implementation into the final wasm. Edge targets that include N-API headers before linking `napi_quickjs`, such as `edge_environment_core`, must still define `NAPI_EXTERN=` for the QuickJS provider. Otherwise wasm objects can disagree on the import module for the same `napi_*` symbols (`napi` versus `env`) and fail at the final `wasm-ld` link.

11.1.4 Current Result

The structured-clone API split required Edge messaging call sites to use the three-argument `unofficial_napi_structured_clone(...)` for no-transfer clones and `unofficial_napi_structured_clone_with_transfer(...)` when a transfer list is present. After that source fix, the native root build and native QuickJS CLI build passed.

The QuickJS WASIX build then failed at the final link because `edge_environment_core` saw default WASM N-API imports from the common header. Adding `NAPI_EXTERN=` for that target under `EDGE_NAPI_PROVIDER=quickjs` resolved the mismatch. Verified:

```
make build
make build-edge-quickjs-cli
cd $sim$/src/dev/edgejs/quickjs-wasm && ./build.sh
```

`quickjs-wasm/build.sh` produced `build-quickjs-wasix/edge.wasm` and `build-quickjs-wasix/edgejs.wasm`, and its final no-N-API-imports check passed.

The standalone N-API Cargo workflow also needs to stay on the standalone release family instead of following the vendored path manifest blindly. The vendored `napi/Cargo.toml` tracks local `0.702 / 7.2` path crates, but the `crates.io 7.2.0-alpha.2` graph requires Rust 1.92. With the default Rust 1.91 toolchain, `$sim$/src/dev/edgejs/napi/cargo-standalone.sh test --lib -- --nocapture` failed before tests started. Pinning `napi/Cargo.standalone.toml` to Wasmer 7.1.0 and WASIX/virtual-fs 0.701.0 restores the standalone `crates.io` graph and the library tests pass.

11.1.5 Working Rule

From this rollback onward, changes needed only for Edge QuickJS should not be made in `lib/`, `napi/v8/`, `napi/src/`, or `napi/include/`. Shared runtime changes should happen only when they are correct for all

providers and can be justified independently of QuickJS.

Chapter 12

Node compatibility test failure analysis

Source note: development/009_node_test_failures_analysis.md

12.1 Node compatibility test failure analysis

Date: 2026-05-07

Command under investigation:

```
make test-only TEST_JOBS=4
NODE_TEST_RUNNER=build-edge/edge ./test/nodejs_test_harness --category=node:buffer,node:console,node:dgram,
--skip-tests=known_issues/test-stdin-is-always-net.socket.js,parallel/test-dns-perf_hooks.js,parallel/test
```

The pasted CI run ends with 92 failures out of 1685 tests. A local rerun from this workspace also exits with code 2, but the local failure count is larger because the current workspace build/environment exposes the same broken surfaces across more tests. The root-cause grouping below is based on the pasted CI signatures and source inspection.

12.1.1 Summary

Most failures are not independent subsystem regressions. They cluster around a small number of incomplete Node compatibility surfaces:

1. `require('v8')` can crash during module evaluation because `internalBinding('config').hasInspector` is advertised as true while the profiler internal binding is not implemented.
2. The global console does not write through to stdio in normal `console.*` calls, which also breaks proxy fixtures that log status/body with `console.log()`.
3. The CLI `-p / --print` path evaluates code but does not print expression results.
4. JS-transferable structured clone support loses File identity and returns a plain object.
5. Some flags are accepted or exposed even though the backing feature is missing or incomplete: inspector, CPU/heap profiling, HTTP/HTTP2 debug warning shape, and string-decoder maximum string checks.

12.1.2 Root Cause 1: node:v8 Profiler Crash

Representative failures:

- test-dgram-async-dispose
- test-dgram-*-cluster-*
- test-diagnostics-channel-process
- test-events-add-abort-listener
- test-http-server-drop-connections-in-cluster
- test-domain-top-level-error-handler-throw
- test-domain-uncaught-exception
- test-http2-ping-settings-heapdump
- several TLS tests using `node:test`, `child_process.fork()`, or coverage/test runner helpers

Observed signature:

node:v8:508

```
takeCoverage: profiler.takeCoverage,
```

```
^
```

TypeError: Cannot read properties of undefined (reading 'takeCoverage')

Why it happens:

- `lib/v8.js` initializes `profiler` from `internalBinding('profiler')` whenever `internalBinding('config').hasInspector` is true.
- `src/internal_binding/binding_config.cc` currently hard-codes `hasInspector = true`.
- `src/internal_binding/dispatch.cc` has no `profiler` resolver, so unresolved bindings return undefined.
- `lib/v8.js` then exports `takeCoverage: profiler.takeCoverage`, which throws if `profiler` is undefined.

Relevant code:

- `lib/v8.js`: `profiler` initialization at lines 54-57, export dereference at lines 508-509.
- `src/internal_binding/binding_config.cc`: `hasInspector` is set true at lines 87-113.
- `src/internal_binding/dispatch.cc`: resolver list has no `profiler` entry and unknown bindings return undefined at lines 211-274 and 338-344.

This is a fan-out bug. Fixing this one surface should remove many unrelated looking dgram, cluster, diagnostics, events, HTTP, HTTP2, TLS, domain, URL, and zlib failures that only happen because those tests load `node:v8` indirectly.

12.1.3 Root Cause 2: Global Console Does Not Write

Representative failures:

- `test-console-clear`
- `test-console-count`
- `test-console-diagnostics-channels`
- `test-console-methods`
- `test-console-stdio-setters`
- `test-console`
- pseudo-TTY console color tests
- HTTP/HTTPS proxy tests whose fixtures use `console.log()`

Observed signatures:

- `console.count()` leaves captured stdout empty instead of writing `default: 1\n`.
- `console.clear()` writes nothing instead of cursor-control escape sequences.
- `new console.log()` does not throw the expected `TypeError`.
- Proxy fetch fixture stdout is empty even though the child exits with code 0.
- Proxy request fixtures often contain only the raw response body because `res.pipe(process.stdout)` works, while status lines emitted with `console.log()` are missing.

Why it happens:

- `lib/internal/console/global.js` creates the global console from `internal/console/constructor` and lazily binds `_stdout` / `_stderr`.
- `lib/internal/console/constructor.js` writes through `stream.write(string, errorHandler)` in `kWriteToConsole`.
- Direct `process.stdout.write()` works in this build, but `console.log()` and `console.error()` produce no output. This points at the global console binding path, not at raw stdio.

Relevant code:

- `lib/internal/console/global.js`: global console binding at lines 34-45.
- `lib/internal/console/constructor.js`: stream binding at lines 196-235 and write path at lines 282-323.

- `test/fixtures/fetch-and-log.mjs`: proxy fetch fixture logs body with `console.log()` at lines 1-3.
- `test/fixtures/request-and-log.js`: status/header lines use `console.log()`, while response body uses `res.pipe(process.stdout)` at lines 35-44.

This explains why proxy tests can show outputs such as `Hello World\n` but miss `Status Code: 200`: the body pipe works, the console status lines do not.

12.1.4 Root Cause 3: `-p / --print` Does Not Print

Representative failures:

- `test-http-max-header-size`
- `test-tls-cipher-list`
- other child-process tests that use `-p`, `-pe`, or `--print`

Observed signatures:

- `build-edge/edge -p "1+1"` exits successfully but prints nothing.
- `test-http-max-header-size` gets empty stdout, coerces it with unary `+`, and observes `0 !== 10`.
- `test-tls-cipher-list` receives empty stdout where it expected `crypto.constants.defaultCipherList` or `tls.DEFAULT_CIPHERS`.

Why it happens:

- The CLI recognizes `-p / --print` and routes to `internal/main/eval_string`.
- `lib/internal/main/eval_string.js` passes the `print` option into `evalScript()`.
- The current end-to-end behavior evaluates without printing the completion value.

Relevant code:

- `src/edge_cli.cc`: `eval/print` route to `internal/main/eval_string` at lines 1536-1545.
- `lib/internal/main/eval_string.js`: reads `--print` and passes it through at lines 30-77.
- `test/parallel/test-http-max-header-size.js`: expects printed `http.maxHeaderSize` at lines 8-11.
- `test/parallel/test-tls-cipher-list.js`: uses `-pe` for cipher checks at lines 11-32.

The HTTP max-header and TLS cipher tests are probably not proving those option values are wrong; they are mostly proving `--print` output is missing.

12.1.5 Root Cause 4: File Structured Clone Loses Prototype

Representative failure:

- `test-file`

Observed signature:

`TypeError: clonedFile.text is not a function`

Why it happens:

- `File` extends `Blob` and defines JS-transferable clone hooks `[kClone]() / [kDeserialize]()`.
- The structured clone implementation should detect cloneable JS-transferable values, create a marker, native-clone the marker payload, and deserialize back through `internal/file:TransferableFile`.
- In this build, `structuredClone(new File(...))` returns a plain object, so it has no `Blob/File` methods such as `.text()`.

Relevant code:

- `lib/internal/file.js`: `File` clone/deserialize hooks at lines 117-128 and `TransferableFile` at lines 131-137.
- `lib/internal/worker/js_transferable.js`: deserializer factory setup at lines 33-49 and `structuredClone` wrapper at lines 112-127.

- `src/internal_binding/binding_messaging.cc`: cloneable-transferable detection at lines 669-689, marker preparation at lines 1059-1092, and restoration at lines 1212-1281.
- `test/parallel/test-file.js`: expected cloned File behavior at lines 162-181.

12.1.6 Root Cause 5: Deprecation and Node-Modules Warning Classification

Representative failures:

- `test-buffer-constructor-node-modules`
- `test-buffer-constructor-node-modules-paths`

Observed signatures:

- `--pending-deprecation` child stderr does not match `/DEP0005/`.
- Synthetic call-site tests do not see the expected `[DEP0005] DeprecationWarning`.

Why it happens:

- `Buffer()` warning emission depends on `getOptionValue('--pending-deprecation')` and `internalBinding('util').isInsideNodeModules()`.
- The warning path is present in JS, but the build does not emit the expected warning for the tested child process.
- The node-modules detection is native and stack-based; if call-site names or CLI option propagation differ from Node, this test flips from warning to no warning.

Relevant code:

- `lib/buffer.js`: warning gate and `process.emitWarning(..., 'DEP0005')` at lines 187-209.
- `src/edge_util.cc`: `isInsideNodeModules()` stack inspection at lines 521-579.
- `test/parallel/test-buffer-constructor-node-modules.js`: expected pending warning at lines 28-36.
- `test/parallel/test-buffer-constructor-node-modules-paths.js`: synthetic path expectations at lines 10-37.

12.1.7 Root Cause 6: Inspector/Profile Flags Are Exposed Without Backing Support

Representative failures:

- `test-domain-dep0097`
- `test-diagnostic-dir-cpu-prof`
- `test-diagnostic-dir-heap-prof`
- `test-crypto-secure-heap` and `heapdump/profiling-adjacent` failures that load `node:v8`

Observed signatures:

- `inspector.open()` throws `ERR_INSPECTOR_NOT_AVAILABLE`.
- CPU/heap profiler `diagnostic-dir` tests expect child status 0 and profile files, but the child exits 1.

Why it happens:

- `internalBinding('config').hasInspector` is true, but `internalBinding('inspector')` is a stub whose `open()` method throws `ERR_INSPECTOR_NOT_AVAILABLE`.
- CLI option tables include CPU/heap profiler flags, but the implementation only forwards a narrow set of V8 `--prof` flags to V8 and does not implement Node's `--cpu-prof` / `--heap-prof` profile file lifecycle.

Relevant code:

- `src/internal_binding/binding_config.cc`: `hasInspector = true` at lines 87-113.
- `src/internal_binding/dispatch.cc`: inspector stub methods at lines 278-335.
- `src/edge_cli.cc`: only `--prof`, `--logfile=`, and `--prof-sampling-interval=` are applied as supported V8 profiler flags at lines 253-285.

- `test/sequential/test-diagnostic-dir-cpu-prof.js`: expects a CPU profile file at lines 27-45.
- `test/sequential/test-diagnostic-dir-heap-prof.js`: expects a heap profile file at lines 70-88.

12.1.8 Root Cause 7: Debug Output Does Not Match Node

Representative failures:

- `test-http-debug`
- `test-http2-debug`

Observed signatures:

- HTTP debug output includes Edge proxy-specific lines such as `http createConnection should use proxy ...`, changing the expected stderr.
- HTTP2 debug output contains native HTTP2 ... lines, but misses Node's sensitive-data warning: Setting the `NODE_DEBUG` environment variable to 'http2' can expose sensitive data.

Why it happens:

- Edge's HTTP agent has extra proxy debug logging.
- Edge's native HTTP2 binding writes directly to stderr when `NODE_DEBUG_NATIVE` contains `http2`.
- The test expects Node's paired JS/native debug behavior, including the warning emitted through Node's debuglog path.

Relevant code:

- `lib/_http_agent.js`: proxy debug line at lines 236-240.
- `src/internal_binding/binding_http2.cc`: native HTTP2 debug writes at lines 321-323 and 402-415.
- `test/parallel/test-http2-debug.js`: expected sensitive-data warning and native lines at lines 10-30.

12.1.9 Root Cause 8: StringDecoder Does Not Surface ERR_STRING_TOO_LONG

Representative failure:

- `test-string-decoder`

Observed signature:

- `Large StringDecoder().write(Buffer.alloc(...))` does not throw `ERR_STRING_TOO_LONG`.

Why it happens:

- `lib/string_decoder.js` delegates decoding directly to `internalBinding('string_decoder').decode`.
- The native string decoder builds strings from byte ranges but does not mirror Node/V8's maximum string length error behavior for this path.

Relevant code:

- `lib/string_decoder.js`: `write()` delegates to native `decode()` at lines 76-87.
- `src/edge_string_decoder.cc`: `DecodeBinding()` creates the output string at lines 493-598.
- `test/parallel/test-string-decoder.js`: expected `ERR_STRING_TOO_LONG` at lines 204-210.

12.1.10 Suggested Fix Order

1. Fix `internalBinding('config').hasInspector` / profiler consistency. Either expose `hasInspector = false` for this build or provide a stable profiler binding with no-op `takeCoverage` / `stopCoverage` semantics where appropriate. This should collapse the largest failure cluster.
2. Fix global `console.*` output before investigating proxy tests. The proxy tests depend heavily on console output from child fixtures.
3. Fix `-p` / `--print` result printing. Recheck `test-http-max-header-size` and `test-tls-cipher-list` after that before touching HTTP parser or TLS cipher logic.

4. Fix File structured clone by ensuring the JS-transferable marker path is used and restored for File.
5. Revisit deprecation warning stack classification after the console and print fixes, because warning tests are sensitive to both stderr and call-site formatting.
6. Decide policy for unsupported inspector/profile features: hide/skip them via config and feature flags, or implement enough stubs to satisfy Node's user-visible contracts.
7. Normalize HTTP/HTTP2 debug output to Node's expected warning and line shape, or mark Edge-specific extra diagnostics outside `NODE_DEBUG/test-visible` stderr.
8. Add a native string length guard in the string decoder path.

12.1.11 Verification Targets

After each fix, use targeted tests before the full category run:

```
build-edge/edge -e "console.log('hello')"
build-edge/edge -p "1 + 1"
build-edge/edge test/parallel/test-console-count.js
build-edge/edge test/parallel/test-file.js
build-edge/edge test/parallel/test-buffer-constructor-node-modules.js
build-edge/edge test/parallel/test-http-max-header-size.js
build-edge/edge test/parallel/test-tls-cipher-list.js
build-edge/edge --test-reporter=test/common/test-error-reporter.js --test-reporter-destination=stdout test/
```

Then rerun the original `make test-only TEST_JOBS=4` command.

Part III

Runtime Cleanup And Containment Subtasks

This section breaks out the coordinated cleanup work that kept QuickJS-specific compatibility changes contained while preserving shared runtime behavior.

Chapter 13

Shared runtime rollback

Source note: `development/dev_001_pr_cleanup_containment/001_shared_runtime_rollback.md`

13.1 Shared runtime rollback

13.1.1 Scope

Restore these directories from `$(sim)/src/dev/wasmer-io/edgejs`:

- `lib/`
- `napi/v8/`
- `napi/src/`
- `napi/include/`

This addresses the PR cleanup request that QuickJS-only changes should not live in shared EdgeJS library code, V8 provider code, or common N-API code.

13.1.2 Status

Done locally.

13.1.3 Verification

These comparisons passed after rollback:

```
diff -qr $(sim)/src/dev/wasmer-io/edgejs/lib $(sim)/src/dev/edgejs/lib
diff -qr $(sim)/src/dev/wasmer-io/edgejs/napi/v8 $(sim)/src/dev/edgejs/napi/v8
diff -qr $(sim)/src/dev/wasmer-io/edgejs/napi/src $(sim)/src/dev/edgejs/napi/src
diff -qr $(sim)/src/dev/wasmer-io/edgejs/napi/include $(sim)/src/dev/edgejs/napi/include
```

13.1.4 Notes

The remaining `git status` modifications under `lib/` are expected because the local branch differs from its own git base; the acceptance check for this task is the donor-tree `diff -qr`, not a clean git diff.

Chapter 14

Native inspector fallback

Source note: development/dev_001_pr_cleanup_containment/002_native_inspector_fallback.md

14.1 Native inspector fallback

14.1.1 Scope

Address Syrus's review comment on `lib/inspector.js`: do not patch the shared JS module. Instead, provide a fake/native inspector fallback from C/C++.

14.1.2 Dependencies

Depends on `001_shared_runtime_rollback.md`, because `lib/inspector.js` must stay donor-clean.

14.1.3 Current Implementation

- `src/internal_binding/dispatch.cc` now resolves `internalBinding("inspector")` to a native fallback object.
- `src/edge_module_loader.cc` special-cases public builtin require of `inspector / node:inspector` so `require("inspector")` returns the native fallback while `internalBinding("config").hasInspector` remains false.
- Keeping `hasInspector` false is important: setting it true activates broad bootstrap inspector paths such as console wrapping and async inspector hooks.

14.1.4 Verification

This passed:

```
./build-edge-quickjs-cli/edge -e "const inspector=require('inspector'); console.log(typeof inspector.Session"
```

Expected output shape:

```
function undefined
ERR_INSPECTOR_NOT_AVAILABLE
ERR_INSPECTOR_NOT_AVAILABLE
```

Also passed:

```
./build-edge-quickjs-cli/edge -e "console.log(process.features.inspector, internalBinding('config').hasInsp"
```

Expected:

```
false false
```

14.1.5 Review Findings

- `require("inspector")` and `require("node:inspector")` return the same native fallback object, and `internalBinding("config").hasInspector / process.features.inspector` remain false.
- The native public fallback is intentionally smaller than upstream `lib/inspector.js`: `new inspector.Session()` does not inherit from `EventEmitter`, so passive code can import the module and active APIs report unavailable, but consumers that attach listeners before calling `connect()` will see `typeof session.on === "undefined"`.

- Builtin metadata still reports inspector in `internalBinding("builtins").builtinCategories.cannotBeRequired` even though public `require("inspector")` now succeeds through the native loader special-case. That is a contract mismatch to resolve or explicitly document if any tests or embedders consume the category metadata.

Lightweight probes run during review:

```
./build-edge-quickjs-cli/edge -e "const a=require('inspector'); const b=require('inspector'); const c=require('inspector'); console.log(a, b, c);"
./build-edge-quickjs-cli/edge -e "const p=require('inspector/promises'); console.log(typeof p.Session, type"
./build-edge-quickjs-cli/edge -e "const inspector=require('inspector'); const s=new inspector.Session(); co"
./build-edge-quickjs-cli/edge -e "const cats=internalBinding('builtins').builtinCategories; console.log(cats"
```

14.1.6 Ownership

Code ownership for this subtask is `src/internal_binding/dispatch.cc` and the inspector special-case in `src/edge_module_loader.cc`.

Chapter 15

Intl fallback module

Source note: `development/dev_001_pr_cleanup_containment/003_intl_fallback_module.md`

15.1 Intl fallback module

15.1.1 Scope

Address Syrus's review comment on `src/edge_runtime.cc`: move the minimal `Intl.DateTimeFormat` fallback into its own module named `edge_intl_fallback.cc`, and implement it with N-API instead of raw JavaScript evaluation.

15.1.2 Current Implementation

- Added `src/edge_intl_fallback.h`.
- Added `src/edge_intl_fallback.cc`.
- Added the source file to `edge_runtime` in `CMakeLists.txt`.
- Replaced the old `InstallMinimalIntlFallback(...)` string-eval block in `src/edge_runtime.cc` with `EdgeInstallMinimalIntlFallback(...)`.

15.1.3 Intended Behavior

If a real `globalThis.Intl.DateTimeFormat` function exists, leave it untouched. Otherwise install a deliberately minimal fallback that supports:

- `new Intl.DateTimeFormat(locales, options)`
- `.format(value)`
- `.resolvedOptions()`
- `Intl.DateTimeFormat.supportedLocalesOf()`

This fallback is only meant to unblock framework bootstrap formatting, not to claim full ECMA-402 compatibility.

15.1.4 Verification Needed

After rebuild, run:

```
./build-edge-quickjs-cli/edge -e "const f = new Intl.DateTimeFormat([], { hour: '2-digit', minute: '2-digit'
```

Expected shape:

string 2-digit

15.1.5 Ownership

Code ownership for this subtask is `src/edge_intl_fallback.{h,cc}`, `src/edge_runtime.cc`, and the `edge_runtime` source list in `CMakeLists.txt`.

15.1.6 Review Notes

2026-05-07 source review, with no source edits:

- The fallback has been split into `src/edge_intl_fallback.{h,cc}` and wired into the `edge_runtime` target.
- `src/edge_runtime.cc` now delegates to `EdgeInstallMinimalIntlFallback(...)` instead of keeping a local string-eval fallback block.
- The fallback implementation uses N-API calls to inspect/install `globalThis.Intl.DateTimeFormat`; no raw JavaScript evaluation appears in `src/edge_intl_fallback.cc`.
- The implemented minimal behavior covers the current subtask's documented new `Intl.DateTimeFormat(locales, options)` path, `.format(value)`, `.resolvedOptions()`, and static `supportedLocalesOf()`.
- Caveat: the earlier Astro troubleshooting note described the fallback as "callable/constructible", but this subtask's intended behavior only names the new `Intl.DateTimeFormat(...)` constructor path. If call-without-new compatibility is part of the review ask, it should be verified separately because the current N-API class constructor path is not explicitly shaped for that behavior.

Chapter 16

Compile-time trace diagnostics

Source note: development/dev_001_pr_cleanup_containment/004_trace_diagnostics_macro.md

16.1 Compile-time trace diagnostics

16.1.1 Scope

Preserve the native C/C++ tracing that helped diagnose QuickJS TTY, HTTP, stream, module, and bootstrap issues, but make it compile-time disableable so production builds can compile the trace checks out.

16.1.2 Current Implementation

- Added `src/edge_trace.h` for Edge runtime code.
- Added `napi/quickjs/src/internal/quickjs_trace.h` for QuickJS N-API code.
- Added `EDGE_ENABLE_TRACE_DIAGNOSTICS=${IF:${CONFIG:Debug},1,0}` to `edge_runtime` and `napi_quickjs` compile definitions.
- Converted `std::getenv("EDGE_TRACE_*") != nullptr` trace checks to `EDGE_TRACE_ENABLED("EDGE_TRACE_*")`.

16.1.3 Trace Families

Known trace switches currently covered:

- `EDGE_TRACE_BOOTSTRAP`
- `EDGE_TRACE_BUILTINS`
- `EDGE_TRACE_INTERNAL_BINDING`
- `EDGE_TRACE_NET`
- `EDGE_TRACE_QUICKJS_CONTEXTIFY`
- `EDGE_TRACE_QUICKJS_MODULES`
- `EDGE_TRACE_TTY`

16.1.4 Verification Needed

After rebuild, check that there are no raw `std::getenv("EDGE_TRACE_...")` checks left in `src/` or `napi/quickjs/src/`:

```
rg -n 'std::getenv\("EDGE_TRACE|EDGE_TRACE_[A-Z_]+\") != nullptr' src napi/quickjs/src
```

Also confirm a Release-style build compiles with `EDGE_ENABLE_TRACE_DIAGNOSTICS` set to `0` and the QuickJS smoke tests still pass.

16.1.5 Ownership

Code ownership for this subtask spans trace call sites in `src/` and `napi/quickjs/src/`, plus the new trace headers and CMake compile definitions.

16.1.6 Review 2026-05-07

Lightweight read-only review checked:

```
rg -n "EDGE_ENABLE_TRACE_DIAGNOSTICS|EDGE_TRACE_ENABLED|EDGE_TRACE_[A-Z_]+|std::getenv\(|getenv\" CMakeLists
rg -n "getenv\\(\\s*\"EDGE_TRACE|std::getenv\\(\\s*\"EDGE_TRACE\" . --glob '!plans/**' --glob '!build*/**' --glob
rg -n "#include .*edge_trace|#include .*quickjs_trace|EDGE_TRACE_ENABLED" src napi/quickjs/src
```

Findings:

- No raw `std::getenv("EDGE_TRACE_*") / getenv("EDGE_TRACE_*")` call sites were found outside the trace helper headers.
- Current `EDGE_TRACE_ENABLED(...)` callers include `edge_trace.h` or `internal/quickjs_trace.h` directly in the same translation unit.
- The reviewed CMake files define `EDGE_ENABLE_TRACE_DIAGNOSTICS` for `edge_runtime` and `napi_quickjs`; `edge_cli.cc` uses the header fallback rather than a target-specific compile definition.
- Full Release-style compile verification was not run in this review because this pass was intentionally limited to lightweight read-only commands.

Chapter 17

Verification and remaining cleanup

Source note: development/dev_001_pr_cleanup_containment/005_verification.md

17.1 Verification and remaining cleanup

17.1.1 Scope

Tie together the rollback, native inspector fallback, Intl fallback split, and compile-time trace diagnostics.

17.1.2 Current Build State

The focused rebuilds have passed after the structured-clone API split and the QuickJS WASIX NAPI_EXTERN= target fix:

```
make build
make build-edge-quickjs-cli
cd $sim$/src/dev/edgejs/quickjs-wasm && ./build.sh
```

The WASIX build produced build-quickjs-wasix/edge.wasm and edgejs.wasm, and the script's final no-N-API-imports check passed.

17.1.3 Required Smoke Tests

Run after rebuild:

```
./build-edge-quickjs-cli/edge --version
./build-edge-quickjs-cli/edge -e "console.log('hello from quickjs')"
./build-edge-quickjs-cli/edge -e "const inspector=require('inspector'); console.log(typeof inspector.Session)"
./build-edge-quickjs-cli/edge -e "const f = new Intl.DateTimeFormat([], { hour: '2-digit', minute: '2-digit'})"
```

Also rerun the donor-tree comparisons from 001_shared_runtime_rollback.md.

When touching targets that include N-API headers and link into the embedded QuickJS WASIX binary, verify the target gets NAPI_EXTERN= for EDGE_NAPI_PROVIDER=quickjs; otherwise the final wasm link can fail with napi versus env import module mismatches.

17.1.4 Known Unrelated Dirty State

wasmer.toml was already modified before this cleanup thread. Do not revert it unless explicitly asked.

Part IV

Troubleshooting Registry

The registry chapter is the table of contents for issue-level investigations across Astro SSR, Vite, Next.js, and Wasmer deployment.

Chapter 18

Edge QuickJS framework troubleshooting

Source note: `development/troubleshooting/index.md`

18.1 Edge QuickJS framework troubleshooting

		Remarks
Status	Active	Active registry for framework troubleshooting issue notes.
Severity	Low	Documentation registry only; it does not block runtime behavior.

This registry lives under the QuickJS WASIX development notes so framework compatibility work stays attached to the development timeline. Use it to find existing plans and findings before changing QuickJS runtime code or framework adapters.

18.1.1 Workflow

For every new Astro SSR, Vite app, Next app, or Wasmer deploy troubleshooting issue:

- write an issue-specific plan in the matching subdirectory before changing code;
- name issue files with a numeric prefix, for example `001_<issue_name>.md`;
- keep the plan focused on one observed failure or behavior;
- update the most recent plan pointer in the repo root `AGENTS.md`;
- rerun the targeted reproduction before broadening the fix.

18.1.2 Astro SSR

For each new Astro SSR troubleshooting issue, create a plan under `astro-ssr/` before changing code.

18.1.2.1 Active `001_es_module_lexer_webassembly.md`: `es-module-lexer` `WebAssembly` import

Why: the Astro SSR native ESM entry resolved `es-module-lexer` to its default WASM-backed build, which expects `globalThis.WebAssembly` and fails under the QuickJS runtime.

What to investigate: QuickJS-only resolver compatibility for the bare `es-module-lexer` specifier so it can resolve to the pure JS export already known to work.

18.1.2.2 Done `002_depd_callsite_methods.md`: `depd` `CallSite` method compatibility

Why: after moving past the `es-module-lexer` issue, Astro SSR reached `depd`, which expects Node/V8-style structured stack `CallSite` methods.

What was fixed: QuickJS stack construction now honors the public `Error.prepareStackTrace` property, and the native `CallSite` prototype exposes the Node/V8-compatible methods needed by `depd`.

18.1.2.3 Done 003_cjs_reexport_named_exports.md: CommonJS re-export named exports

Why: React's public CommonJS entry delegates to another file, so QuickJS's synthetic ESM facade did not declare named exports such as `createElement` before module linking.

What was fixed: QuickJS now discovers conservative recursive CommonJS re-export names in `quickjs_cjs_exports.cc`, declares those names before ESM linking, and assigns the evaluated `require(...)` properties onto the synthetic facade. This exists because Node's V8 path already has native `ModuleWrap/loader` integration; QuickJS needs an equivalent compatibility bridge until more of that flow can be delegated back to Node's JS loaders.

18.1.2.4 Done with caveats 004_missing_intl.md: missing Intl global

Why: after the `depd` `CallSite` fix, the Astro SSR entry reaches the generated Astro server adapter chunk, which creates new `Intl.DateTimeFormat(...)` during module evaluation.

What was fixed: EdgeJS now installs a deliberately minimal runtime-level `Intl.DateTimeFormat` fallback when no real implementation exists. It covers simple framework bootstrap time formatting and is not a full ECMA-402 Intl implementation.

18.1.2.5 Done with caveats 005_listen_eperm.md: listen EPERM on localhost

Why: after the minimal Intl fallback, the Astro SSR entry reaches server startup and logs `listen EPERM: operation not permitted ::1:4321`.

What to investigate: whether the listen failure is sandbox policy, app configuration, or an EdgeJS TCP/listen runtime behavior difference.

18.1.2.6 Done 006_floating_ui_utils_dom.md: Floating UI utils DOM subpath

Why: once the Astro server starts and a browser requests `/`, route rendering fails because QuickJS cannot load `@floating-ui/utils/dom`.

What was fixed: QuickJS package subpath resolution now keeps trying runtime conditions when a nested condition key such as `types` does not resolve to a file, allowing `@floating-ui/utils/dom` to reach its nested default import target.

18.1.2.7 Done 007_react_remove_scroll_bar_constants.md: React Remove Scroll Bar constants subpath

Why: after the Floating UI subpath fix, route rendering reaches `react-remove-scroll-bar/constants` and QuickJS cannot load that package subpath.

What was fixed: QuickJS package subpath resolution now tries a subpath directory's own `package.json` entry metadata after parent exports do not resolve it, matching the CommonJS-compatible shape used by this package.

18.1.2.8 Done 008_zustand_ind_create_export.md: Zustand ind create export

Why: after the React Remove Scroll Bar constants fix, route rendering reaches Zustand and QuickJS reports that the resolved `zustand/ind` module does not export `create`.

What to investigate: whether the resolved module path is truncated, whether a CommonJS re-export pattern is missing from synthetic named export discovery, or whether the generated app import is invalid.

What was fixed: QuickJS package exports scanning now handles nested condition objects, root `"."` exports, and the simple `"./"` wildcard shape so Zustand resolves to its ESM import targets.

18.1.2.9 Done 009_zustand_esm_default_export.md: Zustand ESM default export

Why: after the Zustand package exports fix, route rendering reaches a new module linking failure where an import expects a default export from the resolved `zustand/esm` module.

What to investigate: whether the generated app import is valid, whether zustand/esm directory resolution should be allowed, and whether QuickJS's export declarations match the resolved module.

What was fixed: QuickJS now canonicalizes resolved module filenames through symlinks, so pnpm symlinked packages resolve their own dependency graph from the real `.pnpm/.../node_modules` path.

18.1.2.10 Done 010_use_gesture_controller_export.md: Use Gesture Controller export

Why: after pnpm symlink canonicalization, route rendering reaches a new module linking failure where an import expects `Controller` from a truncated `.pnpm/@use-... package` path.

What to investigate: identify the full package/module path and determine whether QuickJS is missing a valid export declaration or resolving the wrong package entry.

What was fixed: QuickJS now prefers package export runtime conditions in `import`, `module`, `default` order, so bundler-oriented packages like `@use-gesture/core` resolve to their ESM module target.

18.1.2.11 Done 011_route_stack_overflow.md: Route stack overflow

Why: after the Use Gesture export fix, route rendering reaches a runtime `Maximum call stack size exceeded` failure without a detailed stack in Astro's router log.

What to investigate: capture the real thrown stack and identify whether the recursion is in module resolution, CommonJS facade evaluation, framework rendering, or a runtime polyfill.

What was fixed: Edge-created QuickJS runtimes now use a 4 MiB stack guard. The 1 MiB default was too tight for Relay's CommonJS dependency graph plus the Astro/React SSR render path; 2 MiB fixed module evaluation, and 4 MiB allowed the full / route to render.

18.1.2.12 Done 012_wasix_pnpm_symlink_resolution.md: WASIX pnpm symlink resolution

Why: the Wasmer-packaged Astro standalone entry reaches `dist/server/renderers.mjs`, but the QuickJS C++ module resolver cannot load bare `react` from the app's pnpm symlinked `/app/node_modules` tree.

What was fixed: shared QuickJS module path helpers now canonicalize symlink components during CommonJS and ESM resolution, and the fs binding retries `stat` through resolved symlink components for Node's `JS realpathSync()`.

18.1.2.13 Done 013_lucide_react_chevrondown_export.md: Lucide React ChevronDown export

Why: after the pnpm symlink fix, the Astro standalone server starts, but rendering / fails because QuickJS does not declare the `ChevronDown` named export on the synthetic ESM facade for `lucide-react`.

What was fixed: `.js` CJS/ESM classification now reads parsed package metadata and only treats a package as ESM for top-level `"type": "module"`, avoiding the false positive from `lucide-react's "repository": { "type": "git" }` plus top-level `"module"` entry.

18.1.2.14 Done 014_pnpm_deploy_externalized_runtime_links.md: pnpm deploy externalized runtime links

Why: the full `stackmachine.com` source tree runs under Wasmer, but the pruned `.deploy` artifact built from `pnpm deploy --prod --legacy` failed at startup because Astro's generated server chunks bare-imported externalized packages such as `piccolore` that existed only inside pnpm's virtual store.

What was fixed: the app's `edge:prepare-deploy` script now scans `.deploy/dist/server` for bare runtime imports and makes those packages addressable from the deploy root `node_modules`; `graphql-ws` imports it at runtime. Later Wasmer deploy packaging work materialized this graph into a symlink-free artifact.

18.1.3 Vite App

For each new Vite app troubleshooting issue, create a plan under `vite-app/` before changing code.

18.1.3.1 Done with caveats 001_standalone_build.md: standalone build findings

Why: plain Vite SPA builds do not emit an Astro-style standalone HTTP server entry, so the existing `server/router.cjs` adapter is real runtime plumbing.

What was found: `vite-plugin-standalone` may package an explicit server entry, but a plain SPA still needs server semantics from an adapter, generated entry, or different framework/runtime shape.

18.1.4 Next App

For each new Next app troubleshooting issue, create a plan under `next-app/` before changing code.

18.1.4.1 Done 001_standalone_v8_serdes.md: require("v8") / serdes findings

Why: the standard Next.js standalone server reaches `require("v8")`, and the QuickJS `internalBinding("serdes")` currently returns an empty object.

What changed: QuickJS now exports `Serializer` and `Deserializer` constructors from `internalBinding("serdes")`; native and WASIX smoke tests verified `require("v8")` and a plain object `v8.serialize() / v8.deserialize()` round-trip.

18.1.4.2 Done 002_standalone_inspector_stub.md: require("inspector") unavailable-inspector stub

Why: after the QuickJS serdes fix, the `private-poker` Next.js standalone server advances to `require("inspector")`, where `lib/inspector.js` currently throws `ERR_INSPECTOR_NOT_AVAILABLE` during module load.

What changed: real inspector support remains disabled, but passive consumers can import the public builtin and receive `no-op url()`, `close()`, and network hooks while active inspector APIs continue to report inspector-unavailable errors.

18.1.4.3 Active 003_route_stack_exhausted.md: route request stack exhaustion

Why: after the serdes and inspector startup fixes, the `private-poker` Next.js standalone server starts under Wasmer, but the first request to `/` exits with `RuntimeError: call stack exhausted`.

Plan: isolate whether the exhaustion is in HTTP dispatch, module loading, React/Next rendering, user app code, or WASIX/QuickJS stack sizing, then apply a narrow runtime fix.

18.1.5 Wasmer Deploy

For each new Wasmer deploy troubleshooting issue, create a plan under `wasmer-deploy/` before changing deploy packaging code.

18.1.5.1 Done with caveats 001_pnpm_directory_symlinks_webc.md: pnpm directory symlinks in WEBC package

Why: the prepared `.deploy` directory works with local `wasmer run --net .`, but the app deployed to `wasmer.io` exits during startup because `/app` cannot resolve bare `react` from the serialized package.

What was changed: the app deploy preparation now emits a flattened, symlink-free `.deploy` tree. It builds a runtime import closure, prunes unimported packages, materializes the remaining package links, removes `.pnpm`, rewrites virtual-store source imports, copies `wasmer.toml` and optional `app.yaml`, and validates the final artifact before `wasmer deploy`.

18.1.5.2 Done [002_quickjs_wasix_napi_import_module_mismatch.md](#): QuickJS WASIX N-API import module mismatch

Why: `quickjs-wasm/build.sh` reached the final WASM link but failed because `edge_environment_core` declared `napi_*` calls as imports from the `napi` module while other QuickJS Edge objects referenced the same symbols through `env`.

What was fixed: the QuickJS provider now compiles `edge_environment_core` with `NAPI_EXTERN=`, matching the embedded `napi_quickjs` linkage. The WASIX build now emits `edge.wasm` and `edgejs.wasm`, and the script's final no-N-API-imports check passes.

18.1.5.3 Done with caveats [003_ci_safe_mode_missing_quickjs_artifact.md](#): CI safe-mode missing QuickJS artifact

Why: `build-wasix-linux` built the legacy `build-wasix/edgejs.wasm` artifact, but the active root `wasmer.toml` points at `build-quickjs-wasix/edgejs.wasm`, so `wasmer run .` failed while loading the manifest before the safe-mode JavaScript cases could execute.

What changed: added a QuickJS WASIX Makefile target, pointed CI at it, packaged `build-quickjs-wasix` as the WASIX dist, and skipped the legacy host-N-API `napi_wasmer` smoke path in this embedded-QuickJS job. Local structural checks passed; full execution still needs the CI WASIX/Wasmer toolchain. The native `build-linux` job was also switched from V8 N-API/default Edge builds to the QuickJS N-API and QuickJS Edge build directory.

Part V

Astro SSR Compatibility Investigations

Astro SSR surfaced most of the package-graph, CommonJS/ESM facade, minimal Intl, module resolution, and deployment-layout work. The chapters here are ordered by issue number.

Chapter 19

Astro SSR: es-module-lexer WebAssembly Import

Source note: [development/troubleshooting/astro-ssr/001_es_module_lexer_webassembly.md](#)

19.1 Astro SSR: es-module-lexer WebAssembly Import

		Remarks
Status	Active	Planned runtime compatibility investigation.
Severity	High	The Astro SSR entry cannot start if the WebAssembly-dependent lexer path is selected.

19.1.1 Issue

Running the Astro SSR native ESM entry directly with the QuickJS Edge CLI fails:

```
$\sim$/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Observed error:

```
ReferenceError: WebAssembly is not defined
    at <anonymous> (.../node_modules/es-module-lexer/dist/lexer.js:2:356)
```

Node can run the same entry:

```
node ./dist/server/entry.mjs
```

19.1.2 Diagnosis

Astro's server output imports es-module-lexer.

QuickJS currently resolves the bare package import to:

```
node_modules/es-module-lexer/dist/lexer.js
```

That is the package's default ESM/WASM build. It expects `globalThis.WebAssembly`, which the QuickJS runtime does not currently expose.

The existing bundled CJS path already avoids this by aliasing:

```
'es-module-lexer': 'es-module-lexer/js'
```

The `es-module-lexer/js` export resolves to the pure JS build and has already been verified to import successfully under the QuickJS Edge CLI.

19.1.3 Plan

Implement a QuickJS-only resolver compatibility alias in:

```
napi/quickjs/src/unofficial_napi.cc
```

When resolving the bare specifier:

`es-module-lexer`

resolve it as:

`es-module-lexer/js`

This should route through the package exports["./js"] target and load:

`node_modules/es-module-lexer/dist/lexer.asm.js`

19.1.4 Constraints

- Only modify files under `napi/quickjs/`.
- Do not modify the Astro app, `node_modules`, or non-QuickJS Edge runtime code.
- Do not implement a fake `WebAssembly` global for this issue.
- Keep the alias narrow and package-specific.

19.1.5 Validation

Rebuild:

```
cmake --build build-edge-quickjs-cli --target edge -j4
```

Smoke test resolver behavior:

```
$\sim$/src/dev/edgejs/build-edge-quickjs-cli/edge \  
-e "import('es-module-lexer').then(m=>console.log(typeof m.parse))"
```

Expected output:

`function`

Then rerun the Astro SSR entry:

```
$\sim$/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

If a new failure appears, capture it as the next issue-specific plan in this directory before making further changes.

Chapter 20

Astro SSR: depd CallSite Method Compatibility

Source note: `development/troubleshooting/astro-ssr/002_depd_callsite_methods.md`

20.1 Astro SSR: depd CallSite Method Compatibility

		Remarks
Status	Done	Fixed in the vendored QuickJS CallSite and stack trace implementation.
Severity	High	Astro SSR stopped during startup when depd could not use Node-style CallSite methods.

20.1.1 Issue

After aliasing `es-module-lexer` to its pure-JS export, the Astro SSR native ESM entry advances to a new failure under the QuickJS Edge CLI:

```
$\sim$/src/dev/edgejs/build-edge-quickjs-cli/edge \  
-e "import('./dist/server/entry.mjs')"
```

Observed error:

```
TypeError: not a function  
    at callSiteLocation (<input>:270:23)  
    at depd (<input>:111:31)
```

The failure was first reproduced from:

```
$\sim$/src/dev/christoph/astro-app
```

It was later reproduced from:

```
$\sim$/src/dev/stackmachine.com
```

20.1.2 Diagnosis

The failing dependency is `depd`.

Its `callSiteLocation(...)` helper expects V8/Node-style structured stack frame objects with methods such as:

```
callSite.getFileName()  
callSite.getLineNumber()  
callSite.getColumnNumber()  
callSite.isEval()  
callSite.getEvalOrigin()  
callSite.getFunctionName()
```

The QuickJS-backed runtime had two compatibility gaps:

- Node bootstrap replaces `Error.prepareStackTrace` with a writable data property, but QuickJS stack construction still read only its hidden `ctx->error_prepare_stack` slot. Userland assignments such as `depd's temporary Error.prepareStackTrace = prepareObjectStackTrace` were ignored, so `Error.captureStackTrace(obj)` produced a string instead of `CallSite[]`.
- The native QuickJS `CallSite` prototype exposed only a small method subset. `depd` also expects methods such as `isEval()`, `getEvalOrigin()`, `getThis()`, `getTypeName()`, and `getMethodName()`.

This is separate from the earlier `es-module-lexer` WebAssembly issue, which no longer appears after the resolver alias.

The existing bundled CJS Astro path has already avoided this dependency behavior by using `edge-depd-stub.cjs`, but the native ESM SSR entry currently imports the real `depd` package.

20.1.3 Fix

Updated `quickjs/quickjs.c` so `build_backtrace(...)` reads the public `Error.prepareStackTrace` property at stack-build time, falling back to the hidden QuickJS slot only when the public property is not callable.

Also extended the native QuickJS `CallSite` prototype with conservative Node/V8-compatible methods:

```
isEval
isConstructor
isTopLevel
isAsync
isPromiseAll
getMethodName
getTypeName
getThis
getEvalOrigin
getScriptNameOrSourceURL
getPromiseIndex
toString
```

Where QuickJS does not currently track the exact V8 metadata, these methods return stable conservative values (`false`, `null`, or `undefined`) instead of throwing.

20.1.4 Constraints

- Do not modify the Astro app, `node_modules`, or generated `dist` files.
- Keep any resolver compatibility aliases narrow and package-specific.
- Compare with the V8 backend or existing bundled adapter behavior before implementing.
- Fix one behavior at a time and rerun the targeted Astro SSR import before moving to any next failure.

20.1.5 Validation

Rebuild:

```
cmake --build build-edge-quickjs-cli --target edge -j4
```

Focused `depd` check:

```
cd $sim$/src/dev/stackmachine.com
$sim$/src/dev/edgejs/build-edge-quickjs-cli/edge \
-e "try { require('depd')('x'); console.log('depd ok') } catch(e) { console.error(e && (e.stack || e.message)) }"
```

Observed result after the fix:

```
depd ok
```

Rerun the focused import:


```
cd $sim$/src/dev/stackmachine.com
$sim$/src/dev/edgejs/build-edge-quickjs-cli/edge \
./dist/server/entry.mjs
```

Observed result after the fix:

```
ReferenceError: Intl is not defined
    at $sim$/src/dev/stackmachine.com/dist/server/chunks/_@astrojs-ssr-adapter_BqW-
NUXY.mjs:734:28
```

This is a later, different Astro SSR failure captured in 004_missing_intl.md.

Chapter 21

Astro SSR: CommonJS Re-Export Named Exports

Source note: [development/troubleshooting/astro-ssr/003_cjs_reexport_named_exports.md](#)

21.1 Astro SSR: CommonJS Re-Export Named Exports

		Remarks
Status	Done	Fixed by recursive CommonJS named-export discovery and shared QuickJS module resolution helpers.
Severity	High	React named exports failed to link without this compatibility behavior.

21.1.1 Issue

The Astro standalone SSR entry for `stackmachine.com` starts under native Node and native V8 EdgeJS, but fails under native QuickJS EdgeJS:

```
$\sim$/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Focused reproduction:

```
cd $\sim$/src/dev/stackmachine.com
$\sim$/src/dev/edgejs/build-edge-quickjs-cli/edge \
  -e "import('./dist/server/entry.mjs').catch(e=>{ console.error(e && e.message); process.exitCode=1; })"
```

Observed error:

```
Could not find export 'createElement' in module
'$\sim$/src/dev/stackmachine.com/node_modules/react/index'
```

21.1.2 Diagnosis

The failing Astro renderer imports React as:

```
import React__default, { createElement } from 'react';
```

Node and the V8-backed Edge runtime expose CommonJS named exports on the ESM namespace for `react`. QuickJS EdgeJS creates a synthetic CommonJS module facade because QuickJS links ESM imports before a CommonJS file can execute and produce `module.exports`.

The original QuickJS facade statically scanned only the immediate wrapper file. React's public `index.js` delegates to another CommonJS file:

```
module.exports = require('./cjs/react.development.js');
```

The real `exports.createElement = ...` assignment is in the delegated file, so QuickJS declares only `default` and `module.exports` before module linking. LLDB confirmed the failure is thrown before `QuickjsCommonJsModuleInit(...)` runs, so this must be fixed in export-name declaration rather than later evaluation.

21.1.3 Why this compatibility layer exists

This is not because V8 has CommonJS and QuickJS does not. V8, like QuickJS, is a JavaScript engine; Node implements CommonJS, ESM loading, and the CJS/ESM bridge around the engine.

The difference is that Node's V8 path already has mature native `ModuleWrap`, loader, and translator integration. The QuickJS N-API backend is implementing that V8-shaped `unofficial_napi` surface over QuickJS. QuickJS asks the host C module loader to normalize and load imports during `JS_ResolveModule`, and it requires declared ESM export names before link time. CommonJS modules only know their true export object after evaluation.

The facade/export-name logic is therefore a bridge for Node compatibility:

- resolve package and file specifiers in the QuickJS host module loader;
- decide whether a `.js` file should be compiled as ESM or evaluated through the CommonJS loader;
- predeclare conservative named exports for CommonJS facades so ESM imports can link;
- after `require(...)` evaluates the CommonJS file, copy the actual properties onto the declared QuickJS module exports.

This behavior is needed until QuickJS `ModuleWrap` can delegate more of the flow back to Node's JS loaders/translators. The behavior is valid runtime plumbing; the older ad hoc JSON/package parsing and broad CJS heuristics are the pieces that should be treated as technical debt.

21.1.4 Fix

CommonJS named export discovery was moved out of `unofficial_napi.cc` into `napi/quickjs/src/quickjs_cjs_exports.cc`. The scanner preserves the direct export patterns and adds recursive literal re-export discovery for patterns such as:

```
module.exports = require('./target.js')
Object.assign(exports, require('./target.js'))
```

Use the resulting names both when declaring QuickJS C module exports and when setting those exports after `require(...)` evaluates.

Related follow-up fixes moved path/package resolution into shared helpers in `napi/quickjs/src/unofficial_module_loader.cc` and tightened `.js` CJS/ESM classification so package metadata is parsed instead of matched with broad string heuristics.

21.1.5 Constraints

- Keep the fix in the QuickJS-backed N-API/runtime path.
- Do not modify the Astro app, `node_modules`, or generated `dist` files.
- Keep the scanner conservative; do not execute CommonJS while computing link names.
- Preserve `default` and `module.exports` behavior.

21.1.6 Validation

Rebuild:

```
cmake --build build-edge-quickjs-cli --target edge -j4
```

Check the focused namespace behavior:

```
cd $sim$/src/dev/stackmachine.com
$sim$/src/dev/edgejs/build-edge-quickjs-cli/edge \
-e "import('react').then(m=>console.log(Object.prototype.hasOwnProperty.call(m, 'createElement')))"
```

Then rerun the Astro SSR entry:

```
$\sim$/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Observed current state:

- `react` named exports such as `createElement` link successfully.
- Later Astro SSR failures were captured in subsequent issue-specific notes (004 through 014) rather than folded into this issue.
- The full Astro standalone path has since reached a Wasmer-served 200 `text/html` response after the later resolver, symlink, stack, and deploy packaging fixes.

Remaining design cleanup: reduce the QuickJS host-loader facades as `ModuleWrap` support becomes closer to Node/V8, or keep the behavior but ensure all resolver and package metadata handling lives in the shared QuickJS module loader rather than local ad hoc parsers.

Chapter 22

Astro SSR: Missing Intl Global

Source note: `development/troubleshooting/astro-ssr/004_missing_intl.md`

22.1 Astro SSR: Missing Intl Global

		Remarks
Status	Done with caveats	Minimal runtime compatibility fallback implemented.
Severity	Medium	The fallback enables Astro startup but is intentionally incomplete compared with ECMA-402 Intl.

22.1.1 Issue

After the `depd` CallSite compatibility fix, the Astro standalone SSR entry for `stackmachine.com` advances to a new QuickJS runtime failure:

```
cd $sim$/src/dev/stackmachine.com
$sim$/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Observed error:

```
ReferenceError: Intl is not defined
      at $sim$/src/dev/stackmachine.com/dist/server/chunks/_@astrojs-ssr-adapter_BqW-
NUXY.mjs:734:28
```

The generated Astro chunk creates a logger timestamp formatter at module load:

```
const dateTimeFormat = new Intl.DateTimeFormat([], {
  hour: "2-digit",
  minute: "2-digit",
  second: "2-digit",
  hour12: false
});
```

22.1.2 Diagnosis

The native QuickJS Edge runtime currently exposes no `globalThis.Intl`. Astro's SSR adapter expects at least `Intl.DateTimeFormat` to exist during module evaluation.

This is separate from the `depd` stack compatibility issue: a focused `require('depd')('x')` check now succeeds, and the Astro entry reaches this later import-time failure.

22.1.3 Fix

Implemented the narrow runtime-level fallback option in `src/edge_runtime.cc`. During early runtime bootstrap, before the `process` object is installed, EdgeJS now checks for `globalThis.Intl.DateTimeFormat`. If a real implementation already exists, it is left untouched. If it is missing, EdgeJS installs a small JS fallback with:

- a callable/constructible `Intl.DateTimeFormat`;

- `format(value)` for local-time `HH:mm:ss / h:mm:ss AM|PM` output;
- `resolvedOptions()`;
- `supportedLocalesOf()`;
- `Symbol.toStringTag`.

This is intentionally not a full ECMA-402 implementation. It does not perform real locale negotiation, ICU-backed calendar selection, numbering-system formatting, or time-zone conversion. It exists to satisfy framework bootstrap paths, including Astro's SSR logger timestamp formatter, without modifying app code or generated output.

Choosing this minimal fallback keeps native QuickJS and WASIX QuickJS moving while leaving the door open for a future real Intl provider.

22.1.4 Constraints

- Do not modify the Astro app, `node_modules`, or generated `dist` files.
- Fix one behavior at a time and rerun the targeted Astro SSR entry before moving to any next failure.
- Avoid pretending to implement full ECMA-402 Intl unless the implementation is actually backed by a real formatter.

22.1.5 Validation

Rebuild:

```
cmake --build build-edge-quickjs-cli --target edge -j4
```

Focused Intl check:

```
$\sim$/src/dev/edgejs/build-edge-quickjs-cli/edge \
-e "const f = new Intl.DateTimeFormat([], { hour: '2-digit', minute: '2-digit', second: '2-digit', hour12: true })"
```

Observed result:

```
string
```

Then rerun the Astro SSR entry:

```
cd $\sim$/src/dev/stackmachine.com
$\sim$/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Expected result for this issue: the `Intl is not defined` failure disappears.

Observed result after the fix: Astro reaches a later runtime/server issue and prints timestamped logger output, proving the fallback is being used:

```
13:03:14 [ERROR] [@astrojs/node] Unhandled rejection while rendering undefined
Error: listen EPERM: operation not permitted ::1:4321
```

The later listen failure is captured in `005_listen_eperm.md`.

Chapter 23

Astro SSR: Listen EPERM On Localhost

Source note: development/troubleshooting/astro-ssr/005_listen_eperm.md

23.1 Astro SSR: Listen EPERM On Localhost

		Remarks
Status	Done with caveats	Resolved as sandbox bind policy, not a QuickJS runtime bug. The server works when localhost binding is allowed; this is a local verification constraint.
Severity	Low	

23.1.1 Issue

After the minimal Intl.DateTimeFormat fallback, the Astro standalone SSR entry for stackmachine.com advances past module evaluation and reaches server startup:

```
cd $sim$/src/dev/stackmachine.com
$sim$/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Observed output:

```
(node:91514) [DEP0093] DeprecationWarning: The crypto.fips is deprecated. Please use crypto.getFips()
(node:91514) [DEP0176] DeprecationWarning: fs.F_OK is deprecated, use fs.constants.F_OK instead
13:03:14 [ERROR] [@astrojs/node] Unhandled rejection while rendering undefined
Error: listen EPERM: operation not permitted ::1:4321
    at UVExceptionWithHostPort (<input>:704:5)
    at setupListenHandle (<input>:1920:25)
    at listenInCluster (<input>:1999:21)
    at <input>:2208:23
    at onlookupall (<input>:136:17) {
  code: 'EPERM',
  errno: -1,
  syscall: 'listen',
  address: '::1',
  port: 4321
}
```

The process exited with status 0 in this sandboxed run despite logging the unhandled rejection.

23.1.2 Diagnosis

This is separate from the Intl issue. The logger timestamp is formatted, so the Astro adapter has advanced to the server listen path. The failing address is IPv6 localhost (::1) on port 4321.

Focused node:net bind checks show this is caused by the current sandbox disallowing local socket binds, not by QuickJS TCP/listen behavior or Astro's default host/port selection.

Under normal sandboxed execution, native QuickJS Edge fails to listen on all tested TCP server addresses:

```
127.0.0.1 ephemeral port -> error EPERM -1 listen 127.0.0.1 undefined
::1 ephemeral port      -> error EPERM -1 listen ::1 undefined
unspecified ephemeral   -> error EPERM -1 listen 0.0.0.0 undefined
```

With the same QuickJS Edge binary allowed to run outside the sandbox, the same checks succeed:

```
127.0.0.1 ephemeral port -> listening 127.0.0.1 IPv4 <port>; closed
::1 ephemeral port      -> listening ::1 IPv6 <port>; closed
unspecified ephemeral   -> listening :: IPv6 <port>; closed
```

The available V8-backed Edge binary shows the same pattern: sandboxed 127.0.0.1 and ::1 binds fail with EPERM, while both succeed outside the sandbox. That comparison rules out a QuickJS-specific TCP bind regression.

23.1.3 Plan

Investigate with the narrowest server-listen repro before changing runtime code:

- ☑ run a tiny `node:net` server under native QuickJS Edge on 127.0.0.1, ::1, and an ephemeral port;
- ☑ compare the same repro under native V8 EdgeJS if available;
- ☑ rerun the Astro entry outside the sandbox to verify it reaches listen;
- ☑ only change Edge TCP/listen behavior if the focused repro shows a runtime bug rather than sandbox policy or application configuration.

No runtime code change is needed for this issue.

23.1.4 Constraints

- Do not modify the Astro app, `node_modules`, or generated `dist` files.
- Keep the investigation focused on listen/bind behavior, not warnings from `crypto.fips` or `fs.F_OK`.
- If a command fails due to sandboxed network permissions, rerun the important repro with explicit escalation instead of inferring a runtime bug.

23.1.5 Validation

Focused local listen check:

```
$\sim$src/dev/edgejs/build-edge-quickjs-cli/edge \
-e "const net=require('node:net'); const s=net.createServer(); s.listen(0, '127.0.0.1', () => { console.l
```

Then rerun the Astro SSR entry:

```
cd $\sim$src/dev/stackmachine.com
$\sim$src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Expected result for this issue: the Astro standalone server can bind or the failure is clearly attributed to sandbox permissions/app configuration rather than QuickJS runtime behavior.

Observed validation:

sandboxed Astro entry:

```
Error: listen EPERM: operation not permitted ::1:4321
```

escalated Astro entry:

```
[@astrojs/node] Server listening on http://localhost:4321
```

Decision: treat `listen EPERM` as sandbox policy in the current Codex execution environment. Continue Astro SSR compatibility work from the server-listening state by running the entry with local bind permission when server startup is the behavior under test.

Chapter 24

Astro SSR: Floating UI Utils DOM Subpath

Source note: development/troubleshooting/astro-ssr/006_floating_ui_utils_dom.md

24.1 Astro SSR: Floating UI Utils DOM Subpath

		Remarks
Status	Done	Fixed in QuickJS package exports subpath resolution.
Severity	High	The Astro route could not render while this dependency subpath failed to load.

24.1.1 Issue

After the Astro standalone SSR server starts successfully outside the Codex local-bind sandbox, opening the site in Firefox triggers a route render failure:

```
cd $sim$/src/dev/stackmachine.com
$sim$/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Observed server output:

```
(node:15190) [DEP0093] DeprecationWarning: The crypto.fips is deprecated. Please use crypto.getFips()
(node:15190) [DEP0176] DeprecationWarning: fs.F_OK is deprecated, use fs.constants.F_OK instead
13:10:51 [@astrojs/node] Server listening on http://localhost:4321
13:11:02 [ERROR] [router] Error while trying to render the route /
13:11:02 [ERROR] ReferenceError: could not load module '@floating-ui/utils/dom'
    at runMicrotasks (native)
    at processTicksAndRejections (<input>:105:5)
```

24.1.2 Diagnosis

This is separate from the listen issue. The server starts and the failure appears only when a request renders /.

The failing specifier is the package subpath:

@floating-ui/utils/dom

QuickJS throws ReferenceError: could not load module ... from its module load callback when it cannot resolve/load a module.

The package metadata is valid Node-style package exports:

```
"./dom": {
  "import": {
    "types": "./dist/floating-ui.utils.dom.d.mts",
    "default": "./dist/floating-ui.utils.dom.mjs"
  },
  "types": "./dist/floating-ui.utils.dom.d.ts",
  "module": "./dist/floating-ui.utils.dom.esm.js",
```

```
"default": "./dist/floating-ui.utils.dom.umd.js"
}
```

The QuickJS resolver's string scanner found the `import` condition but then picked the nested key name `types` as though it were a target. It tried to resolve a file named `types` and stopped before trying the nested `default` runtime target.

24.1.3 Fix

Updated the QuickJS package resolver so `TryResolvePackageSubpath(...)` does not stop after the first condition string if that candidate does not resolve to a runtime file. The current shared implementation lives in `napi/quickjs/src/unofficial_module_loader.cc` and tries runtime condition targets in order, returning the first target that resolves.

This keeps the fix narrow: it does not add a full JSON parser or a complete Node package exports implementation, but it handles the nested condition shape that exposed this issue.

24.1.4 Constraints

- Do not modify the Astro app, `node_modules`, or generated `dist` files.
- Keep the fix in EdgeJS/QuickJS runtime code if a runtime fix is required.
- Fix one behavior at a time and rerun the focused import before rerendering the Astro route.

24.1.5 Validation

Focused import check:

```
cd $sim$/src/dev/stackmachine.com
$sim$/src/dev/edgejs/build-edge-quickjs-cli/edge \
  -e "import('@floating-ui/utils/dom').then(m=>console.log('loaded', Object.keys(m).length)).catch(e=>{ con
```

Observed result after the fix:

```
loaded 20
```

Then rerun the server and request /:

```
cd $sim$/src/dev/stackmachine.com
$sim$/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
```

Expected result for this issue: the `@floating-ui/utils/dom` module loads, or the failure is narrowed to a different concrete module/runtime behavior and captured in a follow-up note.

Observed route result after the fix: the Floating UI error disappears and route rendering reaches a later module resolution failure:

```
ReferenceError: could not load module 'react-remove-scroll-bar/constants'
```

The later failure is captured in `007_react_remove_scroll_bar_constants.md`.

Chapter 25

Astro SSR: React Remove Scroll Bar Constants Subpath

Source note: `development/troubleshooting/astro-ssr/007_react_remove_scroll_bar_constants.md`

25.1 Astro SSR: React Remove Scroll Bar Constants Subpath

		Remarks
Status	Done	Fixed in QuickJS package subpath directory entry resolution.
Severity	High	The Astro route could not render until this package subpath resolved correctly.

25.1.1 Issue

After the `@floating-ui/utils/dom` package exports subpath fix, the Astro SSR server on `stackmachine.com` starts and the `/` route advances to a new module load failure:

```
13:16:26 [ERROR] [router] Error while trying to render the route /
13:16:26 [ERROR] ReferenceError: could not load module 'react-remove-scroll-bar/constants'
    at runMicrotasks (native)
    at processTicksAndRejections (<input>:105:5)
```

Focused route validation was run against a temporary server on port 4322 because port 4321 was already occupied by another server instance.

25.1.2 Diagnosis

This is separate from the Floating UI issue: the focused `@floating-ui/utils/dom` import now loads successfully, and the route reaches a later package subpath.

The failing specifier is:

`react-remove-scroll-bar/constants`

The package publishes `constants` as a subdirectory with its own `package.json`:

```
{
  "main": "../dist/es5/constants.js",
  "module": "../dist/es2015/constants.js"
}
```

Native CommonJS resolution accepts this shape:

```
require.resolve('react-remove-scroll-bar/constants')
=> ../node_modules/react-remove-scroll-bar/dist/es5/constants.js
```

Native ESM rejects the same bare directory import with `ERR_UNSUPPORTED_DIR_IMPORT`, so this is a CommonJS-compatible package subpath case rather than strict ESM package resolution.

The QuickJS resolver already had permissive CommonJS-style package resolution for bare package entries and subpaths, but its directory handling only tried `index` files. It did not inspect a package subpath directory's own `package.json`, so it missed this published entrypoint.

25.1.3 Fix

Updated the QuickJS package resolver so `TryResolvePackageSubpath(...)` tries a subpath directory's own package entry metadata when parent package exports do not resolve the subpath. The current shared implementation lives in `napi/quickjs/src/unofficial_module_loader.cc`. This keeps the compatibility fallback narrow:

- parent package exports still win first;
- only actual directory subpaths get the nested package entry fallback;
- normal file and `index` fallback behavior remains unchanged.

25.1.4 Plan

Investigated with narrow import checks before changing runtime code:

- inspected `react-remove-scroll-bar` package metadata and files in `$\sim$/src/dev/stackmachine.com/node_modules`;
- ran a focused QuickJS Edge import check for `react-remove-scroll-bar/constants`;
- compared against native Node CommonJS and ESM resolution;
- updated the QuickJS resolver for the CommonJS-compatible subpath directory entrypoint case.

25.1.5 Constraints

- Do not modify the Astro app, `node_modules`, or generated `dist` files.
- Keep any fix in EdgeJS/QuickJS runtime code if a runtime fix is required.
- Fix one behavior at a time and rerun the focused import before rerendering the Astro route.

25.1.6 Validation

Focused import check:

```
cd $\sim$/src/dev/stackmachine.com
$\sim$/src/dev/edgejs/build-edge-quickjs-cli/edge \
  -e "import('react-remove-scroll-bar/constants').then(m=>console.log('loaded', Object.keys(m).length)).cat"
```

Then rerun the server and request `/`:

```
cd $\sim$/src/dev/stackmachine.com
PORT=4322 $\sim$/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
curl -i http://localhost:4322/
```

Expected result for this issue: the `react-remove-scroll-bar/constants` module loads, or the failure is narrowed to a different concrete module/runtime behavior and captured in a follow-up note.

Observed focused import result after the fix:

```
loaded 4
string string
```

With module tracing enabled, the resolver selected:

```
$\sim$/src/dev/stackmachine.com/node_modules/react-remove-scroll-bar/dist/es2015/constants.js
```

Chapter 26

Astro SSR: Zustand Ind Create Export

Source note: [development/troubleshooting/astro-ssr/008_zustand_ind_create_export.md](#)

26.1 Astro SSR: Zustand Ind Create Export

		Remarks
Status	Done	Fixed in QuickJS package exports condition and wildcard resolution. The Astro route could not link because Zustand resolved to a module without the required named export.
Severity	High	

26.1.1 Issue

After the `react-remove-scroll-bar/constants` package subpath directory fix, the Astro SSR server on `stack-machine.com` starts and the `/` route advances to a new module linking failure:

```
13:20:43 [ERROR] [router] Error while trying to render the route /
13:20:43 [ERROR] SyntaxError: Could not find export 'create' in module '$\sim$/src/dev/stackmachine.com/node_modules/zustand/index.d.mts'
    at runMicrotasks (native)
    at processTicksAndRejections (<input>:105:5)
```

Focused route validation was run against a temporary server on port 4322 because port 4321 was already occupied by another server instance.

26.1.2 Diagnosis

This is separate from the scroll-bar constants issue: the focused `react-remove-scroll-bar/constants` import now loads successfully, and the route reaches a later module export mismatch.

The failing resolved module path is:

```
$\sim$/src/dev/stackmachine.com/node_modules/zustand/index.d.mts
```

Zustand's package metadata exposes the ESM entry via nested package exports conditions:

```
"." : {
  "import": {
    "types": "./esm/index.d.mts",
    "default": "./esm/index.mjs"
  }
},
"./*" : {
  "import": {
    "types": "./esm/*.d.mts",
    "default": "./esm/*.mjs"
  }
}
```

Native ESM resolves `import('zustand')` to:

```
file:///sim$/src/dev/stackmachine.com/node_modules/zustand/esm/index.mjs
```

Before this fix, QuickJS resolved the same import to:

```
$\sim$/src/dev/stackmachine.com/node_modules/zustand/index.js
```

That CommonJS file re-exports values dynamically through `Object.keys(...).forEach(...)`, a pattern the synthetic named export scanner does not currently declare. The missing `create` export was therefore caused by resolving the package to its CommonJS fallback instead of the ESM import target.

26.1.3 Fix

Updated the QuickJS package exports scanner so it handles the Zustand export shape. The current shared implementation lives in `napi/quickjs/src/unofficial_module_loader.cc`:

- when a condition value is an object, choose its nested `default` runtime target instead of treating nested metadata keys like `types` as targets;
- resolve the root `"."` package export before falling back to `main`;
- support the simple wildcard export key `"./*"` and substitute the requested subpath into targets such as `"./esm/*.mjs"`.

This keeps the fix focused on published package exports metadata rather than adding synthetic named export support for broader dynamic CommonJS patterns.

The considered causes were:

- truncated or extensionless module resolution;
- CommonJS synthetic named export discovery missing a re-export pattern;
- ESM facade generation for a CommonJS entrypoint;
- or a genuinely invalid import emitted by the app build.

The actual cause was package export condition/wildcard resolution selecting the CommonJS fallback.

26.1.4 Plan

Investigated with the narrowest checks before changing runtime code:

- inspected Zustand package metadata and files in `$\sim$/src/dev/stackmachine.com/node_modules`;
- reproduced the issue with focused `import('zustand')` and `import('zustand/react')` probes;
- compared against native Node ESM resolution for the same specifier and import shape;
- updated the QuickJS resolver only because the package exposes a valid Node-compatible path/export shape that QuickJS misses.

26.1.5 Constraints

- Do not modify the Astro app, `node_modules`, or generated `dist` files.
- Keep any fix in EdgeJS/QuickJS runtime code if a runtime fix is required.
- Fix one behavior at a time and rerun the focused import before rerendering the Astro route.

26.1.6 Validation

Focused import check:

```
cd $sim$/src/dev/stackmachine.com
EDGE_TRACE_QUICKJS_MODULES=1 \
  $sim$/src/dev/edgejs/build-edge-quickjs-cli/edge \
  -e "import('zustand').then(m=>{ console.log('keys', Object.keys(m).join(',')); console.log('create', type
```

Observed result after the fix:

```
quickjs-module normalize ... spec=zustand -> ${sim$}/src/dev/stackmachine.com/node_modules/zustand/esm/index.m
quickjs-module normalize ... spec=zustand/vanilla -> ${sim$}/src/dev/stackmachine.com/node_modules/zustand/esm
quickjs-module normalize ... spec=zustand/react -> ${sim$}/src/dev/stackmachine.com/node_modules/zustand/esm/r
keys create,createStore,useStore
create function
```

Regression checks for the two previous module resolution fixes still pass:

```
@floating-ui/utils/dom -> .../node_modules/@floating-ui/utils/dist/floating-ui.utils.dom.mjs
react-remove-scroll-bar/constants -> .../node_modules/react-remove-scroll-
bar/dist/es2015/constants.js
```

Then rerun the server and request /:

```
cd ${sim$}/src/dev/stackmachine.com
PORT=4322 ${sim$}/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
curl -i http://localhost:4322/
```

Expected result for this issue: the module exposes create, or the failure is narrowed to a different concrete module/runtime behavior and captured in a follow-up note.

Chapter 27

Astro SSR: Zustand ESM Default Export

Source note: [development/troubleshooting/astro-ssr/009_zustand_esm_default_export.md](#)

27.1 Astro SSR: Zustand ESM Default Export

		Remarks
Status	Done	Fixed in QuickJS module path canonicalization for pnpm symlinks. The Astro route linked the wrong package instance until symlinked module paths were canonicalized.
Severity	High	

27.1.1 Issue

After the Zustand package exports condition/wildcard fix, the Astro SSR server on [stackmachine.com](#) starts and the / route advances to a new module linking failure:

```
13:24:35 [ERROR] [router] Error while trying to render the route /  
13:24:35 [ERROR] SyntaxError: Could not find export 'default' in module '$\sim$/src/dev/stackmachine.com/node_modules/zustand/esm/index.mjs'  
    at runMicrotasks (native)  
    at processTicksAndRejections (<input>:105:5)
```

Focused route validation was run against a temporary server on port 4322 because port 4321 was already occupied by another server instance.

27.1.2 Diagnosis

This is separate from the earlier missing create export: after the 008 fix, focused `import('zustand')` resolves to the ESM entry and exposes create.

The new failing resolved module path is:

```
$\sim$/src/dev/stackmachine.com/node_modules/zustand/esm
```

The import site that exposed the failure is in `@react-three/fiber`:

```
import create from 'zustand';
```

The app uses a pnpm-style dependency layout where `node_modules/@react-three/fiber` is a symlink into `.pnpm/...`. Native Node resolves the module filename through that symlink before resolving nested dependencies, so `@react-three/fiber` finds its compatible Zustand version:

```
.../node_modules/.pnpm/zustand@3.7.2_react@18.3.1/node_modules/zustand/esm/index.mjs
```

Before this fix, QuickJS preserved the symlinked module path:

```
.../node_modules/@react-three/fiber/dist/react-three-fiber.esm.js
```

Its nested import `'zustand'` then climbed the wrong `node_modules` tree and selected top-level Zustand 5:

```
.../node_modules/zustand/esm/index.mjs
```


Zustand 5 has named exports such as `create` but no default export, so the default import from `@react-three/fiber` failed during module linking.

27.1.3 Fix

Updated the QuickJS module path helpers so resolved module filenames are canonicalized with `std::filesystem::weakly_canonical(...)` before they are returned to QuickJS. The current shared implementation lives in `napi/quickjs/src/unofficial_module_loader.cc`. That makes later relative and package resolution use the real pnpm package path, matching Node's dependency lookup behavior for symlinked packages.

The considered causes were:

- an invalid generated import that expects a default export where none exists;
- directory resolution selecting `esm/index.mjs` for a specifier Node rejects;
- or an import-facade/export declaration mismatch in QuickJS.

The actual cause was non-canonical symlink paths causing package resolution to choose the wrong installed dependency version.

27.1.4 Plan

Investigated with narrow checks before changing runtime code:

- found the dependency import site that requests `default` from `zustand`;
- inspected top-level Zustand 5 and the compatible dependency-scoped Zustand 3 package;
- compared native Node and QuickJS behavior for `import('@react-three/fiber')`;
- updated runtime code because the app is using a valid Node-compatible resolution/export shape that QuickJS mishandles.

27.1.5 Constraints

- Do not modify the Astro app, `node_modules`, or generated `dist` files.
- Keep any fix in EdgeJS/QuickJS runtime code if a runtime fix is required.
- Fix one behavior at a time and rerun the focused import before rerendering the Astro route.

27.1.6 Validation

Focused import check:

```
cd $sim$/src/dev/stackmachine.com
EDGE_TRACE_QUICKJS_MODULES=1 \
  $sim$/src/dev/edgejs/build-edge-quickjs-cli/edge \
  -e "import('@react-three/fiber').then(m=>console.log('fiber loaded', Object.keys(m).length))"
```

Observed result after the fix:

```
quickjs-module normalize ... spec=@react-three/fiber -> .../node_modules/.pnpm/@react-three+fiber.../node_modules/@react-three/fiber/dist/react-three-fiber.esm.js
quickjs-module normalize ... spec=zustand -> .../node_modules/.pnpm/zustand@3.7.2_react@18.3.1/node_modules/zustand
fiber loaded 31
```

Then rerun the server and request /:

```
cd $sim$/src/dev/stackmachine.com
PORT=4322 $sim$/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
curl -i http://localhost:4322/
```

Expected result for this issue: the default export mismatch is explained or fixed, or the failure is narrowed to a different concrete module/runtime behavior and captured in a follow-up note.

Chapter 28

Astro SSR: Use Gesture Controller Export

Source note: `development/troubleshooting/astro-ssr/010_use_gesture_controller_export.md`

28.1 Astro SSR: Use Gesture Controller Export

		Remarks
Status	Done	Fixed in QuickJS package exports condition preference.
Severity	High	The Astro route could not link until the ESM module target was preferred over the CommonJS default.

28.1.1 Issue

After the pnpm symlink canonicalization fix, the Astro SSR server on `stackmachine.com` starts and the `/` route advances to a new module linking failure:

```
13:28:38 [ERROR] [router] Error while trying to render the route /
13:28:38 [ERROR] SyntaxError: Could not find export 'Controller' in module '$\sim$/src/dev/stackmachine.com/node_modules/@use-gesture/core'
    at runMicrotasks (native)
    at processTicksAndRejections (<input>:105:5)
```

Focused route validation was run against a temporary server on port 4322 because port 4321 was already occupied by another server instance.

28.1.2 Diagnosis

This is separate from the Zustand default export issue: focused `@react-three/fiber` imports now resolve through pnpm real paths and load.

The displayed module path was truncated at:

```
$\sim$/src/dev/stackmachine.com/node_modules/.pnpm/@use-
```

The full package behind the path is `@use-gesture/core`. The import site that exposed the failure is in `@use-gesture/react`:

```
import { Controller, parseMergedHandlers } from '@use-gesture/core';
```

`@use-gesture/core` publishes package exports with both `module` and `default` conditions:

```
"": {
  "module": "./dist/use-gesture-core.esm.js",
  "default": "./dist/use-gesture-core.cjs.js"
}
```

Before this fix, QuickJS preferred `default` before `module`, resolving the ESM import to the CommonJS wrapper:

```
.../node_modules/@use-gesture/core/dist/use-gesture-core.cjs.js
```

That CommonJS wrapper then required environment-specific files and did not provide the ESM export declarations expected by the importing ESM module. Native Node also resolves this package to the CJS default because the package does not declare an `import` condition, but this Astro/Vite SSR bundle uses the package's legacy `module` condition as the ESM target.

28.1.3 Fix

Updated the QuickJS package exports scanner so it prefers runtime targets in this order. The current shared implementation lives in `napi/quickjs/src/unofficial_module_loader.cc`:

```
import, module, default
```

The same order is used for legacy package entry candidates. This keeps the runtime compatible with bundler-oriented packages that expose ESM through `module` without an explicit `import` condition.

28.1.4 Plan

Investigated with narrow checks before changing runtime code:

- searched the pnpm dependency tree for modules exporting or importing `Controller`;
- identified `@use-gesture/core` behind the truncated `@use-...` path;
- compared native Node and QuickJS import behavior for the exact package;
- updated runtime package condition preference for the bundler-compatible `module` export shape.

28.1.5 Constraints

- Do not modify the Astro app, `node_modules`, or generated `dist` files.
- Keep any fix in EdgeJS/QuickJS runtime code if a runtime fix is required.
- Fix one behavior at a time and rerun the focused import before rerendering the Astro route.

28.1.6 Validation

Focused import check:

```
cd $sim$/src/dev/stackmachine.com
EDGE_TRACE_QUICKJS_MODULES=1 \
  $sim$/src/dev/edgejs/build-edge-quickjs-cli/edge \
  -e "import('@use-gesture/core').then(m=>{ console.log('core', Object.keys(m).join(',')); console.log('Con
```

Observed result after the fix:

```
quickjs-module    normalize    ...    spec=@use-gesture/core    ->    .../node_modules/@use-
gesture/core/dist/use-gesture-core.esm.js
core Controller,parseMergedHandlers
Controller function
```

`import('@use-gesture/react')` also loads and resolves `@use-gesture/core/actions`, `@use-gesture/core/utils`, and `@use-gesture/core/types` to their ESM module targets.

Then rerun the server and request `/`:

```
cd $sim$/src/dev/stackmachine.com
PORT=4322 $sim$/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
curl -i http://localhost:4322/
```

Expected result for this issue: the `Controller` export mismatch is explained or fixed, or the failure is narrowed to a different concrete module/runtime behavior and captured in a follow-up note.

Chapter 29

Astro SSR: Route Stack Overflow

Source note: `development/troubleshooting/astro-ssr/011_route_stack_overflow.md`

29.1 Astro SSR: Route Stack Overflow

		Remarks
Status	Done	Fixed with a larger QuickJS runtime stack guard.
Severity	High	The Astro route streamed a partial response and failed without the larger stack guard.

29.1.1 Issue

After the `@use-gesture/core` package export condition fix, the Astro SSR server on `stackmachine.com` starts and the `/` route advances to a runtime failure:

```
13:31:35 [ERROR] [router] Error while trying to render the route /  
13:31:35 [ERROR] Maximum call stack size exceeded
```

Focused route validation was run against temporary servers on ports 4322 and 4323 because port 4321 was already occupied by another server instance.

29.1.2 Diagnosis

This is separate from the previous module linking failures: route rendering got past the missing named/default exports and reached an actual runtime stack overflow.

The first focused reproduction showed that importing the page module failed before rendering:

```
cd $sim$/src/dev/stackmachine.com  
$sim$/src/dev/edgejs/build-edge-quickjs-cli/edge -e "import('./dist/server/pages/index.astro.mjs')"
```

The failure narrowed to the page's `stackmachine` dependency. That package loads `relay-runtime`, whose large CommonJS dependency graph exceeded QuickJS's default 1 MiB stack guard during module evaluation.

Raising the guard to 2 MiB fixed `relay-runtime`, `stackmachine`, and the page module import, but the full Astro render still overflowed in the middle of the HTML stream. Raising the Edge-created QuickJS runtime guard to 4 MiB allowed the full route render to complete.

This is not an infinite recursion in the app or in package resolution. It is a compatibility difference between the default QuickJS stack guard and the depth of modern framework/package evaluation plus React SSR rendering.

29.1.3 Plan

The runtime fix is intentionally narrow:

- keep the stack guard enabled;

- increase the default stack size only for Edge-created QuickJS environments;
- do not modify the Astro app, generated `dist`, or `node_modules`;
- keep the package-resolution fixes from previous notes unchanged.

Implemented in:

`napi/quickjs/src/unofficial_napi.cc`

The default Edge QuickJS stack guard is now `4 * 1024 * 1024` bytes.

29.1.4 Constraints

- Do not modify the Astro app, `node_modules`, or generated `dist` files.
- Keep any fix in EdgeJS/QuickJS runtime code if a runtime fix is required.
- Fix one behavior at a time and rerun the focused reproduction before rerendering the full Astro route.

29.1.5 Validation

Focused checks after the fix:

```
cd $sim$/src/dev/stackmachine.com
$sim$/src/dev/edgejs/build-edge-quickjs-cli/edge -e "const r=require('relay-runtime'); console.log('require relay 93')"
```

Result:

```
require relay 93
```

```
$sim$/src/dev/edgejs/build-edge-quickjs-cli/edge -e "import('relay-runtime').then(m=>console.log('import relay 104'))"
```

Result:

```
import relay 104
```

```
$sim$/src/dev/edgejs/build-edge-quickjs-cli/edge -e "import('stackmachine').then(m=>console.log('stackmachine StackMachine,createZip'))"
```

Result:

```
stackmachine StackMachine,createZip
```

```
$sim$/src/dev/edgejs/build-edge-quickjs-cli/edge -e "import('./dist/server/pages/index.astro.mjs').then(m=>console.log('index.astro.mjs page,renderers'))"
```

Result:

```
page page,renderers
```

Then rerun the server and request `/`:

```
cd $sim$/src/dev/stackmachine.com
PORT=4323 $sim$/src/dev/edgejs/build-edge-quickjs-cli/edge ./dist/server/entry.mjs
curl -i http://localhost:4323/
```

Result:

```
HTTP/1.1 200 OK
content-type: text/html
```

```
...
```

```
<!DOCTYPE html><html lang="en">
```

The response completed successfully and the server emitted no follow-up render error.

Chapter 30

Astro SSR: WASIX pnpm Symlink Resolution

Source note: [development/troubleshooting/astro-ssr/012_wasix_pnpm_symlink_resolution.md](#)

30.1 Astro SSR: WASIX pnpm Symlink Resolution

		Remarks
Status	Done	Fixed by shared QuickJS module path resolution and fs stat symlink fallback. The Astro standalone server cannot start in Wasmer when react cannot resolve from /app/node_modules.
Severity	High	

30.1.1 Issue

Running the stackmachine.com Astro standalone server through the Wasmer package fails before the server starts:

```
cd $sim$/src/dev/stackmachine.com
wasmer run --net .
```

Observed failure:

ReferenceError: could not load module 'react'

The failing import is in the generated Astro renderer:

```
import React__default, { createElement } from 'react';
```

30.1.2 Diagnosis

Native QuickJS can import react from the app:

```
cd $sim$/src/dev/stackmachine.com
$sim$/src/dev/edgejs/build-edge-quickjs-cli/edge \
-e "import('react').then(m=>console.log(Object.keys(m).length))"
```

The same app mounted at /app under Wasmer can see the pnpm symlink through the JavaScript fs API:

```
/app/node_modules/react lstat true false false
link .pnpm/react@18.3.1/node_modules/react
/app/node_modules/react/package.json lstat false false true
```

But the QuickJS C++ module normalizer misses the bare package:

```
quickjs-module normalize base=file:///app/dist/server/entry.mjs spec=./renderers.mjs -
> /app/dist/server/renderers.mjs
quickjs-module normalize-miss base=/app/dist/server/renderers.mjs spec=react
```

This points at the C++ resolver's `std::filesystem` checks across pnpm symlink path components inside a Wasmer-mounted directory. The runtime should resolve symlink components before checking candidate

package files, so `/app/node_modules/react/index.js` is tested through its real pnpm store path.

30.1.3 Plan

- Keep the fix in the QuickJS runtime module resolver.
- Add a small path helper that follows symlink components lexically before `TryResolveAsFile(...)` checks candidates.
- Preserve native behavior and the existing package exports condition order.
- Do not modify the Astro app, `node_modules`, or generated `dist` files.

30.1.4 Validation

Focused Wasmer checks:

```
cd /private/tmp/edge-wasmer-probe
wasmer run . -- -e "import('/app/dist/server/renderers.mjs').then(()=>console.log('renderers ok'))"
wasmer run --net . -- /app/dist/server/entry.mjs
```

Observed after the fix:

```
renderers ok
/app/node_modules/.pnpm/react@18.3.1/node_modules/react/index.js
[@astrojs/node] Server listening on http://127.0.0.1:3311
```

The targeted CJS peer dependency probe now resolves `react` from the real pnpm package path when the parent module is under `react-dom`, and the generated `dist/server/renderers.mjs` import succeeds under Wasmer. The standalone server advances past this issue and reaches the next route-rendering failure captured in `013_lucide_react_chevrondown_export.md`.

Chapter 31

Astro SSR: Lucide React ChevronDown Export

Source note: `development/troubleshooting/astro-ssr/013_lucide_react_chevrondown_export.md`

31.1 Astro SSR: Lucide React ChevronDown Export

		Remarks
Status	Done	Fixed by precise package "type": "module" detection.
Severity	High	The Astro standalone server starts, but rendering / fails before a response can be produced.

31.1.1 Issue

After the WASIX pnpm symlink resolution fix, the `stackmachine.com` Astro standalone server starts under Wasmer:

```
cd $sim$/src/dev/stackmachine.com
wasmer run --net .
```

Observed route-rendering failure:

```
[@astrojs/node] Server listening on http://localhost:4321
[ERROR] [router] Error while trying to render the route /
[ERROR] SyntaxError: Could not find export 'ChevronDown' in module
'/app/node_modules/.pnpm/lucide-react@0.462.0_react@18.3.1/node_'
```

31.1.2 Diagnosis

The generated Astro route imports named icons from `lucide-react`, including `ChevronDown`. The package exposes that name from its CommonJS entry:

```
lucide-react/dist/cjs/lucide-react.js
exports.ChevronDown = ChevronDown;
```

Focused checks showed that the package itself can be required through `createRequire(...)` and exposes thousands of named properties at runtime.

The failing path was:

```
/app/node_modules/.pnpm/lucide-react@0.462.0_react@18.3.1/node_modules/lucide-
react/dist/cjs/lucide-react.js
```

QuickJS incorrectly classified that `.js` file as ESM because `FileLooksCommonJs(...)` used a substring check against the nearest `package.json`: if the file contained both `"type"` and `"module"` anywhere, it treated the package as ESM. `lucide-react/package.json` has `"repository": { "type": "git" }` and a top-level `"module"` entry, but it does not have top-level `"type": "module"`.

That made QuickJS compile the CommonJS bundle as ESM. The static linker then could not see `exports.ChevronDown = ...`, so route linking failed before the CommonJS runtime body could execute.

31.1.3 Plan

- Reproduced the failure with a focused import of the generated layout chunk.
- Confirmed the resolver targets `lucide-react/dist/cjs/lucide-react.js`.
- Confirmed the CommonJS export scanner already handles `exports.ChevronDown = ...`; the bad behavior was earlier CJS/ESM classification.
- Moved package type detection to the shared parsed package metadata helper so only top-level `"type": "module"` affects `.js` classification.
- Kept the fix in the QuickJS runtime; no app, `node_modules`, or generated Astro output changes were needed.

31.1.4 Validation

Focused checks:

```
cd /private/tmp/edge-wasmer-probe
wasmer run . -- -e "import('/app/dist/server/chunks/Layout_BoMatPJA.mjs').then(()=>console.log('layout ok'))"

cd $sim$/src/dev/stackmachine.com
wasmer run --net .
```

Expected result: `lucide-react` named exports link successfully, and requesting `/` advances past the `ChevronDown` missing export.

Observed after the fix:

```
object
layout ok
200 text/html
```

Validation commands run:

```
cmake --build build-edge-quickjs-cli --target edge -j4
cd $sim$/src/dev/edgejs/quickjs-wasm/ && ./build.sh
wasmer run . -- -e "import { ChevronDown } from '/app/node_modules/.pnpm/lucide-react@0.462.0_react@18.3.1/"
wasmer run . -- -e "import('/app/dist/server/chunks/Layout_BoMatPJA.mjs').then(()=>console.log('layout ok'))"
wasmer run --net --env PORT=3311 --env HOST=127.0.0.1 .
```

The final app-level request to `http://127.0.0.1:3311/` returned `200 text/html`.

Chapter 32

Astro SSR: pnpm Deploy Externalized Runtime Links

Source note: `development/troubleshooting/astro-ssr/014_pnpm_deploy_externalized_runtime_links.md`

32.1 Astro SSR: pnpm Deploy Externalized Runtime Links

		Remarks
Status	Done	Fixed in the app deploy preparation step; later Wasmer deploy packaging work materialized the runtime package graph into a symlink-free <code>.deploy</code> artifact.
Severity	Medium	The full app works from the source tree, but the pruned <code>.deploy</code> artifact failed at startup without addressable runtime packages.

32.1.1 Issue

The `stackmachine.com` Astro SSR app can run under Wasmer from the full project tree, but the smaller `.deploy` directory prepared with `pnpm deploy --prod` failed immediately:

```
cd $sim$/src/dev/stackmachine.com
npm run edge:prepare-deploy
cd .deploy
wasmer run --net .
```

Observed failure:

```
file:///app/dist/server/entry.mjs:1
import { renderers } from './renderers.mjs';
```

ReferenceError: could not load module 'piccolore'

32.1.2 Diagnosis

The missing `piccolore` package exists inside the `pnpm` virtual store in the deploy artifact:

`.deploy/node_modules/.pnpm/piccolore@0.1.3/node_modules/piccolore`

However, `pnpm deploy --prod --legacy` only materializes top-level `node_modules` links for packages that are reachable as deployed package dependencies. Astro's generated server chunks still contain bare runtime imports for packages that were externalized into `dist/server`, including `piccolore`.

That means the runtime starts resolving from `/app/dist/server/...`, walks up to `/app/node_modules`, and expects a top-level link:

`/app/node_modules/piccolore`

The package was present in the virtual store but not addressable from the standard Node-style package lookup path. This differs from the earlier WASIX pnpm symlink issue: QuickJS symlink and package resolution were already working once a top-level package entry existed.

The first generated top-level `piccolore` link still failed because it pointed at Astro's dependency symlink:

```
.pnpm/astro@.../node_modules/piccolore
```

Rebuilding the link against the resolved package directory fixed that part:

```
.pnpm/piccolore@0.1.3/node_modules/piccolore
```

After the server started, the first route render exposed a second packaging issue: `graphql-ws` imported `graphql` at runtime, but `graphql` was listed only under app `devDependencies`. Since `pnpm deploy --prod --legacy` correctly omits dev dependencies, the deploy artifact did not include `graphql`.

32.1.3 Plan

- Keep the standard pnpm deploy flow so the artifact remains much smaller than the full project tree.
- Copy the Astro `dist`, `wasmer.toml`, optional `app.yaml`, and deployed production `node_modules` into `.deploy`.
- Scan `.deploy/dist/server` for bare `import`, `dynamic import(...)`, and `require(...)` specifiers.
- For each bare server-runtime package that is absent from `.deploy/node_modules`, find the matching package under the pnpm virtual store and make it addressable from the deploy root `node_modules`.
- Move packages needed by runtime peers into production dependencies so `pnpm deploy --prod` includes them normally.
- Treat this as app deploy packaging glue rather than an Edge QuickJS runtime change.

32.1.4 Resolution

`$\sim$/src/dev/stackmachine.com/scripts/prepare-edge-deploy.cjs` first fixed this local `.deploy` failure by building the Astro app, running `pnpm deploy --prod --legacy`, assembling `.deploy`, and making externalized runtime packages from the generated server bundle addressable at the deploy root `node_modules`.

`graphql` was moved from `devDependencies` to `dependencies` in `$\sim$/src/dev/stackmachine.com/package.json`, because it is imported at runtime through `stackmachine/graphql-ws`.

The first fixed `prepare` run added access to:

```
@oslojs/encoding, cookie, cssesc, destr, devalue, es-module-lexer,
html-escaper, mrmime, piccolo, send, server-destroy, sharp
```

That symlink-based local fix was later superseded by `../wasmer-deploy/001_pnpm_directory_symlinks_webc.md`, after `wasmer deploy` showed that the cloud package path can lose pnpm directory symlink contents. The current deploy preparation step builds a runtime package closure, prunes unimported packages, materializes package links as real directories, removes `.pnpm`, rewrites virtual-store source imports, and asserts the final artifact contains no symlinks or nested module stores.

The current deploy directory is:

```
347M    $\sim$/src/dev/stackmachine.com/.deploy
```

32.1.5 Validation

Prepare command:

```
cd $\sim$/src/dev/stackmachine.com
npm run edge:prepare-deploy
```

Runtime check:

```
cd $\sim$/src/dev/stackmachine.com/.deploy
wasmer run --net --env PORT=3311 --env HOST=127.0.0.1 .
curl -i http://127.0.0.1:3311/
```

Observed result:

```
[@astrojs/node] Server listening on http://127.0.0.1:3311
HTTP/1.1 200 OK
content-type: text/html
```

The generated HTML rendered the StackMachine landing page from the `.deploy` artifact.

The later flattened artifact also passed the Wasmer deploy packaging invariants: no sym-links, only one root `node_modules`, no `.pnpm` directory, and no source imports targeting `node_modules/.pnpm/.../node_modules/...`.

Part VI

Vite App Standalone Investigation

The Vite notes capture the standalone build boundary and the adapter/runtime choices for SPA-style deployments.

Chapter 33

Vite App Standalone Build Findings

Source note: `development/troubleshooting/vite-app/001_standalone_build.md`

33.1 Vite App Standalone Build Findings

		Remarks
Status	Done with caveats	Findings captured for the plain Vite standalone build shape. This is an architecture note; the current custom static server path remains viable.
Severity	Low	

33.1.1 Query

Can a plain Vite app use a standalone build shape like the Astro app, so the app does not provide its own `server/router.cjs` and the server entry is produced as part of the standard build?

33.1.2 Summary

Not in the same way as the Astro app by default, but `vite-plugin-standalone` is a relevant candidate if the Vite app has, or is given, a server entrypoint.

The Astro app can use this shape because it is configured for Astro server output with the Node adapter in standalone mode:

```
export default defineConfig({
  output: 'server',
  adapter: node({
    mode: 'standalone',
  }),
});
```

That build emits a server entry under `dist/server/entry.mjs`. The local EdgeJS compatibility step then bundles that emitted server entry into `dist/server/entry.cjs`, which Wasmer can execute through Edge QuickJS.

The Vite app is a plain SPA build. Its standard build is only:

```
vite build
```

That emits static client files under `dist`; it does not emit an HTTP server entrypoint equivalent to Astro's standalone `dist/server/entry.mjs`.

There is a third-party plugin, `vite-plugin-standalone`, whose README describes it as a Vite plugin that uses `@vercel/nft` and `esbuild` to create a standalone server build so deployment can copy only the build folder rather than `node_modules`. The plugin supports an explicit `entry` option, including multiple named entries:

```
standalone({
  entry: {
    index: './server/index.ts',
    worker: './server/worker.js',
  },
});
```

```
},
});
```

The plugin source applies only to SSR builds (`env.isSsrBuild`), then finds the Rollup bundle entries, rebundles them with esbuild for Node, and traces/copies native or external dependencies with `@vercel/nft`. In other words, it packages a server entry; it does not remove the need for server semantics to exist somewhere.

Its examples also point in that direction:

- the Rakkas example uses `standalone()` alongside the Rakkas Vite plugin;
- the Vike example uses a server entry at `./src/server/index.ts` and `viteNode({ entry: './src/server/index.ts', standalone: true })`;
- that Vike server entry creates an Express app and listens on `PORT`.

For this reason, if the Vite app remains a plain SPA, its `server/router.cjs` is currently real runtime plumbing, not incidental project code. It provides the HTTP/static adapter that Vite's SPA build does not generate:

- serve files from `/dist`;
- support GET and HEAD;
- reject path traversal;
- map directories to `index.html`;
- fall back to `/dist/index.html` for client-side routes;
- send explicit content types and cache headers.

33.1.3 Options For Next Work

1. Keep an app-owned `server/router.cjs`. This is the simplest shape and matches the current Vite static SPA adapter notes for Edge QuickJS WASIX.
2. Generate or bundle `server/router.cjs` during the app build. This can remove hand-maintained router code from the app, but it would still be an EdgeJS adapter/template rather than standard Vite output.
3. Try `vite-plugin-standalone` with an explicit minimal server entry. This may let the Vite app produce a build-contained server artifact, but it still needs compatibility testing under Edge QuickJS WASIX. The default plugin output is ESM and Node-targeted, and examples use Express/framework server code, so it may need an esbuild format/config adjustment or a small CJS wrapper for the current Edge QuickJS execution path.
4. Move the app to a framework/runtime that emits a standalone server adapter. Astro SSR, Rakkas, or Vike are examples. This is a larger migration and changes the app architecture.
5. Add a reusable EdgeJS static SPA adapter package or runtime entry. This is likely the cleanest longer-term direction: Vite apps could point at a shared adapter instead of carrying their own `server/router.cjs`.

33.1.4 Conclusion

It is possible to remove app-owned `server/router.cjs`, but not by flipping a Vite `standalone` option. Plain Vite does not currently provide an Astro-style standalone HTTP server entry as part of `vite build`.

`vite-plugin-standalone` is worth a focused experiment. The experiment should answer whether a minimal static SPA HTTP entry can be bundled into a build-owned artifact that Edge QuickJS WASIX can execute. If that works, the app can avoid carrying a bespoke router file. If it does not, the best next story is probably a reusable EdgeJS static SPA adapter so Vite apps can keep the standard `vite build` output while avoiding per-app router copies.

Part VII

Next.js Standalone Investigations

The Next.js notes track the standard standalone output, V8/serdes compatibility, unavailable inspector behavior, and the remaining route stack exhaustion blocker.

Chapter 34

Next App Standalone: require("v8") / Serdes Findings

Source note: development/troubleshooting/next-app/001_standalone_v8_serdes.md

34.1 Next App Standalone: require("v8") / Serdes Findings

		Remarks
Status	Done	QuickJS serdes constructors implemented and verified. The Next standalone server cannot start until the v8 serdes surface is handled.
Severity	High	

34.1.1 Context

The next-app project was switched to the standard Next.js standalone output:

- next.config.ts uses output: "standalone".
- The standalone build emits .next/standalone/server.js.
- Node can run the standalone server directly.
- The old custom server/router scripts are no longer part of the intended path.

The failing Edge QuickJS repro copied the standalone output into a test folder:

```
cp -rf ./next/standalone ./testing/app/  
cd ./testing  
$sim$/src/dev/edgejs/build-edge-quickjs-cli/edge ./app/server.js
```

The failure looked like:

```
undefined[Error: Failed to execute builtin 'internal/main/run_main_module':  
  at normalizeRequirableId (<input>:302:35)  
  at <anonymous> (<input>:1367:43)  
  ...  
]
```

Node.js v24.13.2

34.1.2 Reduction

The standalone server failure reduces to loading Next's normal startup module:

```
require("next/dist/server/lib/start-server")
```

Tracing module resolution showed the failing dependency was the public builtin:

```
require("v8")
```

The smaller reproducer is:

```
$\sim$/src/dev/edgejs/build-edge-quickjs-cli/edge /private/tmp/edge-repro-v8.js
```

with /private/tmp/edge-repro-v8.js containing:

```
require("v8");
```

34.1.3 LLDB Findings

Breaking at TakePendingExceptionInfo only showed Edge's final wrapped top-level exception:

```
Error: Failed to execute builtin 'internal/main/run_main_module'
```

Breaking earlier in DescribeAndClearPendingException(...), before the native main-builtin wrapper rewrites the error, revealed the original exception:

```
TypeError: cannot read property 'prototype' of undefined
```

The source location corresponds to lib/v8.js:

```
Serializer.prototype._getDataCloneError = Error;
```

So require("v8") is not failing because the v8 builtin is missing. It is failing because:

- lib/v8.js loads internalBinding("serdes").
- It destructures { Serializer, Deserializer } from that binding.
- On QuickJS, Serializer is currently undefined.
- The first access to Serializer.prototype throws.

34.1.4 Code Evidence

The V8 backend creates real serializer/deserializer constructors in:

```
napi/v8/src/unofficial_napi.cc
```

around unofficial_napi_create_serdes_binding(...), where it exports:

- Serializer
- Deserializer

The QuickJS backend currently returns an empty object:

```
napi/quickjs/src/unofficial_napi.cc
```

```
napi_status NAPI_CDECL unofficial_napi_create_serdes_binding(napi_env env,
                                                             napi_value *result_out)
{
    if (!CheckEnv(env) || result_out == nullptr)
        return napi_invalid_arg;
    return WrapOwned(env, JS_NewObject(Ctx(env)), result_out);
}
```

That empty binding explains the exact LLDB exception.

34.1.5 Impact On Standalone Next

Next's standalone server imports v8 from:

```
.next/standalone/node_modules/next/dist/server/lib/start-server.js
```

The observed usage is heap telemetry:

```
v8.getHeapStatistics()
```

So the app-level standalone build is valid and works under Node. The blocker is Edge QuickJS compatibility with Node's v8 builtin, specifically the incomplete QuickJS serdes internal binding.

34.1.6 Likely Runtime Fix

Implement a minimal QuickJS-backed `internalBinding("serdes")` that exports stable `Serializer` and `Deserializer` constructors.

For the immediate Next standalone path, it may be enough that `require("v8")` loads cleanly and `getHeapStatistics()` continues to work. Full `v8.serialize()` / `v8.deserialize()` behavior can be added using the existing QuickJS structured clone helpers based on `JS_WriteObject` / `JS_ReadObject`.

Candidate implementation area:

`napi/quickjs/src/unofficial_napi.cc`

Relevant existing helpers:

- `unofficial_napi_serialize_value(...)`
- `unofficial_napi_deserialize_value(...)`
- `JS_WriteObject(...)`
- `JS_ReadObject(...)`

34.1.7 Current Conclusion

The next-app standalone setup itself is correct. The failure under Edge QuickJS is caused by `require("v8")` loading `lib/v8.js`, which assumes `internalBinding("serdes").Serializer` exists. QuickJS currently returns an empty `serdes` binding, so the builtin throws before Next's standalone server can start.

34.1.8 Resolution

`napi/quickjs/src/unofficial_napi.cc` now exports QuickJS-backed `Serializer` and `Deserializer` constructors from `internalBinding("serdes")`. Native and WASIX smoke tests verified that `require("v8")` loads and that `v8.serialize()` / `v8.deserialize()` can round-trip a plain object.

Chapter 35

Next App Standalone: `require("inspector")` Stub

Source note: `development/troubleshooting/next-app/002_standalone_inspector_stub.md`

35.1 Next App Standalone: `require("inspector")` Stub

		Remarks
Status	Done	Unavailable-inspector stub implemented and verified. The Next standalone server cannot start while <code>require("inspector")</code> throws during module load.
Severity	High	

35.1.1 Context

After implementing a QuickJS-backed `internalBinding("serdes")`, the `private-poker` Next.js standalone server advanced past the previous `require("v8")` failure and reached a new startup error:

```
Failed to execute builtin 'internal/main/run_main_module':
undefinedError: Inspector is not available
    at NodeError (<input>:447:11)
    at anonymous (<input>:27:13)
```

The app is mounted through:

```
$\sim$/src/ImperfectProtocol/private-poker/wasmer.toml
```

which runs:

```
/app/server.js
```

from `.next/standalone`.

35.1.2 Reduction

Next 16 standalone output imports the public `inspector` builtin from startup logging and profiling helpers:

```
.next/standalone/node_modules/next/dist/server/lib/app-info-log.js
.next/standalone/node_modules/next/dist/server/lib/cpu-profile.js
```

The immediate startup path only needs `inspector.url()` to be callable and return `undefined` when no inspector is active.

35.1.3 Action Plan

1. Keep `internalBinding("config").hasInspector` false so the runtime does not claim real inspector support.
2. Change `lib/inspector.js` to export a conservative unavailable-inspector stub instead of throwing at module load when `hasInspector` is false.

3. Make passive APIs such as `url()`, `close()`, and `Network` hooks no-op.
4. Keep active inspector APIs such as `open()`, `waitForDebugger()`, and `Session.connect()` throwing inspector-specific errors.
5. Rebuild native QuickJS and WASIX, then rerun `private-poker` until it reaches the next runtime blocker or starts serving.

35.1.4 Resolution

`lib/inspector.js` now exports a conservative unavailable-inspector stub when `internalBinding("config").hasInspector` is false. Passive consumers can import the builtin and call `url()`, while active debugging APIs still report inspector unavailability or disconnected state.

Native and WASIX smoke tests verified that `require("inspector").url()` returns `undefined`. The `private-poker` Next standalone server then advanced past startup and reached the request-time stack exhaustion captured in [003_route_stack_exhausted.md](#).

Chapter 36

Next App Standalone: Route Request Stack Exhaustion

Source note: `development/troubleshooting/next-app/003_route_stack_exhausted.md`

36.1 Next App Standalone: Route Request Stack Exhaustion

		Remarks
Status	Active	Planned investigation after startup compatibility fixes.
Severity	High	The Next standalone server starts, but the first HTTP request crashes the Wasmer process.

36.1.1 Context

After fixing the QuickJS serdes binding and adding an unavailable-inspector stub, `private-poker` starts under Wasmer:

```
▲ Next.js 16.1.6
- Local:      http://localhost:3000
- Network:    http://0.0.0.0:3000
```

```
✓ Starting...
✓ Ready in 1303ms
```

A request to the root route then fails:

```
curl -i --max-time 10 http://127.0.0.1:3000/
```

with the client receiving:

```
curl: (52) Empty reply from server
```

and Wasmer reporting:

```
RuntimeError: call stack exhausted
```

36.1.2 Impact

This is past module-load compatibility and server startup. The remaining blocker is now request-time rendering or request dispatch under the Next standalone runtime.

36.1.3 Action Plan

1. Reproduce with the smallest route possible, starting with `/`, then known static asset paths, then any simple route if available.
2. Enable focused tracing if useful to find whether the exhaustion occurs in HTTP dispatch, module loading, React/Next rendering, or user app code.

3. Compare native QuickJS CLI behavior against WASIX to determine whether this is QuickJS JS stack, native/wasm stack, or request lifecycle recursion.
4. If it is a known stack-depth issue, adjust the narrow WASIX/QuickJS stack configuration rather than changing broad module semantics.
5. Rerun private-poker under Wasmer and verify at least one HTML route returns an HTTP response.

36.1.4 Native Versus Wasmer Samples

Two macOS sample captures from private-poker show different runtime shapes:

- `\sim$\src\dev\edgejs\next-server.txt`: native Edge QuickJS.
- `\sim$\src\dev\edgejs\wasmer-sample.txt`: wasmer run of the same app package.

The native sample is quiescent in the expected way. Its main thread is parked in `RunEventLoopUntilQuiescent(...)` → `uv_run(...)` → `uv__io_poll(...)` → `kevent`, and libuv worker threads are blocked in `uv_cond_wait(...)`. This matches a server that has no pending CPU work while waiting for socket activity.

The Wasmer sample is not just the same libuv loop behind a WASIX boundary. It shows Wasmer/WASIX task machinery on the hot path:

- `TokioTaskManager Thread Pool_thread_1`
- `stack_call_trampoline`
- `corosensei::coroutine::on_stack::wrapper`
- `wasmer_wasix::syscalls::wasix::sched_yield`
- `wasmer_wasix::syscalls::wasix::thread_sleep_internal`
- `wasmer_wasix::syscalls::handle_rewind_ext`
- a very deep repeated sequence of unknown generated wasm frames

Other TokioTaskManager pool threads are blocked in:

```
stack_call_trampoline
corosensei::coroutine::on_stack::wrapper
wasmer_wasix::syscalls::wasix::futex_wait
wasmer_wasix::syscalls::__asyncify_with_deep_sleep
virtual_mio::waker::block_on
parking::Inner::park
_pthread_cond_wait
```

This points away from QuickJS JavaScript promise continuations as the direct stack-growth mechanism. In our Edge QuickJS layer, `async/await` is represented as QuickJS promise jobs drained by `JS_ExecutePendingJob(...)` through `unofficial_napi_process_microtasks(...)`. The N-API backend also installs QuickJS promise hooks that capture, enter, and leave async context frames around promise reaction jobs.

The coroutine names in the Wasmer sample are from Wasmer/WASIX `asyncify` and task scheduling, not from our QuickJS promise-job implementation. The suspicious pattern is that a WASIX syscall path that should suspend/yield through `asyncify` appears to keep re-entering generated wasm frames until the host reports `RuntimeError: call stack exhausted`.

36.1.5 Stack Size Probe

Increasing Wasmer's runtime stack changes the failure mode but does not resolve the route:

- default stack: first request returns an empty reply and Wasmer reports `RuntimeError: call stack exhausted`;
- `--stack-size 4194304`: same `call stack exhausted` on the first request;
- `--stack-size 6291456`: same `call stack exhausted` on the first request;
- `--stack-size 8388608`: the server reaches `Ready`, but HTTP requests hang without returning bytes.

This suggests stack size is only exposing the next symptom. The likely blocker is a WASIX `coroutine/asyncify`

re-entry or wakeup issue around a syscall used by the Next request path, rather than a simple need for a larger QuickJS JS stack.

36.1.6 Updated Investigation Plan

1. Capture a focused Wasmer sample during the first hanging or overflowing HTTP request and map the generated wasm addresses if possible.
2. Reproduce outside Next with a small Edge QuickJS WASIX script that performs the likely syscall pattern: HTTP accept, read, response write, and any `setImmediate` / Promise continuation used by the request path.
3. Add tracing around `uv_run` callbacks and QuickJS microtask checkpoints to verify whether JavaScript continuations are re-entering normally or whether execution is stuck before returning from a WASIX syscall suspension.
4. If the small repro shows the same deep `corosensei` / `handle_rewind_ext` stack, treat this as a Wasmer/WASIX asyncify task scheduling issue rather than a QuickJS promise-hook issue.

Part VIII

Wasmer Deployment And WASIX Packaging

The Wasmer deployment chapters cover pnpm symlink packaging, QuickJS WASIX N-API import-module consistency, and CI safe-mode artifact handling.

Chapter 37

Wasmer Deploy: pnpm Directory Symlinks In WEBC Package

Source note: development/troubleshooting/wasmer-deploy/001_pnpm_directory_symlinks_webc.md

37.1 Wasmer Deploy: pnpm Directory Symlinks In WEBC Package

		Remarks
Status	Done with caveats	Deploy preparation now emits a symlink-free, flattened .deploy artifact and passes local wasmer run --net .; wasmer.io still needs a fresh deploy confirmation.
Severity	High	The app runs locally from .deploy, but the app deployed to wasmer.io exits during startup when bare react cannot resolve.

37.1.1 Issue

The prepared stackmachine.com deploy directory works locally:

```
cd $sim$/src/dev/stackmachine.com
npm run edge:prepare-deploy
cd .deploy
wasmer run --net .
```

The same app created by wasmer deploy on wasmer.io exits with:

```
file:///app/dist/server/entry.mjs:1
import { renderers } from './renderers.mjs';
```

```
ReferenceError: could not load module 'react'
Instance exited with code ExitCode::1
```

This means the cloud runtime reached the Astro server entry, then failed while resolving the bare react import from /app/dist/server/renderers.mjs.

37.1.2 Source Findings

Wasmer has two different deploy/package paths that matter here.

Remote autobuild ZIP creation lives in:

```
$sim$/src/dev/wasmer/lib/sdk/src/app/deploy_remote_build.rs
create_zip_archive(...) configures the ignore walker with standard_filters(true), .ignore(true),
.git_ignore(true), .git_exclude(true), .git_global(true), .require_git(true), and .follow_links(false). It also explicitly rejects symlinks:
```

cannot deploy projects containing symbolic links

So if `wasmer deploy` takes the remote autobuild ZIP path for a tree containing pnpm links, it should fail before runtime. This path also enables standard and git ignore filtering, so hidden paths and ignored paths can be filtered there.

The local package publish path is used when the app manifest points at a local package and a `wasmer.toml` is present. That path is in:

```
$\sim$/src/dev/wasmer/lib/cli/src/commands/app/deploy.rs
$\sim$/src/dev/wasmer/lib/sdk/src/package/publish.rs
$\sim$/src/dev/wasmer/lib/package/src/package/package.rs
```

The package walker uses `standard_filters(false)`, `.follow_links(false)`, `.parents(true)`, and `.add_custom_ignore_filename(".wasmerignore")`. Therefore this path does not skip hidden directories merely because their names begin with `..`. In the original pnpm-deployed artifact, the prepared app had no `.deploy/.wasmerignore`, `.gitignore`, or `.ignore`, and `.deploy/node_modules/.pnpm` existed locally.

The important remaining behavior is symlink handling. Package volumes are created through:

```
$\sim$/src/dev/wasmer/lib/package/src/package/volume/fs.rs
```

That calls `WEBC's Directory::from_path_with_walker(...)` with the same walker. The WEBC crate code inspected locally is:

```
/Users/sadhbh/.cargo/registry/src/index.crates.io-1949cf8c6b5b557f/webc-
11.0.0/src/from_path_with_walker.rs
```

Because the walker has `follow_links(false)`, it sees pnpm top-level package links but does not descend into linked directories. WEBC then classifies entries from the path shape. A directory symlink such as:

```
/app/node_modules/react -> .pnpm/react@18.3.1/node_modules/react
```

can become a directory entry without the package contents that live behind the symlink.

37.1.3 Diagnosis

This is not the same as the earlier WASIX local filesystem symlink issue. Local `wasmer run --net .` sees the host `.deploy` tree and follows the real pnpm symlinks, so QuickJS can resolve:

```
/app/node_modules/react/package.json
```

The cloud package is serialized first. If the WEBC package contains `/app/node_modules/react` as an empty directory, a non-followed symlink entry, or otherwise not as the real React package directory, QuickJS correctly fails Node-style bare package resolution and reports:

```
ReferenceError: could not load module 'react'
```

The hidden `.pnpm` directory is less likely to be the primary issue for the local package publish path, because that path disables standard filters. The top-level `node_modules/react` directory symlink is the sharper suspect.

37.1.4 Action Plan

- Keep the existing `.deploy` preparation flow as the starting point.
- Copy deploy manifests that Wasmer may need at upload time, including `wasmer.toml` and optional `app.yaml`.
- Build a runtime package closure from literal imports in `.deploy/dist/server`, then recursively scan included package source files for literal `import(...)`, `static import/export ... from`, and `require(...)` specifiers.
- Prune top-level packages outside that import closure.
- Materialize the remaining pnpm package links as real directories, skipping nested `node_modules` directories so every module exists only at the root `.deploy/node_modules` level.
- Remove pnpm runtime scaffolding (`.pnpm` and `.bin`) from the final artifact.
- Rewrite source references from:

```
node_modules/.pnpm/<store-entry>/node_modules/<package>
```

to:

`node_modules/<package>`

- Fail deploy preparation if any of these invariants are violated: no symlinks, no nested `node_modules`, no `.pnpm` directory, and no remaining source imports targeting pnpm virtual-store paths.
- Verify the deploy artifact before upload:

```
cd $sim$/src/dev/stackmachine.com/.deploy
find . -type l -print
find . -type d -name node_modules -print
find . -type d -name .pnpm -print
rg -n 'node_modules/\.pnpm/\.+/node_modules/' .
```

37.1.5 Resolution

`$sim$/src/dev/stackmachine.com/scripts/prepare-edge-deploy.cjs` now creates a flattened deploy artifact:

- builds Astro and runs `pnpm deploy --prod --legacy` into a temporary `.deploy-pnpm`;
- copies `dist`, `wasmer.toml`, optional `app.yaml`, `package.json`, and the deployed `node_modules`;
- links missing server-runtime bare imports from the pnpm virtual store;
- expands the runtime import closure by scanning included source files;
- prunes packages outside that closure;
- materializes the remaining package links as real directories;
- removes `.pnpm` and `.bin`;
- rewrites source references to materialized `node_modules/<package>` paths;
- asserts the final artifact has a single root `node_modules` tree and no symlinks.

The fresh local artifact now reports:

```
Runtime package closure: 488 package(s)
Pruned 34 unimported top-level packages
Materialized 488 package links
347M    $sim$/src/dev/stackmachine.com/.deploy
```

37.1.6 Validation

Fresh prepare:

```
cd $sim$/src/dev/stackmachine.com
npm run edge:prepare-deploy
```

Final artifact checks:

```
find $sim$/src/dev/stackmachine.com/.deploy -type l -print
find $sim$/src/dev/stackmachine.com/.deploy -type d -name node_modules -print
find $sim$/src/dev/stackmachine.com/.deploy -type d -name .pnpm -print
rg -n 'node_modules/\.pnpm/\.+/node_modules/' $sim$/src/dev/stackmachine.com/.deploy
```

Observed:

- no symlinks;
- only `$sim$/src/dev/stackmachine.com/.deploy/node_modules`;
- no `.pnpm` directory;
- no pnpm virtual-store import paths;
- `$sim$/src/dev/stackmachine.com/.deploy/app.yaml` exists when the source app has `app.yaml`, and matches the source file.

Runtime check:

```
cd $sim$/src/dev/stackmachine.com/.deploy
wasmer run --net .
curl -m 30 -s -o /tmp/stackmachine-deploy-check.html \
  -w '%{http_code} %{content_type} %{size_download}\n' \
  http://127.0.0.1:4321/
```

Observed:

```
[@astrojs/node] Server listening on http://localhost:4321
200 text/html 167960
```

37.1.7 Open Questions

- Whether Wasmer Cloud was reached through local package publish or remote autobuild for the reported run. The runtime failure suggests the local package path, because the remote ZIP path should reject symlinks during archive creation.
- Whether Wasmer should preserve directory symlinks in WEBC packages, reject them like the remote ZIP path, or document that publish inputs must be symlink-free.

Chapter 38

Wasmer Deploy: QuickJS WASIX N-API Import Module Mismatch

Source note: [development/troubleshooting/wasmer-deploy/002_quickjs_wasix_napi_import_module_mismatch.md](#)

38.1 Wasmer Deploy: QuickJS WASIX N-API Import Module Mismatch

		Remarks
Status	Done	Fixed by applying the embedded QuickJS N-API declaration mode to <code>edge_environment_core</code> ; <code>quickjs-wasm/build.sh</code> now completes and the final no-N-API-imports check passes. Blocks the QuickJS WASIX artifact from linking, so no deployable <code>edgejs.wasm</code> can be produced.
Severity	High	

38.1.1 Issue

`$\sim$/src/dev/edgejs/quickjs-wasm/build.sh` reached the final WASM link and failed with `wasm-ld` import module mismatches for several `napi_*` symbols:

```
wasm-ld: error: import module mismatch for symbol: napi_get_reference_value
>>> defined as env in libedge_runtime.a(edge_runtime.cc.obj)
>>> defined as napi in libedge_environment_core.a(edge_environment.cc.obj)
```

The same pattern appeared for `napi_delete_reference`, `napi_create_reference`, `napi_is_exception_pending`, `napi_get_global`, `napi_get_named_property`, `napi_create_string_utf8`, `napi_call_function`, `napi_strict_equals`, `napi_create_array`, `napi_set_element`, and `napi_add_env_cleanup_hook`.

38.1.2 Diagnosis

The QuickJS WASIX build uses `EDGE_NAPI_PROVIDER=quickjs`, so the final `wasm` should link against the embedded QuickJS N-API implementation and should not import `napi_*`, `node_api_*`, or `unofficial_napi_*` symbols. The `napi_quickjs` target exposes `NAPI_EXTERN=` to prevent the common N-API header from annotating declarations as WASM imports.

`edge_runtime` already linked `napi_quickjs`, but `edge_environment_core` includes N-API headers and only linked `uv_a`. Under `__wasm__`, the default `NAPI_EXTERN` macro annotates declarations with `__import_module__("napi")`. That made `edge_environment_core` object files disagree with other QuickJS Edge objects that referenced the same unresolved N-API calls through the default `env` import module. `wasm-ld` rejects the same symbol being imported from two different modules.

38.1.3 Fix

For `EDGE_NAPI_PROVIDER=quickjs`, `edge_environment_core` now compiles with:

```
EDGE_EMBEDDED_NAPI_PROVIDER=1  
NAPI_EXTERN=
```

This keeps N-API declarations consistent with the embedded QuickJS provider without changing the shared common N-API headers.

38.1.4 Verification

The fixed build was verified with:

```
cd $sim$/src/dev/edgejs/quickjs-wasm  
./build.sh
```

Result:

Built QuickJS WASIX targets at \$sim\$/src/dev/edgejs/build-quickjs-wasix/edge.wasm and \$sim\$/src/dev/edgejs/build-quickjs-wasix/edgejs.wasm

The script also completed its final no-N-API-imports check.

38.1.5 Follow-Up Rule

When adding new Edge static libraries or object targets that include N-API headers and participate in embedded QuickJS WASIX linking, make sure the target inherits or explicitly defines `NAPI_EXTERN=` for the QuickJS provider.

Chapter 39

Wasmer Deploy: CI safe-mode missing QuickJS artifact

Source note: [development/troubleshooting/wasmer-deploy/003_ci_safe_mode_missing_quickjs_artifact.md](#)

39.1 Wasmer Deploy: CI safe-mode missing QuickJS artifact

		Remarks
Status	Done with caveats	CI wiring fixed locally; full WASIX execution still needs the GitHub Actions toolchain.
Severity	High	Blocks the WASIX Linux job before the safe-mode smoke tests can start.

39.1.1 Issue

The `build-wasix-linux` workflow builds the legacy WASIX artifact through:

```
make build-wasix
```

That writes:

```
build-wasix/edgejs.wasm
```

The root `wasmer.toml` now describes the embedded QuickJS package and points at:

```
build-quickjs-wasix/edgejs.wasm
```

The safe-mode smoke test therefore fails before executing JavaScript:

```
Unable to read the "edge" module's file from "./build-quickjs-wasix/edgejs.wasm"
No such file or directory
```

39.1.2 Action Plan

1. Add a Makefile target for the current QuickJS WASIX build script so CI does not need to know the script path directly.
2. Update the `build-wasix-linux` workflow to build the embedded QuickJS WASIX artifact before running `wasmer run ..`
3. Update WASIX dist packaging so `BUILD_DIR=build-quickjs-wasix` is treated as a wasm distribution directory, not as a native Edge build.
4. Remove or bypass the legacy `napi_wasmer` smoke path from this job because it belongs to `EDGE_NAPI_PROVIDER=imports`, while this package now embeds the QuickJS provider.
5. Verify the edited Makefile targets and safe-mode test script syntax locally; full WASIX execution still requires the CI Wasmer/WASIX toolchain.

39.1.3 Fix

The Makefile now exposes:

```
make build-quickjs-wasix
```

That target runs `quickjs-wasm/build.sh`, producing the `build-quickjs-wasix/edgejs.wasm` artifact referenced by the active root manifest.

The `build-wasix-linux` workflow now builds the QuickJS WASIX artifact before the safe-mode smoke test and packages `BUILD_DIR=build-quickjs-wasix` for the WASIX zip. The legacy `napi_wasmer` smoke path was removed from this job because it tests the host-import N-API artifact, not the embedded QuickJS package.

`make dist-only` now treats both `build-wasix` and `build-quickjs-wasix` as WASIX distribution directories, so it copies `edgejs.wasm`, `wasmer.toml`, and certificates instead of looking for native `edge` and `edgeenv` binaries.

39.1.4 Verification

Local structural checks passed:

```
make -n build-quickjs-wasix
make -n dist-only BUILD_DIR=build-quickjs-wasix ZIP_NAME=edge-wasix.zip
python3 -m py_compile scripts/test-wasix-safe-mode.py
ruby -e "require 'yaml'; YAML.load_file('.github/workflows/test-and-build.yml')"
git diff --check
```

The full `make build-quickjs-wasix` and `make test-wasix-safe-mode` flow was not run locally because it depends on the WASIX/Wasmer CI toolchain.

39.1.5 Follow-Up: Native Linux Job

The native `build-linux` job still built the V8 N-API test target and default V8-backed Edge binary. It now follows the same provider direction as the WASIX job:

```
make build-napi-quickjs CMAKE_BUILD_TYPE=Release JOBS=4
make test-napi-quickjs-only TEST_JOBS=4
make build-edge-quickjs-cli CMAKE_BUILD_TYPE=Release JOBS=4
make dist-only BUILD_DIR=build-edge-quickjs-cli JOBS=4 ZIP_NAME=edge-linux-amd64.zip
make test-only BUILD_DIR=build-edge-quickjs-cli TEST_JOBS=4
```

`make build-edge-quickjs-cli` now builds both the `edge` and `edgeenv` targets so native dist packaging has the executables expected by `dist-only`.